

시험장 환경 정보

Python: 3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 20:34:20) [MSC v.1916 64 bit (AMD64)]

|모듈|버전| |----|----| |pandas|0.25.1| |numpy|1.18.5| |sklearn|0.21.3| |scipy|1.5.2| |mlxtend|0.15.0.0| |statsmodels|0.11.1| |xgboost|0.80|

강사: 멀티캠퍼스 강선구(sunku0316.kang@multicampus.com, sun9sun9@gmail.com)

```
In [1]: # 리스트 언패킹(list unpacking)
...
리스트 언패킹(list unpacking)은 파이썬에서 리스트나 다른 시퀀스의 요소들을
개별 변수에 할당하는 기법입니다. 이를 통해 시퀀스의 요소들을 쉽게 분리하여 사용할 수 있습니다.
리스트 언패킹은 `*` 연산자를 사용하여 가변 길이의 시퀀스를 처리할 수도 있습니다.

### 기본 리스트 언패킹
기본적인 리스트 언패킹은 리스트의 각 요소를 개별 변수에 할당하는 방식입니다.

# 예시 리스트
numbers = [1, 2, 3]

# 리스트 언패킹
a, b, c = numbers

print(a) # 출력: 1
print(b) # 출력: 2
print(c) # 출력: 3

위 코드에서는 리스트 `numbers`의 세 요소가 각각 변수 `a`, `b`, `c`에 할당됩니다.

### 변수 수와 리스트 요소 수가 맞지 않을 때 만약 변수의 수와 리스트의 요소 수가 맞지 않으면 에러가 발생합니다.

# 예시 리스트
numbers = [1, 2, 3, 4]

# 언패킹 변수의 수가 맞지 않으면 에러 발생
a, b, c = numbers # ValueError: too many values to unpack (expected 3)
```

가변 길이 언패킹 (* 연산자 사용)
리스트의 요소 수가 일정하지 않을 때는 * 연산자를 사용하여 가변 길이의 요소를 하나의 변수에 할당할 수 있습니다.

```
# 예시 리스트
numbers = [1, 2, 3, 4, 5]
```

```
# 가변 길이 언패킹
a, *b, c = numbers
```

```
print(a) # 출력: 1
print(b) # 출력: [2, 3, 4]
print(c) # 출력: 5
```

위 코드에서 a와 c는 리스트의 첫 번째와 마지막 요소를 받으며, *b는 나머지 요소들을 리스트로 받습니다.

함수 인자에서의 리스트 언패킹
리스트 언패킹은 함수 호출 시에도 유용하게 사용할 수 있습니다.

```
# 예시 함수
def add(x, y, z):
    return x + y + z
```

```
# 예시 리스트
numbers = [1, 2, 3]
```

```
# 리스트 언패킹을 사용하여 함수 호출
result = add(*numbers)
```

```
print(result) # 출력: 6
```

위 코드에서는 리스트 numbers가 *numbers 형태로 함수 add의 인자로 전달되어 언패킹됩니다.

중첩 리스트 언패킹
리스트 언패킹은 중첩된 리스트에서도 사용할 수 있습니다.

```
# 예시 중첩 리스트
nested_list = [1, [2, 3], 4]
```

```
# 중첩 리스트 언패킹
a, (b, c), d = nested_list
```

```
print(a) # 출력: 1
print(b) # 출력: 2
print(c) # 출력: 3
print(d) # 출력: 4
```

위 코드에서 중첩된 리스트의 요소 `(b, c)`를 각각 변수 `b`와 `c`에 할당합니다.

결론

리스트 언패킹은 파이썬의 강력한 기능 중 하나로, 시퀀스의 요소들을 쉽게 변수에 할당하거나 함수 인자로 전달할 수 있게 합니다. 다양한 상황에서 리스트 언패킹을 사용하면 코드의 가독성과 효율성을 높일 수 있습니다.

```
print('')
```

In [2]: `#딕셔너리 언패킹(dictionary unpacking)`

```
...
```

딕셔너리 언패킹(dictionary unpacking)은 파이썬에서 딕셔너리의 키-값 쌍을 개별 인수로 분해하여 함수에 전달하거나 다른 딕셔너리에 병합하는 기법입니다. 딕셔너리 언패킹은 `\*\*` 연산자를 사용하여 수행됩니다.

딕셔너리 언패킹 예제

함수 인자에서 딕셔너리 언패킹
딕셔너리 언패킹은 함수에 인수를 전달할 때 유용합니다.

```
def greet(name, age):
    print(f"Hello, my name is {name} and I am {age} years old.")
```

```
# 딕셔너리 정의
person = {'name': 'Alice', 'age': 30}
```

```
# 딕셔너리 언패킹을 사용하여 함수 호출
greet(**person)
```

위 코드에서 `person` 딕셔너리는 `\*\*person` 형태로 함수 `greet`에 전달되며, 이는 `name='Alice'`, `age=30`과 동일합니다. 출력 결과는 다음과 같습니다:
Hello, my name is Alice and I am 30 years old.

딕셔너리 병합
딕셔너리 언패킹을 사용하여 두 개 이상의 딕셔너리를 병합할 수 있습니다.

```
# 두 개의 딕셔너리 정의
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 3, 'c': 4}

# 딕셔너리 언패킹을 사용하여 병합
merged_dict = {**dict1, **dict2}
print(merged_dict)
```

위 코드에서 `dict1`과 `dict2`는 언패킹되어 새로운 딕셔너리 `merged\_dict`로 병합됩니다.
병합된 딕셔너리는 다음과 같습니다:

```
{'a': 1, 'b': 3, 'c': 4}
```

`dict2`의 값이 `dict1`의 값을 덮어씁니다.

가변 길이 인수를 사용하는 함수에서 딕셔너리 언패킹
가변 길이 인수를 사용하는 함수에서 딕셔너리 언패킹을 사용할 수 있습니다.

```
def print_args(*args, **kwargs):
    print("Positional arguments:", args)
    print("Keyword arguments:", kwargs)
```

딕셔너리 정의

```
kwargs = {'arg1': 'value1', 'arg2': 'value2'}
```

딕셔너리 언패킹을 사용하여 함수 호출

```
print_args(1, 2, 3, **kwargs)
```

위 코드에서 `print\_args` 함수는 위치 인수와 키워드 인수를 모두 받습니다.
딕셔너리 `kwargs`는 `\*\*kwargs` 형태로 전달됩니다. 출력 결과는 다음과 같습니다:

```
Positional arguments: (1, 2, 3)
Keyword arguments: {'arg1': 'value1', 'arg2': 'value2'}
```

딕셔너리 언패킹의 다양한 사용 예

클래스 생성자에서 딕셔너리 언패킹
클래스 생성자에서 딕셔너리 언패킹을 사용하여 객체를 초기화할 수 있습니다.

```
class Person:
    def __init__(self, name, age, city):
        self.name = name
        self.age = age
        self.city = city
```

```
# 딕셔너리 정의
person_info = {'name': 'Bob', 'age': 25, 'city': 'New York'}

# 딕셔너리 언패킹을 사용하여 객체 생성
person = Person(**person_info)

print(person.name) # 출력: Bob
print(person.age)  # 출력: 25
print(person.city) # 출력: New York

#### 딕셔너리의 일부 키-값 쌍 추출
딕셔너리 언패킹을 사용하여 딕셔너리의 일부 키-값 쌍만 추출할 수 있습니다.

# 딕셔너리 정의
data = {'a': 1, 'b': 2, 'c': 3, 'd': 4}

# 일부 키-값 쌍을 추출
{'a': a_value, 'b': b_value, **remaining} = data

print(a_value)      # 출력: 1
print(b_value)      # 출력: 2
print(remaining)    # 출력: {'c': 3, 'd': 4}

이 예에서는 `data` 딕셔너리의 키 `a`와 `b`의 값이 각각 `a_value`와 `b_value`에 할당되고, 나머지 키-값 쌍은 `remaining`에 할당됩니다.

#### 결론
딕셔너리 언패킹은 함수 인자 전달, 딕셔너리 병합, 클래스 초기화 등 다양한 상황에서 유용하게 사용할 수 있습니다.
`**` 연산자를 사용하여 딕셔너리의 키-값 쌍을 분해하고 쉽게 조작할 수 있는 강력한 기능을 제공합니다.
...
print('')
```

```
In [3]: '''
3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 20:34:20) [MSC v.1916 64 bit (AMD64)]
pandas 0.25.1
numpy 1.18.5
sklearn 0.21.3
scipy 1.5.2
statsmodels 0.11.1
'''
```

```
# 실행 환경 확인
```

```
import pandas as pd
import numpy as np
import sklearn
import scipy
import statsmodels
# import mlxtend
import sys
# import xgboost as xgb

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image

print(sys.version)
for i in [pd, np, sklearn, scipy, statsmodels]:
    print(i.__name__, i.__version__)
```

```
3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 20:34:20) [MSC v.1916 64 bit (AMD64)]
pandas 0.25.1
numpy 1.18.5
sklearn 0.21.3
scipy 1.5.2
statsmodels 0.11.1
```

In [161...

```
# 시각화 모듈 설정
# 참고용 차트를 출력하기 위함
import matplotlib.pyplot as plt
import seaborn as sns

import matplotlib as mpl
import matplotlib.font_manager as fm
plt.rc('font', family='Malgun Gothic')
mpl.rcParams['axes.unicode_minus'] = False

Image('1-1.png')
```

Out[161]: **문제 개요**

삼성전자 A공장은 전기료 효율화를 통한 원가절감 방안을 찾고 있다.

공장 가동 시 일시적인 과부하로 최대수요전력이 증가하면 불리한 기본요금이 산정되고 이는 원가상승 요인이 된다.

전기 요금은 최대수요전력을 바탕으로 시간 단위로 산정된다.

최대수요전력을 예측하여 선제적으로 원가상승에 대응하고자 한다.

당일 데이터를 바탕으로, 그 다음 날의 최대수요전력을 예측하는 모델을 개발한다.

보유한 데이터는 A공장에서 2021년 1월 1일 ~ 2021년 9월 14일 (9개월) 동안 변압기에 감지센서를 부착하여 15분 단위로 최대수요전력을 관측한 데이터이다.

데이터는 2종류이며, 'elec.csv' 데이터는 최대수요전력을 측정한 데이터이며,

'info.csv' 데이터는 측정 당시 기상정보 및 생산량과 인건비율 등의 정보가 포함되어 있다.

- 데이터 파일: elec.csv (24,672 rows, 4 columns, 데이터셋명-elec)

변수명	내용	타입
Date	측정 날짜	문자열
DateHour	측정 날짜와 시간	문자열
Minute	15분 간격으로 측정 (15분, 30분, 45분, 60분)	문자열
Value	최대수요전력(피크전기 사용량) 최대수요전력이란 공장 내 설치된 변압기를 통해 일정 시간 단위로 누적 계산되는 전력을 의미	정수형

- 데이터 파일: info.csv (6,168 rows, 9 columns, 데이터셋명-info)

변수명	내용	타입
Date	측정 날짜와 시간	문자열
생산량	해당 시점에 생산해야 할 생산량	정수형

기온	해당 시점의 기온	실수형
풍속	해당 시점의 풍속	실수형
습도	해당 시점의 습도	정수형
강수량	해당 시점의 강수량	실수형
전기요금(계절)	해당시점의 전기요금	실수형
공장인원	해당 시점에 공장이 보유한 생산력	실수형
인건비	야간 및 주간 인건비에 대한 비율값	실수형

In [162]: `Image('1-2.png')`

Out[162]: **전처리(Preprocessing)**

데이터를 분석하기 위하여 다음에 안내된 순서대로 데이터를 가공하시오.

단계 1

데이터셋 elec를 이용하여 다음 그림과 같이 재구성하시오.

- 최대수요전력 (Value)을 시간당 15분 간격으로 관측하여 얻은 4번의 결과
- DateHour 변수 기준으로 시간당 한 행에 4번 측정결과가 나오도록 재구성함.
- 재구성된 데이터셋은 elec1로 표기(6,168 rows, 5 columns)

DateHour	15분	30분	45분	60분
2021-01-01 00:00:00	62	61	61	61
2021-01-01 01:00:00	96	93	116	113
2021-01-01 02:00:00	106	96	106	107
2021-01-01 03:00:00	92	110	110	109
2021-01-01 04:00:00	108	105	106	108


```
In [5]: '''
코드는 CSV 파일을 읽어오는 과정에서 특정 열들을 날짜 형식으로 파싱하는 작업을 수행합니다.
### 코드 설명

- `df_elec = pd.read_csv('elec.csv', parse_dates=['Date', 'DateHour'])`
- `parse_dates` 인자는 리스트로, 여기서는 `['Date', 'DateHour']`로 지정되었습니다.
  이 인자는 `pandas`에게 해당 열들을 날짜 형식으로 파싱(parsing)하라는 지시를 합니다.

-. 날짜 형식 파싱:
  - `parse_dates=['Date', 'DateHour']` 인자에 의해 `Date`와 `DateHour` 열의 데이터는
    자동으로 날짜 형식(`datetime`)으로 변환됩니다.
  - CSV 파일의 `Date`와 `DateHour` 열에 있는 문자열이 날짜 형식으로 변환되어,
    `pandas` 데이터프레임 내에서 날짜/시간 연산을 보다 쉽게 할 수 있게 됩니다.

### 예시
예를 들어, `elec.csv` 파일에 다음과 같은 데이터가 있다고 가정합니다.

      Date      DateHour  Value
2024-01-01,2024-01-01 00:00:00,100

2024-01-01,2024-01-01 01:00:00,110

2024-01-02,2024-01-02 00:00:00,120

이 데이터를 `pd.read_csv`로 읽어오면, `Date`와 `DateHour` 열의 데이터는 문자열이 아니라 `datetime` 객체로 변환됩니다.
이로 인해 데이터프레임은 다음과 같이 나타낼 수 있습니다:

      Date      DateHour  Value
0 2024-01-01 2024-01-01 00:00:00    100

1 2024-01-01 2024-01-01 01:00:00    110

2 2024-01-02 2024-01-02 00:00:00    120

이 데이터프레임에서 `Date`와 `DateHour` 열은 이제 `datetime` 객체로 처리되며, 날짜 관련 연산이나 필터링을 쉽게 수행할 수 있습니다.
'''
print('')
```

```
In [6]: # 일자형 불러오기
# read_csv의 parse_dates를 이용하여 datetime 형식으로 불러 옵니다.
```

```
df_elec = pd.read_csv('elec.csv', parse_dates=['Date', 'DateHour'])
display(df_elec.info())
df_elec.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 24672 entries, 0 to 24671
Data columns (total 4 columns):
Date          24672 non-null datetime64[ns]
DateHour      24672 non-null datetime64[ns]
Minute        24672 non-null object
Value         24672 non-null int64
dtypes: datetime64[ns](2), int64(1), object(1)
memory usage: 771.1+ KB
None
```

```
Out[6]:
```

	Date	DateHour	Minute	Value
0	2021-01-01 00:15:00	2021-01-01 00:00:00	15분	62
1	2021-01-01 00:30:00	2021-01-01 00:00:00	30분	61
2	2021-01-01 00:45:00	2021-01-01 00:00:00	45분	61
3	2021-01-01 01:00:00	2021-01-01 00:00:00	60분	61
4	2021-01-01 01:15:00	2021-01-01 01:00:00	15분	96

```
In [7]: pd.crosstab(index=df_elec['DateHour'], columns=df_elec['Minute'])
```

Out[7]:

	Minute	15분	30분	45분	60분
DateHour					
2021-01-01 00:00:00	1	1	1	1	1
2021-01-01 01:00:00	1	1	1	1	1
2021-01-01 02:00:00	1	1	1	1	1
2021-01-01 03:00:00	1	1	1	1	1
2021-01-01 04:00:00	1	1	1	1	1
...	...	...	...	...	...
2021-09-14 19:00:00	1	1	1	1	1
2021-09-14 20:00:00	1	1	1	1	1
2021-09-14 21:00:00	1	1	1	1	1
2021-09-14 22:00:00	1	1	1	1	1
2021-09-14 23:00:00	1	1	1	1	1

6168 rows × 4 columns

```
In [8]: (pd.crosstab(index=df_elec['DateHour'], columns=df_elec['Minute']) <= 1)
```

Out[8]:

	Minute	15분	30분	45분	60분
DateHour					
2021-01-01 00:00:00	True	True	True	True	True
2021-01-01 01:00:00	True	True	True	True	True
2021-01-01 02:00:00	True	True	True	True	True
2021-01-01 03:00:00	True	True	True	True	True
2021-01-01 04:00:00	True	True	True	True	True
...	...	...	...	...	...
2021-09-14 19:00:00	True	True	True	True	True
2021-09-14 20:00:00	True	True	True	True	True
2021-09-14 21:00:00	True	True	True	True	True
2021-09-14 22:00:00	True	True	True	True	True
2021-09-14 23:00:00	True	True	True	True	True

6168 rows × 4 columns

```
In [9]: (pd.crosstab(index=df_elec['DateHour'], columns=df_elec['Minute']) <= 1).all()
```

```
Out[9]: Minute
15분      True
30분      True
45분      True
60분      True
dtype: bool
```

```
In [10]: '''
주어진 코드는 `pandas` 라이브러리를 사용하여 데이터프레임의 특정 열에 대해 교차표를 생성하고,
이 교차표의 값이 특정 조건을 만족하는지 확인하는 작업을 수행합니다.

### 코드 설명

1. 교차표 생성:
- `pd.crosstab` 함수는 교차표(피벗 테이블)를 생성하는 함수입니다.
```

- `index` 인자는 교차표의 행을 지정하며, 여기서는 `df\_elec['DateHour']`을 사용합니다.
- `columns` 인자는 교차표의 열을 지정하며, 여기서는 `df\_elec['Minute']`을 사용합니다.
- 결과적으로, `DateHour`와 `Minute`의 조합에 대한 빈도수를 계산한 표를 만듭니다.

2. 조건 비교:

- 교차표에서 각 값이 1 이하인지 (`<= 1`)를 확인합니다.
- 이 조건은 교차표의 모든 셀에 대해 True 또는 False 값을 가진 동일한 크기의 DataFrame을 반환합니다.

3. 조건 만족 여부 확인:

- 첫 번째 `all()` 메서드는 각 열(column)별로 모든 값이 True인지 확인합니다. 결과는 각 열에 대한 Boolean 값입니다.
- 두 번째 `all()` 메서드는 각 열에 대한 Boolean 값이 모두 True인지 확인합니다.
- 결과적으로, 모든 행과 열의 값이 1 이하인지 확인합니다.

최종 결과

```
'''
전체 과정은 교차표의 모든 값이 1 이하인지 확인하여, 조건이 만족되면 `True`, 그렇지 않으면 `False`를 반환합니다.
'''
print('')
```

```
In [11]: # DateHour, Minute에 대응하는 값이 하나만 있는지 확인하고,
# pivot을 쓸 수 있을지 확인합니다.
(pd.crosstab(index=df_elec['DateHour'],
             columns=df_elec['Minute']) <= 1).all().all()
```

Out[11]: True

```
In [12]: '''
`pivot()` 함수와 `pivot_table()` 함수는 둘 다 pandas 라이브러리에서 데이터 프레임의 형태를 변환하는 데 사용되지만,
기능과 유연성에서 차이가 있습니다.
```

pivot() 함수

`pivot()` 함수는 데이터 프레임을 재구성하여 새로 만든 행과 열 인덱스를 지정하는 데 사용됩니다.
주로 하나의 값 열만 처리할 수 있습니다.

특징

- 간단한 변환: `pivot()` 함수는 단순히 행, 열, 값을 지정하여 데이터를 재구성합니다.
- 고유한 인덱스 값: 행, 열 인덱스 조합이 고유해야 합니다.
즉, 동일한 행/열 인덱스 조합이 여러 개 있을 경우 `pivot()` 함수는 오류를 발생시킵니다.
- 매개변수:

- ``index``: 새로운 데이터 프레임에서 행을 나타낼 열 이름(들)
- ``columns``: 새로운 데이터 프레임에서 열을 나타낼 열 이름(들)
- ``values``: 새로운 데이터 프레임에서 값을 채울 열 이름

예제

```
import pandas as pd
```

```
df = pd.DataFrame({
    'date': ['2020-01-01', '2020-01-02', '2020-01-01', '2020-01-02'],
    'city': ['NY', 'NY', 'LA', 'LA'],
    'temperature': [30, 28, 35, 33]
})
```

```
df_pivot = df.pivot(index='date', columns='city', values='temperature')
print(df_pivot)
```

```
city      LA  NY
date
2020-01-01  35  30
2020-01-02  33  28
```

`pivot_table()` 함수

``pivot_table()`` 함수는 더 유연하고 강력한 데이터 재구성 도구로, 특히 다중 인덱스와 다중 값을 처리하는 데 적합합니다. 또한 그룹화된 데이터를 요약할 때 유용합니다.

특징

- 집계 기능: 동일한 행/열 인덱스 조합이 여러 개 있을 경우, 지정된 집계 함수를 사용하여 데이터를 요약할 수 있습니다.
- 다중 값 열: 여러 값 열을 처리할 수 있습니다.
- 기본 집계 함수: 평균(``mean``)을 기본 집계 함수로 사용하지만, 다른 집계 함수(``sum``, ``count``, ``min``, ``max`` 등)를 사용할 수 있습니다
- 매개변수:
 - ``index``: 새로운 데이터 프레임에서 행을 나타낼 열 이름(들)
 - ``columns``: 새로운 데이터 프레임에서 열을 나타낼 열 이름(들)
 - ``values``: 새로운 데이터 프레임에서 값을 채울 열 이름(들)
 - ``aggfunc``: 집계 함수, 기본값은 ``mean``
 - ``fill_value``: 결측값을 채울 값 (옵션)
 - ``margins``: 총계 또는 소계 행/열 추가 여부 (옵션)

예제

```
df_pivot_table = df.pivot_table(index='date', columns='city', values='temperature', aggfunc='mean')
print(df_pivot_table)
```

```

city      LA    NY
date
2020-01-01 35.0 30.0
2020-01-02 33.0 28.0

### 차이점 요약

1. 단일 vs. 다중 값 열:
   `pivot()`은 단일 값 열을 처리하지만, `pivot_table()`은 다중 값 열을 처리할 수 있습니다.
2. 고유 인덱스 조합:
   `pivot()`은 행/열 인덱스 조합이 고유해야 하는 반면,
   `pivot_table()`은 동일한 조합이 있을 경우 집계 함수를 사용하여 데이터를 요약할 수 있습니다.
3. 기본 집계 함수:
   `pivot_table()`은 기본적으로 평균(`mean`)을 사용하여 데이터를 요약합니다.
4. 결측값 처리:
   `pivot_table()`은 결측값을 채울 수 있는 옵션(`fill_value`)을 제공합니다.
5. 총계/소계 추가:
   `pivot_table()`은 총계 또는 소계 행/열을 추가할 수 있는 옵션(`margins`)을 제공합니다.
...
print('')

```

```

In [13]: '''
`pivot_table()` 함수는 다중 열을 처리할 수 있습니다.

### 코드 설명
1. index: `date` 열을 행 인덱스로 사용합니다.
2. columns: `city`와 `type` 열을 열 인덱스로 사용합니다. 이로 인해 다중 인덱스가 생성됩니다.
3. values: `sales`와 `profit` 열을 값으로 사용합니다.
4. aggfunc: 집계 함수로 `mean`을 사용하여 동일한 그룹의 평균을 계산합니다.

### 다중 인덱스 접근
결과는 다중 인덱스를 사용한 피벗 테이블입니다.
각 `date`에 대해 `city`와 `type` 조합별로 `sales`와 `profit`의 평균값이 계산되어 있습니다.

다중 인덱스 열에 접근하려면 다음과 같이 할 수 있습니다:
# 특정 값 열에 접근하는 방법
pivot_df['sales']
pivot_df['profit']

이를 통해 각 `date`에 대한 `city`와 `type` 조합별 `sales`와 `profit` 값을 각각 확인할 수 있습니다.

```

```
'''
print('')
```

```
In [14]: import pandas as pd

data = {
    'date': ['2020-01-01', '2020-01-01', '2020-01-02', '2020-01-02',
            '2020-01-01', '2020-01-01', '2020-01-02', '2020-01-02'],
    'city': ['NY', 'NY', 'NY', 'NY', 'LA', 'LA', 'LA', 'LA'],
    'type': ['A', 'B', 'A', 'B', 'A', 'B', 'A', 'B'],
    'sales': [100, 150, 200, 250, 300, 350, 400, 450],
    'profit': [10, 15, 20, 25, 30, 35, 40, 45]
}

df = pd.DataFrame(data)
df
```

```
Out[14]:
```

	date	city	type	sales	profit
0	2020-01-01	NY	A	100	10
1	2020-01-01	NY	B	150	15
2	2020-01-02	NY	A	200	20
3	2020-01-02	NY	B	250	25
4	2020-01-01	LA	A	300	30
5	2020-01-01	LA	B	350	35
6	2020-01-02	LA	A	400	40
7	2020-01-02	LA	B	450	45

```
In [15]: pivot_df = df.pivot_table(
    index='date', # 행 인덱스
    columns=['city', 'type'], # 열 인덱스
    values=['sales', 'profit'], # 값 열
    aggfunc='mean' # 집계 함수
)
```



```
pivot_df
```

```
Out[15]:
```

	profit						sales			
	city	LA		NY		LA		NY		
		type	A	B	A	B	A	B	A	B
			date							
2020-01-01	30	35	10	15	300	350	100	150		
2020-01-02	40	45	20	25	400	450	200	250		

```
In [16]: pivot_df['sales']
```

```
Out[16]:
```

	city		LA		NY	
	type	A	B	A	B	
	date					
2020-01-01	300	350	100	150		
2020-01-02	400	450	200	250		

```
In [17]: pivot_df['profit']
```

```
Out[17]:
```

	city		LA		NY	
	type	A	B	A	B	
	date					
2020-01-01	30	35	10	15		
2020-01-02	40	45	20	25		

```
In [18]: # 다시 문제 풀이로...
# 방법 1: pd.DataFrame.pivot - 셀에 대응하는 데이터가 1개 이므로 성립합니다.
df_elec1 = df_elec.pivot(index='DateHour',
                          columns='Minute',
```

```
values='Value')

df_elec1.head()
```

Out[18]:

	Minute	15분	30분	45분	60분
DateHour					
2021-01-01 00:00:00		62	61	61	61
2021-01-01 01:00:00		96	93	116	113
2021-01-01 02:00:00		106	96	106	107
2021-01-01 03:00:00		92	110	110	109
2021-01-01 04:00:00		108	105	106	108

```
In [19]: # 방법 2: crosstab
# 동일 데이터수의 Series나 DataFrame을 입력받아 처리할 수 있습니다.
# - aggfunc로 index, columns에 대응하는 values들을 집계합니다.
# - 이 경우는 대응하는 변수의 수가 1개이므로,
# sum, max, min,... 등이 성립합니다.
df_elec1 = df_elec.pivot_table(index='DateHour',
                                columns='Minute',
                                values='Value',
                                aggfunc='sum')

df_elec1.head()
```

Out[19]:

	Minute	15분	30분	45분	60분
DateHour					
2021-01-01 00:00:00		62	61	61	61
2021-01-01 01:00:00		96	93	116	113
2021-01-01 02:00:00		106	96	106	107
2021-01-01 03:00:00		92	110	110	109
2021-01-01 04:00:00		108	105	106	108

```
In [20]: # 방법 3: pivot_table
# - 주로 index와 columns에 대응하는 values가 1개가 넘을 경우 사용 합니다.
df_elec1 = pd.crosstab(index=df_elec['DateHour'],
                        columns=df_elec['Minute'],
                        values=df_elec['Value'],
                        aggfunc='min')

df_elec1.head()
```

```
Out[20]:
```

	Minute	15분	30분	45분	60분
DateHour					
2021-01-01 00:00:00		62	61	61	61
2021-01-01 01:00:00		96	93	116	113
2021-01-01 02:00:00		106	96	106	107
2021-01-01 03:00:00		92	110	110	109
2021-01-01 04:00:00		108	105	106	108

```
In [21]: df_info = pd.read_csv('info.csv')

df_info['Date'] = pd.to_datetime(df_info['Date'])

display(df_info.info())

df_info.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6168 entries, 0 to 6167
Data columns (total 9 columns):
Date                6168 non-null datetime64[ns]
생산량              6168 non-null int64
기온                6168 non-null float64
풍속                6165 non-null float64
습도                6168 non-null int64
강수량              6167 non-null float64
전기요금(계절)      6168 non-null float64
공장인원            6151 non-null float64
인건비              6168 non-null float64
dtypes: datetime64[ns](1), float64(6), int64(2)
memory usage: 433.8 KB
None
```

```
Out[21]:
```

	Date	생산량	기온	풍속	습도	강수량	전기요금(계절)	공장인원	인건비
0	2021-01-01 00:00:00	0	-3.2	2.4	71	0.0	109.8	0.0	1.5
1	2021-01-01 01:00:00	0	-4.5	1.5	77	0.0	109.8	0.0	1.5
2	2021-01-01 02:00:00	0	-3.9	2.6	58	0.0	109.8	0.0	1.5
3	2021-01-01 03:00:00	0	-4.1	2.6	56	0.0	109.8	0.0	1.5
4	2021-01-01 04:00:00	0	-4.6	2.6	60	0.0	109.8	0.0	1.5

단계 2

데이터셋 elec1의 DateHour 변수를 이용하여 아래의 파생변수를 생성한다.

-요일명(DayName): DateHour 변수에서 요일명을 숫자로 추출, 0(월)~6(일)으로 반환

-시간(Hour): DateHour 변수에서 시간 추출

-오전/오후(AM): 생성한 시간(Hour) 변수를 기준으로

12시 전(< 12:00)은 오전(0), 이후(>= 12:00)는 오후(1)로 표기, 이때 표기는 숫자 0, 1로 표기

-주말유무(Weekend_yn): 생성한 요일명(DayName) 변수를 이용하여 5,6이면 1(주말), 그외는 0(평일)로 표기

-공휴일(Holiday_yn): 다음은 공휴일 목록임

["2021-01-01", "2021-02-11", "2021-02-12", "2021-03-01", "2021-05-05", "2021-05-19", "2021-08-16"],

목록에 해당하면 1, 아니면 0로 표기한다. DateHour의 일자를 기준으로 판별한다.

-최대수요전력 평균(Avg):

시간당 15분 간격(15분, 30분, 45분, 60분)으로 측정한 최대수요전력 (Value)의 평균값

-최대수요전력 합계(TotalHour):

시간당 15분 간격(15분, 30분, 45분, 60분)으로 측정한 최대수요전력 (Value)의 총합

In [22]: `#df.assign()` 함수

...

`pandas` 라이브러리의 ``df.assign()`` 함수는 데이터프레임에 하나 이상의 새로운 열을 추가하거나 기존 열을 수정할 때 사용됩니다. 이 함수는 원본 데이터프레임을 변경하지 않고, 새로운 데이터프레임을 반환합니다. 이로써 체이닝(chaining)을 통해 데이터를 변형하고 분석할 때 유용합니다.

``df.assign()`` 함수의 기본 구조
`DataFrame.assign(kwarg)`

매개 변수

- ``kwargs``: 새로운 열 이름을 키로 하고, 해당 열에 들어갈 값을 값으로 갖는 키워드 인수들입니다. 값으로는 다음을 사용할 수 있습니다:

- 스칼라 값
- 배열이나 시리즈
- 함수

반환값

- 원본 데이터프레임에 새로운 열이 추가된 새로운 데이터프레임을 반환합니다.

사용 예제

새로운 열 추가
`import pandas as pd`

예제 데이터프레임
`df = pd.DataFrame({`

```
'A': [1, 2, 3],
'B': [4, 5, 6]
})

### 새로운 열 C 추가
df_new = df.assign(C=[7, 8, 9])
print(df_new)

출력 결과:
   A  B  C
0  1  4  7
1  2  5  8
2  3  6  9

#### 기존 열 수정

### 열 A의 값을 수정하여 새로운 데이터프레임 생성
df_modified = df.assign(A=df['A'] * 2)
print(df_modified)

출력 결과:
   A  B
0  2  4
1  4  5
2  6  6

#### 함수 사용
열의 값을 계산하는 함수를 사용할 수도 있습니다.

# 열 D는 A와 B의 합
df_with_function = df.assign(D=lambda x: x['A'] + x['B'])
print(df_with_function)

출력 결과:
   A  B  D
0  1  4  5
1  2  5  7
2  3  6  9

#### 여러 열 추가

여러 개의 열을 동시에 추가할 수도 있습니다.
```

```
### 열 C와 D 추가
df_multiple = df.assign(
    C=[7, 8, 9],
    D=lambda x: x['A'] + x['B']
)
print(df_multiple)
```

출력 결과:

	A	B	C	D
0	1	4	7	5
1	2	5	8	7
2	3	6	9	9

주요 포인트

1. 불변성: `assign()` 함수는 원본 데이터프레임을 수정하지 않고 새로운 데이터프레임을 반환합니다.
2. 체이닝: 데이터프레임 변환 작업을 체이닝 방식으로 연속해서 수행할 수 있습니다.
3. 동적 열 추가: 함수나 계산된 값을 통해 동적으로 새로운 열을 추가할 수 있습니다.

예제: 체이닝을 통한 데이터 변환

원본 데이터프레임

```
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
})
```

체이닝을 통해 여러 변환 수행

```
df_transformed = (df
    .assign(C=lambda x: x['A'] * 2)
    .assign(D=lambda x: x['B'] + x['C'])
    .assign(E=10)
)
print(df_transformed)
```

출력 결과:

	A	B	C	D	E
0	1	4	2	6	10
1	2	5	4	9	10
2	3	6	6	12	10

이 예제에서 `df` 데이터프레임에 대해 `C` 열을 추가하고, 그 다음 `D` 열을 추가하며, 마지막으로 `E` 열을 추가하는 여러 변환을 체이닝 방식으로 수행했습니다.

```
### 결론
```

`df.assign()` 함수는 파이썬의 `pandas` 라이브러리에서 데이터프레임에 새로운 열을 추가하거나 기존 열을 수정하는 데 매우 유용한 함수입니다. 원본 데이터프레임을 수정하지 않고 새로운 데이터프레임을 반환하기 때문에, 데이터 변환을 안전하게 수행할 수 있으며, 체이닝을 통해 복잡한 변환을 간결하게 작성할 수 있습니다.

```
'''
```

```
print('')
```

In [23]:

```
'''
```

주어진 코드는 `pandas`와 `numpy`를 사용하여 데이터프레임 `df_elec1`에 새로운 열들을 추가하는 작업을 수행합니다.

```
### 휴일 리스트 생성
```

- `pd.to_datetime`을 사용하여 주어진 날짜 문자열 리스트를 `datetime` 객체로 변환합니다.
- `.date` 속성을 사용하여 `datetime` 객체를 날짜(`date`) 객체로 변환합니다.
- 결과적으로, `holi`는 날짜 객체들의 리스트가 됩니다.

```
### 새로운 열 추가
```

여기서는 `df_elec1` 데이터프레임에 여러 새로운 열을 추가합니다.

각 열이 어떤 역할을 하는지 설명하겠습니다:

```
1. `DayName`:
```

- `df_elec1.index.dayofweek`는 데이터프레임의 인덱스에서 요일을 나타내는 숫자를 반환합니다. 월요일은 0, 일요일은 6입니다.
- 예를 들어, `2021-01-01`은 금요일이므로 `DayName`은 4입니다.

```
2. `Hour`:
```

- `df_elec1.index.hour`는 인덱스에서 시간 부분을 추출합니다.
- 예를 들어, `2021-01-01 15:00:00`은 `Hour`가 15입니다.

```
3. `AM`:
```

- `(df_elec1.index.hour >= 12).astype(int)`는 시간이 12시 이상이면 1, 그렇지 않으면 0을 반환합니다.
- 오후 시간은 1, 오전 시간은 0으로 표시됩니다.

```
4. `Weekend_yn`:
```

- `df_elec1.index.dayofweek.isin([5, 6]).astype(int)`는 요일이 토요일(5)이나 일요일(6)인 경우 1, 그렇지 않은 경우 0을 반환합니다.
- 주말이면 1, 평일이면 0으로 표시됩니다.

```
5. `Holiday_yn`:
```

- `np.isin(df_elec1.index.date, holi).astype(int)`는 날짜가 `holi` 리스트에 있는 경우 1, 그렇지 않은 경우 0을 반환합니다.
- 휴일이면 1, 평일이면 0으로 표시됩니다.

```
6. `Avg`:
```

- `df_elec1[['15분', '30분', '45분', '60분']].mean(axis=1)`는 지정된 열(`15분`, `30분`, `45분`, `60분`)의 평균을 계산합니다.
- 각 행(row)마다 평균 값을 계산하여 `Avg` 열에 저장합니다.

```
7. `TotalHour`:
```

- `df_elec1[['15분', '30분', '45분', '60분']].sum(axis=1)`는 지정된 열의 합을 계산합니다.
- 각 행마다 합계를 계산하여 `TotalHour` 열에 저장합니다.

```
'''
```



```
df_elec1.index
```

```
Out[23]: DatetimeIndex(['2021-01-01 00:00:00', '2021-01-01 01:00:00',
                        '2021-01-01 02:00:00', '2021-01-01 03:00:00',
                        '2021-01-01 04:00:00', '2021-01-01 05:00:00',
                        '2021-01-01 06:00:00', '2021-01-01 07:00:00',
                        '2021-01-01 08:00:00', '2021-01-01 09:00:00',
                        ...,
                        '2021-09-14 14:00:00', '2021-09-14 15:00:00',
                        '2021-09-14 16:00:00', '2021-09-14 17:00:00',
                        '2021-09-14 18:00:00', '2021-09-14 19:00:00',
                        '2021-09-14 20:00:00', '2021-09-14 21:00:00',
                        '2021-09-14 22:00:00', '2021-09-14 23:00:00'],
                        dtype='datetime64[ns]', name='DateHour', length=6168, freq=None)
```

```
In [24]: holi = pd.to_datetime(["2021-01-01", "2021-02-11", "2021-02-12",
                                "2021-03-01", "2021-05-05", "2021-05-19", "2021-08-16"]).date
# holi = ["2021-01-01", "2021-02-11", "2021-02-12", "2021-03-01",
#         "2021-05-05", "2021-05-19", "2021-08-16"]
# Holiday_yn = df_elec1.index.strftime('%Y-%m-%d').isin(holi).astype('int'),

df_elec1 = df_elec1.assign(
    DayName = df_elec1.index.dayofweek,
    Hour = df_elec1.index.hour,
    AM = (df_elec1.index.hour >= 12).astype(int),
    Weekend_yn = df_elec1.index.dayofweek.isin([5, 6]).astype(int),
    Holiday_yn = np.isin(df_elec1.index.date, holi).astype(int),
    # 열단위 통계를 낼 때는 대상 컬럼들을 명시해주세요.
    Avg = df_elec1[['15분', '30분', '45분', '60분']].mean(axis=1),
    TotalHour = df_elec1[['15분', '30분', '45분', '60분']].sum(axis=1)
)

df_elec1.head()
```

Out[24]:

	Minute	15분	30분	45분	60분	DayName	Hour	AM	Weekend_yn	Holiday_yn	Avg	TotalHour	
DateHour													
2021-01-01 00:00:00	62	61	61	61		4	0	0		0	1	61.25	245
2021-01-01 01:00:00	96	93	116	113		4	1	0		0	1	104.50	418
2021-01-01 02:00:00	106	96	106	107		4	2	0		0	1	103.75	415
2021-01-01 03:00:00	92	110	110	109		4	3	0		0	1	105.25	421
2021-01-01 04:00:00	108	105	106	108		4	4	0		0	1	106.75	427

단계 3

info데이터셋에 결측치가 존재한다. 이 때 발생한 결측치들은 0으로 변경하시오.

- 풍속, 강수량은 바람이 불지 않는 경우, 비가 오지 않는 경우,

기록되지 않는 경우가 존재함, 이는 0의 의미임

- 공장인원의 경우 생산량이 없다던가 인력이 필요하지 않은 경우,

기록되지 않는 경우가 존재함, 이는 0의 의미임

- 결측치를 처리한 후 데이터셋명은 info1로 표기함

```
In [25]: df_info1 = df_info.fillna(0)
df_info1.isna().sum()
```

```
Out[25]: Date      0
생 산 량      0
기 온        0
풍 속        0
습 도        0
강 수 량      0
전기 요금 (계절)  0
공장 인원      0
인 건 비      0
dtype: int64
```

단계 4

elec1와 info1 데이터프레임을 결합하시오.

결합 시 elec1데이터셋 에서는 DateHour 변수,

info1데이터셋 에서는 Date 변수를 기준으로 결합하시오.

- elec1 데이터셋에 info1을 데이터셋 결합
- 결합 방식: inner join, elec1.DateHour == info1.Date
- 결합 후 Date와 DateHour 같다. 두 변수를 이름을 DateHour 하여 하나만 남긴다.
- 결합 후 생성된 데이터셋명 basetable1로 표기(6,168 rows, 20 columns)

```
In [26]: display(df_elec1.head())
display(df_info1.head())
```

Minute	15분	30분	45분	60분	DayName	Hour	AM	Weekend_yn	Holiday_yn	Avg	TotalHour
DateHour											
2021-01-01 00:00:00	62	61	61	61	4	0	0	0	1	61.25	245
2021-01-01 01:00:00	96	93	116	113	4	1	0	0	1	104.50	418
2021-01-01 02:00:00	106	96	106	107	4	2	0	0	1	103.75	415
2021-01-01 03:00:00	92	110	110	109	4	3	0	0	1	105.25	421
2021-01-01 04:00:00	108	105	106	108	4	4	0	0	1	106.75	427

	Date	생산량	기온	풍속	습도	강수량	전기요금(계절)	공장인원	인건비
0	2021-01-01 00:00:00	0	-3.2	2.4	71	0.0	109.8	0.0	1.5
1	2021-01-01 01:00:00	0	-4.5	1.5	77	0.0	109.8	0.0	1.5
2	2021-01-01 02:00:00	0	-3.9	2.6	58	0.0	109.8	0.0	1.5
3	2021-01-01 03:00:00	0	-4.1	2.6	56	0.0	109.8	0.0	1.5
4	2021-01-01 04:00:00	0	-4.6	2.6	60	0.0	109.8	0.0	1.5

In [27]:

```
'''
`df_elec1`와 `df_info1` 데이터프레임을 병합할 때, 결과 데이터프레임에서 `DateHour` 컬럼이 없어지는 이유

병합 시 `left_on`에 지정된 `DateHour`는 `df_elec1`의 인덱스입니다.
`merge` 함수는 인덱스와 열을 병합할 때 다음과 같은 방식으로 처리합니다:

1. `df_elec1`의 인덱스인 `DateHour`를 열로 변환하여 병합합니다.
2. 병합 후 결과 데이터프레임에서 `DateHour`는 인덱스가 아닌 열로 포함되지만,
`right_on`에 사용된 `Date` 열과 같은 값을 가지므로 하나만 남게 됩니다.
따라서, 병합 후 `DateHour`와 `Date`가 같은 값을 가지게 되므로, 결과 데이터프레임에서는 `DateHour` 열이 남지 않게 됩니다.

### 해결 방법
병합 후 `DateHour` 열을 유지하고 싶다면, 다음과 같이 `DateHour`를 인덱스에서 열로 변환한 후 병합할 수 있습니다:

df_elec1_reset = df_elec1.reset_index()
df_basetable1 = df_elec1_reset.merge(
    df_info1, left_on='DateHour', right_on='Date', how='inner'
)
'''
```

이렇게 하면 `DateHour`가 인덱스가 아닌 열로 병합되므로, 결과 데이터프레임에서 `DateHour` 열이 유지됩니다.
 `reset\_index()` 함수를 사용하여 인덱스를 열로 변환합니다.

```
df_basetable1 = df_elec1.merge(
    df_info1, left_on='DateHour', right_on='Date', how='inner'
)

display(df_basetable1.head())
```

	15 분	30 분	45 분	60 분	DayName	Hour	AM	Weekend_yn	Holiday_yn	Avg	TotalHour	Date	생 산 량	기 온	풍 속	습 도	강 수 량	전기요 금(계 절)	공장 인원	인 건 비
0	62	61	61	61	4	0	0	0	1	61.25	245	2021-01-01 00:00:00	0	-3.2	2.4	71	0.0	109.8	0.0	1.5
1	96	93	116	113	4	1	0	0	1	104.50	418	2021-01-01 01:00:00	0	-4.5	1.5	77	0.0	109.8	0.0	1.5
2	106	96	106	107	4	2	0	0	1	103.75	415	2021-01-01 02:00:00	0	-3.9	2.6	58	0.0	109.8	0.0	1.5
3	92	110	110	109	4	3	0	0	1	105.25	421	2021-01-01 03:00:00	0	-4.1	2.6	56	0.0	109.8	0.0	1.5
4	108	105	106	108	4	4	0	0	1	106.75	427	2021-01-01 04:00:00	0	-4.6	2.6	60	0.0	109.8	0.0	1.5

In [28]: # 방법 1: merge - 호출자, 피호출자의 컬럼 / 인덱스를 기준으로 결합
 # 결합후 Date만 남게 됩니다. 이를 rename을 통해 DateHour로 이름을 변경합니다.
 # DateHour는 유일값이라 set_Index로 인덱스로 활용합니다.

```
df_basetable1 = df_elec1.merge(
    df_info1, left_on='DateHour', right_on='Date', how='inner'
).rename(columns={'Date': 'DateHour'}).set_index('DateHour')

display(df_basetable1.head())

df_basetable1.shape
```

	15 분	30 분	45 분	60 분	DayName	Hour	AM	Weekend_yn	Holiday_yn	Avg	TotalHour	생 산 량	기 온	풍 속	습 도	강 수 량	전기요 금(계절)	공장 인원	인 건 비				
DateHour																							
2021-01-01 00:00:00	62	61	61	61		4	0	0		0		1	61.25		245	0	-3.2	2.4	71	0.0	109.8	0.0	1.5
2021-01-01 01:00:00	96	93	116	113		4	1	0		0		1	104.50		418	0	-4.5	1.5	77	0.0	109.8	0.0	1.5
2021-01-01 02:00:00	106	96	106	107		4	2	0		0		1	103.75		415	0	-3.9	2.6	58	0.0	109.8	0.0	1.5
2021-01-01 03:00:00	92	110	110	109		4	3	0		0		1	105.25		421	0	-4.1	2.6	56	0.0	109.8	0.0	1.5
2021-01-01 04:00:00	108	105	106	108		4	4	0		0		1	106.75		427	0	-4.6	2.6	60	0.0	109.8	0.0	1.5

Out[28]: (6168, 19)

```
'''
'Date' 컬럼이 왜 없어지는지에 대한 이유는 'Date' 열이 이미 df_elec1의 인덱스인 'DataHour'로 병합되어 있기 때문입니다.
join 함수는 인덱스를 기준으로 병합하기 때문에 'Date' 컬럼은 결과 데이터프레임에서는 별도의 열로 남지 않고,
인덱스 열로 포함됩니다.

결론적으로, 'Date' 열이 df_info1의 인덱스로 설정되는 것이 아니라, df_elec1의 'DateHour'가 원래부터 인덱스로 설정되어 있으며,
이 인덱스가 병합 결과의 인덱스로 유지되는 것입니다.
'''

# 방법 2: join - 호출자 - 컬럼/인덱스, 피호출자 - 인덱스
# 피호출자는 인덱스만을 기준으로 할 수 있습니다.
# df_info1의 Date를 인덱스로 위치시켜, join의 기준을 삼도록 합니다.

df_basetable1 = df_elec1.join(df_info1.set_index('Date'),
                              on='DateHour', how='inner')
display(df_basetable1.head())
df_basetable1.shape
```

	15 분	30 분	45 분	60 분	DayName	Hour	AM	Weekend_yn	Holiday_yn	Avg	TotalHour	생 산 량	기 온	풍 속	습 도	강 수 량	전기요 금(계절)	공장 인원	인 건 비
DateHour																			
2021-01-01 00:00:00	62	61	61	61	4	0	0	0	1	61.25	245	0	-3.2	2.4	71	0.0	109.8	0.0	1.5
2021-01-01 01:00:00	96	93	116	113	4	1	0	0	1	104.50	418	0	-4.5	1.5	77	0.0	109.8	0.0	1.5
2021-01-01 02:00:00	106	96	106	107	4	2	0	0	1	103.75	415	0	-3.9	2.6	58	0.0	109.8	0.0	1.5
2021-01-01 03:00:00	92	110	110	109	4	3	0	0	1	105.25	421	0	-4.1	2.6	56	0.0	109.8	0.0	1.5
2021-01-01 04:00:00	108	105	106	108	4	4	0	0	1	106.75	427	0	-4.6	2.6	60	0.0	109.8	0.0	1.5

Out[29]: (6168, 19)

In [30]: #방법1과 방법 2의 비교

...

두 코드 모두 `df\_elec1`과 `df\_info1` 데이터프레임을 병합하여 `df\_basetable1`을 생성하는 작업을 수행합니다.
그러나 병합 방식을 다르게 사용하고 있습니다.

첫 번째 방법: `merge`와 `rename` 사용

1. `merge`:

- `df\_elec1`와 `df\_info1`을 병합합니다.
- `left\_on='DateHour'`는 `df\_elec1`의 `DateHour` 열을 기준으로 병합을 의미합니다.
- `right\_on='Date'`는 `df\_info1`의 `Date` 열을 기준으로 병합을 의미합니다.
- `how='inner'`는 교집합(내부 조인)을 사용하여 병합합니다.
두 데이터프레임 모두에서 일치하는 키가 있는 행만 포함됩니다.

2. `rename`:

- `df\_info1`의 `Date` 열 이름을 `DateHour`로 변경합니다.

3. `set\_index`:

- 병합된 데이터프레임의 인덱스를 `DateHour`로 설정합니다.

두 번째 방법: `join` 사용

1. `set\_index`:

```

- `df_info1`의 `Date` 열을 인덱스로 설정합니다.
2. `join`:
- `df_elec1`와 `df_info1`을 병합합니다.
- `on='DateHour'`는 `df_elec1`의 `DateHour` 열을 기준으로 병합을 의미합니다.
- `how='inner'`는 교집합(내부 조인)을 사용하여 병합합니다.
    두 데이터프레임 모두에서 일치하는 키가 있는 행만 포함됩니다.

### 특징
- 이 방법은 `df_info1`의 `Date` 열을 인덱스로 설정한 후, `df_elec1`의 `DateHour` 열과 기준으로 병합합니다.
- 병합 후 `df_info1`의 인덱스 열은 기본 인덱스로 사용되지 않고 `df_elec1`의 인덱스 구조가 유지됩니다.

### 주요 차이점
1. 병합 방식:
- 첫 번째 방법은 `merge`를 사용하여 명시적으로 `left_on`과 `right_on`을 지정하여 병합합니다.
- 두 번째 방법은 `join`을 사용하여 `df_info1`의 인덱스를 설정하고 `on`을 지정하여 병합합니다.

2. 결과 데이터프레임의 인덱스:
- 첫 번째 방법에서는 `rename`과 `set_index`를 사용하여 결과 데이터프레임의 인덱스를 `DateHour`로 설정합니다.
- 두 번째 방법에서는 `df_elec1`의 기존 인덱스가 유지됩니다.

3. 열 이름 충돌 방지:
- 첫 번째 방법에서는 `rename`을 사용하여 병합 후 열 이름 충돌을 방지합니다.
- 두 번째 방법에서는 `join`을 사용하여 열 이름 충돌이 자동으로 처리됩니다.

### 선택 기준
- 명시적 병합: 첫 번째 방법은 `merge`와 `rename`을 통해 명시적으로 열 이름을 지정하고 인덱스를 설정합니다.
    더 많은 제어가 필요한 경우 유용합니다.
- 편의성: 두 번째 방법은 `join`을 사용하여 더 간결하게 병합할 수 있습니다. 특히 인덱스를 기준으로 병합하는 경우 편리합니다.
...
print('')

```

문제 1

basetable1을 이용하여 다음 질문에 답하시오.

단계 1-1

시간당 최대수요전력 합계(TotalHour)의 최대값과 최소값이 차이를 구하고, 이 값을 A라고 한다.

```
In [31]: # 방법 1: max, min 따로 구하여 A를 뽑아봅니다.
A = df_basetable1['TotalHour'].max() - df_basetable1['TotalHour'].min()
A
```

Out[31]: 830

```
In [32]: #메소드 체이닝
'''
`pandas` 라이브러리에서는 데이터프레임에 대한 일련의 변환을 체계적으로 수행할 때 메소드 체이닝을 많이 사용합니다.

## `pandas`에서의 메소드 체이닝
`pandas`에서 메소드 체이닝은 데이터프레임을 일련의 변환 및 필터링 작업을 통해 처리할 때 유용합니다.
예를 들어, 데이터 프레임에서 데이터를 필터링하고, 그룹화하고, 집계하는 작업을 수행한다고 가정해보겠습니다.

#### 예시 데이터프레임 생성
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Edward'],
    'Age': [24, 27, 22, 32, 29],
    'Score': [88, 92, 95, 85, 90]
}

df = pd.DataFrame(data)

## 메소드 체이닝 없이 데이터 처리
메소드 체이닝을 사용하지 않고 각각의 단계를 별도로 수행하면 코드가 장황해질 수 있습니다.

### 필터링
filtered_df = df[df['Age'] > 25]

### 새로운 열 추가
filtered_df['Adjusted Score'] = filtered_df['Score'] * 1.1

### 정렬
sorted_df = filtered_df.sort_values(by='Adjusted Score', ascending=False)
print(sorted_df)

출력:
```

	Name	Age	Score	Adjusted Score
1	Bob	27	92	101.2
4	Edward	29	90	99.0
3	David	32	85	93.5

메소드 체이닝 사용

메소드 체이닝을 사용하면 동일한 작업을 더 간결하고 명확하게 작성할 수 있습니다.

메소드 체이닝을 사용한 데이터 처리

```
result = (df[df['Age'] > 25]
          .assign(Adjusted_Score = df['Score'] * 1.1)
          .sort_values(by='Adjusted_Score', ascending=False))
print(result)
```

출력:

	Name	Age	Score	Adjusted_Score
1	Bob	27	92	101.2
4	Edward	29	90	99.0
3	David	32	85	93.5

`pipe()`를 사용한 메소드 체이닝

`pipe()`를 사용하면 더 복잡한 변환을 수행할 수 있습니다. 함수와 함께 메소드 체이닝을 사용하여 코드를 더욱 유연하게 만들 수 있습니다

```
def adjust_score(df):
    df['Adjusted_Score'] = df['Score'] * 1.1
    return df

result = (df[df['Age'] > 25]
          .pipe(adjust_score)
          .sort_values(by='Adjusted_Score', ascending=False))
print(result)
```

출력:

	Name	Age	Score	Adjusted_Score
1	Bob	27	92	101.2
4	Edward	29	90	99.0
3	David	32	85	93.5

장점

1. 가독성: 코드가 더 간결하고 논리적 흐름이 명확해집니다.
2. 유연성: 여러 단계를 간단하게 결합할 수 있습니다.
3. 모듈화: 각 변환 단계를 별도의 함수로 분리할 수 있어 재사용이 용이합니다.

결론

메소드 체이닝은 객체 지향 프로그래밍과 데이터 분석에서 코드의 가독성을 높이고, 여러 작업을 논리적인 순서로 연결하여 처리할 수 있는 강력한 기법입니다. 특히 `pandas`에서는 데이터 변환 작업을 명확하게 표현할 수 있어 매우 유용합니다.

pipe() 함수

`pipe()` 함수는 파이썬의 `pandas` 라이브러리에서 제공하는 함수로, 메소드 체이닝을 보다 명확하고 유연하게 작성할 수 있게 도와줍니다. `pipe()`를 사용하면 데이터프레임을 다양한 함수로 처리할 수 있으며, 복잡한 변환 과정을 간결하고 읽기 쉽게 표현할 수 있습니다.

기본 사용법

`pipe()` 함수는 데이터프레임이나 시리즈에 연산을 적용할 때 사용됩니다. `pipe()`를 사용하면 함수 체인을 통해 연산을 수행할 수 있습니다.

`DataFrame.pipe(func, *args, kwargs)`

- `func`: 적용할 함수
- `\*args`, `kwargs`: 함수에 전달할 추가 인수들

1. 간단한 함수 적용

```
import pandas as pd
```

```
# 예시 데이터프레임
```

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
```

```
# 함수 정의
```

```
def add_constant(df, const):
    return df + const
```

```
# pipe를 사용하여 함수 적용
```

```
result = df.pipe(add_constant, 10)
print(result)
```

위 코드에서는 `add\_constant`라는 함수가 정의되어 있으며, 이 함수는 데이터프레임에 상수를 더하는 역할을 합니다. `pipe()`를 사용하여 이 함수를 데이터프레임에 적용하였습니다.

```
      A  B
0  11  14
1  12  15
2  13  16
```

2. 메소드 체이닝과 함께 사용

`pipe()`는 메소드 체이닝에서 매우 유용합니다. 예를 들어, 여러 개의 변환을 순차적으로 적용하는 경우에 사용됩니다.

여러 변환 함수를 정의

```
def add_constant(df, const):
    return df + const
```

```
def multiply_constant(df, const):
    return df * const
```

pipe를 사용하여 함수 체이닝

```
result = (df
          .pipe(add_constant, 10)
          .pipe(multiply_constant, 2))
print(result)
```

출력 결과는 다음과 같습니다:

```
   A  B
0  22 28
1  24 30
2  26 32
```

3. 람다 함수와 함께 사용

`pipe()`는 람다 함수와 함께 사용할 수도 있습니다.

pipe와 람다 함수를 사용하여 변환 적용

```
result = df.pipe(lambda x: x + 10).pipe(lambda x: x * 2)
print(result)
```

출력 결과는 이전 예제와 동일하게 나옵니다:

```
   A  B
0  22 28
1  24 30
2  26 32
```

고급 사용법

1. 다른 객체 전달

`pipe()`는 데이터프레임 이외의 다른 객체를 함수에 전달할 때도 사용할 수 있습니다.

```
# 다른 객체와 함께 사용
def calculate_sum(df, col):
    return df[col].sum()
```

```
total = df.pipe(calculate_sum, 'A')
print(total) # 출력: 6
```

위 코드에서는 `calculate\_sum` 함수가 데이터프레임과 열 이름을 받아 해당 열의 합계를 계산합니다. `pipe()`를 사용하여 이 함수를 호출할 수 있습니다.

```
# 여러 개의 인수 전달
여러 개의 인수를 함수에 전달할 때도 `pipe()`를 사용할 수 있습니다.
def add_multiply(df, add_const, mul_const):
    return (df + add_const) * mul_const
```

```
result = df.pipe(add_multiply, 10, 2)
print(result)
```

출력 결과는 다음과 같습니다:

```
   A   B
0  22  28
1  24  30
2  26  32
```

결론

`pipe()` 함수는 `pandas`에서 데이터프레임을 처리할 때 함수 체이닝을 더욱 유연하고 가독성 있게 만들어주는 강력한 도구입니다. 복잡한 데이터 처리 과정을 간결하게 표현할 수 있으며, 다양한 함수와 함께 사용하여 코드의 모듈성을 향상시킬 수 있습니다. `pipe()`를 사용하여 데이터프레임 변환을 보다 직관적이고 구조적으로 작성할 수 있습니다.

```
# 방법 2: 이 내용을 pipe로 받아서 바로 A를 계산합니다.
A = df_basetable1['TotalHour'].agg(['max', 'min']).pipe(
    lambda x: x['max'] - x['min']
)
A
```

Out[32]: 830

In [33]: df_basetable1['TotalHour'].agg(['max', 'min'])

```
Out[33]: max      830
         min        0
         Name: TotalHour, dtype: int64
```

```
In [34]: type(df_basetable1['TotalHour'].agg(['max', 'min']))
```

```
Out[34]: pandas.core.series.Series
```

```
In [35]: print(df_basetable1['TotalHour'].agg(['max', 'min']).iloc[0])
         print(df_basetable1['TotalHour'].agg(['max', 'min']).iloc[1])
         # print(df_basetable1['TotalHour'].agg(['max', 'min']).iloc[2]) -> IndexError

         print(df_basetable1['TotalHour'].agg(['max', 'min']).loc['max'])
         print(df_basetable1['TotalHour'].agg(['max', 'min']).loc['min'])

830
0
830
0
```

단계 1-2

일별로 시간당 최대수요전력 합계(TotalHour)의 최대값을 찾아

최대값이 빈번하게 발생하는 시간을 찾고자 한다.

일별로 최대수요전력 합계(TotalHour)의 최대값이 첫 번째로 빈번하게 발생하는 시간(Hour)을 B,

두 번째로 빈번하게 발생하는 시간(Hour)을 C라고 한다.

하루에 최대값에 해당하는 시간(Hour)이 여러 번 나올 수 있음을 고려하라.

```
In [36]: # 예제
df = pd.DataFrame({'Group': ['A', 'A', 'B', 'B', 'B'],
                   'Value': [1, 2, 3, 4, 5]})

print(df)
print('-----')
```

```
# 그룹별 평균 계산
df['Group Mean'] = df.groupby('Group')['Value'].transform(lambda x : print(x))
print('-----')
```

	Group	Value
0	A	1
1	A	2
2	B	3
3	B	4
4	B	5

```
-----
0    1
1    2
Name: A, dtype: int64
2    3
3    4
4    5
Name: B, dtype: int64
-----
```

In [37]: # transform() 함수

```
...
```

`transform()` 함수는 파이썬의 `pandas` 라이브러리에서 제공되는 함수 중 하나로, 그룹별로 연산을 수행한 후에 원본 데이터프레임의 형태로 결과를 반환합니다. 이 함수는 데이터프레임 또는 시리즈 객체에 적용되며, 각 그룹 내에서 개별 요소의 변환을 수행할 수 있습니다.

`transform()` 함수의 구문

```
DataFrame.transform(func, axis=0)
```

- `func`: 적용할 함수. 이 함수는 그룹별로 호출되며 각 그룹에 대해 시리즈 또는 데이터프레임을 반환해야 합니다.

- `axis`: 0 또는 'index'를 지정하면 각 열에 대해 적용됩니다.

1 또는 'columns'를 지정하면 각 행에 대해 적용됩니다. 기본값은 0입니다.

`transform()` 함수의 동작

`transform()` 함수는 다음과 같은 단계로 동작합니다:

1. 데이터프레임을 그룹화합니다.
2. 각 그룹에 대해 지정된 함수를 호출합니다.
3. 이 함수는 각 그룹의 데이터에 적용되며, 각 그룹에서 반환된 결과는 해당 그룹의 인덱스와 동일한 인덱스를 가지는 시리즈로 반환됩니다.
4. 이러한 결과 시리즈들은 원본 데이터프레임의 인덱스와 동일한 인덱스를 가지며, 각 원본 행에 해당하는 위치에 들어갑니다.

`transform()` 함수의 인자와 반환값

인자

- `func`: 각 그룹에 대해 적용할 함수입니다. 이 함수는 각 그룹에 대한 시리즈나 데이터프레임을 입력으로 받아야 합니다.

반환값

`transform()` 함수는 각 그룹에서 계산된 값을 포함하는 시리즈 또는 데이터프레임을 반환합니다. 반환된 객체는 원본 데이터프레임과 동일한 크기와 구조를 가지며, 각 그룹에서 계산된 값을 해당 그룹의 모든 행에 적용합니다.

예제

```
import pandas as pd
```

```
# 예시 데이터프레임 생성
```

```
df = pd.DataFrame({'Group': ['A', 'A', 'B', 'B', 'B'],
                    'Value': [1, 2, 3, 4, 5]})
```

```
# 그룹별 평균 계산
```

```
df['Group Mean'] = df.groupby('Group')['Value'].transform('mean')
```

```
print(df)
```

```
출력:
```

	Group	Value	Group Mean
0	A	1	1.5
1	A	2	1.5
2	B	3	4.0
3	B	4	4.0
4	B	5	4.0

이 예제에서는 'Group' 열을 기준으로 데이터프레임을 그룹화하고, 각 그룹 내에서 'Value' 열의 평균을 계산하여 'Group Mean' 열에 저장합니다. 이를 `transform()` 함수를 사용하여 각 그룹에서 계산한 평균값을 각 그룹 내의 모든 행에 적용합니다. 결과로 생성된 'Group Mean' 열은 원본 데이터프레임과 동일한 크기를 가지며, 각 그룹에서 계산된 평균값이 해당 그룹의 모든 행에 적용됩니다.

```
...
```

```
...
```

`df_basetable1.index.date`는 `pandas` 데이터프레임 `df_basetable1`의 인덱스에서 날짜 부분만 추출하는 표현입니다. `df_basetable1`의 인덱스가 `DatetimeIndex` 형식인 경우, `.date` 속성을 사용하여 인덱스의 각 요소에서 시간 정보를 제외한 날짜 정보만 추출할 수 있습니다.

좀 더 구체적으로 설명하면:

1. `df_basetable1.index`:

`df_basetable1` 데이터프레임의 인덱스를 나타냅니다.

이 인덱스는 일반적으로 `DatetimeIndex` 형식일 때 시간 정보를 포함한 날짜-시간 값을 가지고 있습니다.


```

2. `df_basetable1.index.date`:
    각 인덱스 값의 날짜 부분만 추출합니다. 결과는 `numpy` 배열 형식으로 반환되며, 각 요소는 `datetime.date` 객체가 됩니다.
    ...

    ...

아래 코드는 다음과 같은 작업을 수행합니다:
1. `df_basetable1.groupby(df_basetable1.index.date)['TotalHour']`:
    데이터프레임 `df_basetable1`을 날짜별로 그룹화하고, 각 그룹 내에서 'TotalHour' 열을 선택합니다.
2. `.transform(lambda x: x == x.max())`:
    -> x에는 각 그룹별 'TotalHour'열의 값들(Series)이 전달됨
    각 그룹에 대해 최댓값과 동일한 값을 가지는지 여부를 확인하기 위해 람다 함수를 적용합니다.
    이 람다 함수는 각 그룹에서 최댓값과 동일한 값을 가지면 True를 반환하고, 그렇지 않으면 False를 반환합니다.
    이를 통해 각 날짜 그룹에서 최댓값을 가지는 행을 True로, 그 외의 행을 False로 하는 시리즈가 생성됩니다.
3. `df_basetable1.loc[...]`:
    이전 단계에서 생성된 시리즈를 사용하여 원래 데이터프레임에서 행을 선택합니다.
    `loc[]` 인덱서를 사용하여 조건을 만족하는 행만 선택합니다.
4. `[ 'Hour' ]`: 선택된 행 중에서 'Hour' 열을 선택합니다.
5. `.value_counts()`: 선택된 'Hour' 열의 값들을 세어 각 값의 빈도를 계산합니다.
6. `.iloc[:2].index`: 빈도가 가장 높은 두 개의 값을 선택하고, 이들의 인덱스를 반환합니다.
7. `B, C =`: 이렇게 계산된 인덱스 두 개의 값을 변수 `B`와 `C`에 할당합니다.

따라서 아래 코드는 각 날짜에 대해 'TotalHour' 열에서 최댓값을 가지는 시간대의 빈도가
가장 높은 두 개의 시간대를 변수 `B`와 `C`에 저장하는 작업을 수행합니다.
...

# 방법1: # groupby ~ transform에서는 일자로 구분했을 때
# 해당 행의 TotalHour가 일자별 최대값인지 아닌지를 뽑아냅니다.
# groupby에는 컬럼명 뿐만아니라, Series, numpy.array도 전달할 수 있습니다.
# transform에는 groupby에서 지정된 수준으로 구분된 Series가 넘어옵니다.
# 일별 최대값 해당 여부를 transform의 lambda 함수에서 구합니다.
# 이 결과를 boolean index로 사용하여 일별 TotalHour의 최대값에 해당하는 행들만 뽑아오고,
# Hour의 빈도수를 구하여 B,C를 구합니다.

B, C = df_basetable1.loc[df_basetable1.groupby(df_basetable1.index.date)['TotalHour']\
                        .transform(lambda x: x == x.max()), 'Hour']\
        .value_counts().iloc[:2].index

B, C

```

Out[37]: (8, 11)

In [38]: df_basetable1.groupby(df_basetable1.index.date)['TotalHour']

Out[38]: <pandas.core.groupby.generic.SeriesGroupBy object at 0x000002C788842DC8>

```
In [39]: '''
df_basetable1.groupby(df_basetable1.index.date)['TotalHour'].transform(lambda x: print(x))
'''
print('')
```

```
In [40]: type(df_basetable1.groupby(df_basetable1.index.date)['TotalHour'])
```

Out[40]: pandas.core.groupby.generic.SeriesGroupBy

```
In [41]: df_basetable1.groupby(df_basetable1.index.date)['TotalHour'].apply(lambda x: x == x.mean()).value_counts()
```

```
Out[41]: False    6163
         True      5
         Name: TotalHour, dtype: int64
```

```
In [42]: df_basetable1.groupby(df_basetable1.index.date)['TotalHour'].transform(lambda x: x == x.mean()).value_counts()
```

```
Out[42]: False    6163
         True      5
         Name: TotalHour, dtype: int64
```

```
In [43]: '''
아래 코드에서 `lambda x`에서 `x`로 전달되는 것은 각 그룹별 `TotalHour` 열의 값들입니다.

### 코드 설명

1. `df_basetable1.groupby(df_basetable1.index.date)` :
   이 부분은 `df_basetable1` DataFrame을 `index.date`로 그룹화합니다. 즉, 각 날짜별로 그룹을 나누는 것입니다.

2. `['TotalHour']` : 그룹화된 각 그룹에서 `TotalHour` 열만 선택합니다.
   이로 인해 각 그룹은 `TotalHour` 열의 값들로 이루어진 Series가 됩니다.

3. `transform(lambda x: x == x.max())` :
   - `transform` 함수는 그룹화된 각 그룹에 대해 주어진 함수를 적용하고, 원래의 인덱스 구조를 유지합니다.
   - `lambda x: x == x.max()` 부분은 그룹화된 각 `TotalHour` Series에 대해 동작합니다.
     - `x`는 각 그룹별 `TotalHour` 열의 값들 (즉, 각 날짜별 `TotalHour` 값들로 이루어진 Series)입니다.
     - `x.max()`는 해당 그룹의 `TotalHour` 값들 중 최대값을 계산합니다.
     - `x == x.max()`는 각 그룹의 `TotalHour` 값들이 최대값과 같은지 여부를 검사하는 Boolean Series를 반환합니다.
'''
```

여기서 각 날짜별로 `TotalHour`의 최대값을 가진 항목들이 `True`로 표시됩니다.
...

```
df_basetable1.groupby(df_basetable1.index.date)['TotalHour'].transform(lambda x: x == x.max())
```

```
Out[43]: DateHour
2021-01-01 00:00:00    False
2021-01-01 01:00:00    False
2021-01-01 02:00:00    False
2021-01-01 03:00:00    False
2021-01-01 04:00:00     True
...
2021-09-14 19:00:00    False
2021-09-14 20:00:00    False
2021-09-14 21:00:00    False
2021-09-14 22:00:00    False
2021-09-14 23:00:00    False
Name: TotalHour, Length: 6168, dtype: bool
```

```
In [44]: df_basetable1.loc[df_basetable1.groupby(df_basetable1.index.date)['TotalHour'].transform(lambda x: x == x.max()), 'Hour']
```

```
Out[44]: DateHour
2021-01-01 04:00:00     4
2021-01-02 04:00:00     4
2021-01-03 21:00:00    21
2021-01-04 09:00:00     9
2021-01-05 00:00:00     0
..
2021-09-11 01:00:00     1
2021-09-12 14:00:00    14
2021-09-12 21:00:00    21
2021-09-13 08:00:00     8
2021-09-14 08:00:00     8
Name: Hour, Length: 293, dtype: int64
```

```
In [45]: df_basetable1.loc[df_basetable1.groupby(df_basetable1.index.date)['TotalHour'].transform(lambda x: x == x.max()), 'Hour']\
        .value_counts()
```

```
Out[45]: 8      57
          11     48
          13     31
          9      28
          4      26
          23     16
          0      15
          14     14
          10      9
          2       9
          16      8
          17      5
          6       5
          21      5
          12      4
          1       4
          7       3
          15      2
          3       2
          22      1
          18      1
          Name: Hour, dtype: int64
```

```
In [46]: df_basetable1.groupby(df_basetable1.index.date)['TotalHour'].apply(lambda x: x == x.max())
```

```
Out[46]: DateHour
2021-01-01 00:00:00    False
2021-01-01 01:00:00    False
2021-01-01 02:00:00    False
2021-01-01 03:00:00    False
2021-01-01 04:00:00     True
...
2021-09-14 19:00:00    False
2021-09-14 20:00:00    False
2021-09-14 21:00:00    False
2021-09-14 22:00:00    False
2021-09-14 23:00:00    False
          Name: TotalHour, Length: 6168, dtype: bool
```

```
In [47]: # apply() 함수
...
`apply()` 함수는 판다스( `pandas` ) 라이브러리에서 제공되는 데이터프레임 또는 시리즈 객체에 사용되는 함수 중 하나입니다.
```

이 함수는 각 행 또는 열에 함수를 적용하여 데이터를 변환하고, 그 결과를 반환합니다.
이 함수는 데이터프레임 또는 시리즈 객체에 사용될 수 있습니다.

`apply()` 함수의 구문

```
DataFrame.apply(func, axis=0, raw=False, result_type=None, args=(), **kwargs)
```

- `func`:

적용할 함수입니다. 이 함수는 시리즈나 데이터프레임을 입력으로 받아 처리하고, 결과를 반환합니다.

- `axis`:

0 또는 'index'를 지정하면 각 열에 대해 함수가 적용됩니다.

1 또는 'columns'를 지정하면 각 행에 대해 함수가 적용됩니다. 기본값은 0입니다.

- `raw`:

True일 경우 함수에 시리즈 객체를 전달합니다. False일 경우 데이터프레임 객체를 전달합니다. 기본값은 False입니다.

- `result\_type`:

반환할 객체의 유형을 지정합니다. None(기본값), 'expand'(시리즈를 데이터프레임으로 확장),
'broadcast'(시리즈를 브로드캐스트) 중 하나를 선택할 수 있습니다.

- `args`: 함수에 전달할 추가 인수들을 튜플로 지정합니다.

- `\*\*kwargs`: 키워드 인수들을 지정합니다.

반환값

- `DataFrame` 또는 `Series`:

함수를 적용한 결과를 반환합니다. 반환된 객체는 입력된 데이터프레임 또는 시리즈와 동일한 형태를 가집니다.

`apply()` 함수의 동작

`apply()` 함수는 다음과 같은 단계로 동작합니다:

1. 데이터프레임 또는 시리즈 객체를 행 또는 열 단위로 나눕니다.
2. 지정된 함수를 각 행 또는 열에 적용합니다.
3. 함수의 결과를 모아서 반환합니다.

예제

```
import pandas as pd
```

```
# 예시 데이터프레임 생성
```

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
```

```
# 함수 정의
```

```
def my_func(x):
    return x ** 2
```

```
# apply 함수를 사용하여 함수 적용
```

```
result = df.apply(my_func)
print(result)
```

출력:

```

      A  B
0  1  16
1  4  25
2  9  36

```

이 예제에서는 함수 `my_func`를 정의하고, 데이터프레임 `df`의 각 요소에 이 함수를 적용하여 제곱값을 계산합니다. `apply()` 함수를 사용하여 각 열에 대해 이 함수를 적용하고, 결과를 반환합니다.

```
...
```

```
...
```

코드는 다음과 같은 작업을 수행합니다:

1. `df_basetable1.groupby(df_basetable1.index.date)['TotalHour']`: 데이터프레임 `df_basetable1`을 날짜별로 그룹화합니다. 그 후, 각 그룹에서 `'TotalHour'` 열을 선택합니다.
2. `.apply(lambda x: x == x.max())`: 각 날짜 그룹에서 `'TotalHour'` 열 값이 해당 그룹의 최댓값과 같은지를 판별하는 함수를 적용합니다. 즉, 각 그룹에서 `'TotalHour'` 열 값이 최댓값인 행은 `True`로, 그 외의 행은 `False`로 반환됩니다.
3. `df_basetable1.loc[...]`: 이전 단계에서 반환된 `True` 또는 `False` 값들을 사용하여 원래 데이터프레임에서 행을 선택합니다. `loc[]` 인덱서를 사용하여 조건을 만족하는 행만 선택합니다.
4. `['Hour']`: 선택된 행 중에서 `'Hour'` 열을 선택합니다.
5. `.value_counts()`: 선택된 `'Hour'` 열의 값들을 세어 각 값의 빈도를 계산합니다.
6. `.iloc[:2].index`: 빈도가 가장 높은 두 개의 값을 선택하고, 이들의 인덱스를 반환합니다.
7. `B, C =`: 이렇게 계산된 인덱스 두 개의 값을 변수 `B`와 `C`에 할당합니다.

따라서 이 코드는 각 날짜에 대해 `'TotalHour'` 열에서 최댓값을 가지는 시간대의 빈도가 가장 높은 두 개의 시간대를 변수 `B`와 `C`에 저장하는 작업을 수행합니다.

```
...
```

#아래 코드는 아나콘다 가상환경에서는 에러가 발생함, 시험환경 가상환경에서는 잘 동작함

방법2: # groupby ~ apply를 사용합니다.

apply는 DataFrame/Series에 대한 N:1 연산 1:1 연산, N:M 연산 모두 지원합니다.

apply는 groupby 다음에 Series를 취할 경우에는 Series가 넘어오고,

여러개의 행을 취하여 DataFrame을 취한다면 DataFrame이 넘어옵니다.

```

B, C = df_basetable1.loc[df_basetable1.groupby(df_basetable1.index.date)['TotalHour'].apply(lambda x: x == x.max()), 'Hour']\
      .value_counts().iloc[:2].index
B, C

```

Out[47]: (8, 11)

In [48]: # 일단 무시할 것!!!

...

위 코드에서 차이점은 `transform`과 `apply` 메서드를 사용하고 있다는 점입니다.

이 두 메서드는 `groupby`와 함께 사용할 때 서로 다른 동작을 수행합니다.

`transform` vs `apply`

1. `transform`:

- `transform` 메서드는 그룹별로 함수를 적용하여 원래의 `DataFrame`과 동일한 크기의 `Series`를 반환합니다.
- `transform`을 사용하면 각 그룹에 대해 함수가 적용되고, 결과는 원래의 `DataFrame`과 동일한 인덱스를 갖습니다.

2. `apply`:

- `apply` 메서드는 그룹별로 함수를 적용하여, 함수의 반환값에 따라 결과가 달라질 수 있습니다.
- `apply`는 함수가 `DataFrame`을 반환하면 `DataFrame`을 반환하고, `Series`를 반환하면 `Series`를 반환합니다.
- 따라서 `apply`를 사용하면 반환된 객체의 크기나 형식이 원래의 `DataFrame`과 다를 수 있습니다.

코드 설명 및 문제

첫 번째 코드 (정상 동작):

- `groupby(df_basetable1.index.date)['TotalHour'].transform(lambda x: x == x.max())`:`
- 각 날짜별로 `TotalHour`의 최대값을 구하고, 원래의 `DataFrame`과 동일한 크기의 `boolean Series`를 반환합니다.
- `df_basetable1.loc[..., 'Hour']`:`
- `boolean Series`가 `True`인 행의 `Hour` 값을 선택합니다.
- `.value_counts().iloc[:2].index`:`
- 가장 많이 등장하는 두 개의 `Hour` 값을 선택합니다.

두 번째 코드 (에러 발생):

- `groupby(df_basetable1.index.date)['TotalHour'].apply(lambda x: x == x.max())`:`
- 각 날짜별로 `TotalHour`의 최대값을 구하지만, 반환된 결과가 원래의 `DataFrame`과 동일한 크기의 `Series`가 아닐 수 있습니다.
- `apply`는 각 그룹별로 최대값과 비교한 `boolean Series`를 반환하지만, 이 `Series`가 원래 `DataFrame`과 동일한 인덱스를 갖지 않을 수 있습니다.
- 그래서 `loc`을 사용하여 인덱싱하려 할 때 문제가 발생합니다.

에러 해결 방법

`apply` 대신 `transform`을 사용하여 원래 `DataFrame`과 동일한 크기의 `Series`를 반환하도록 해야 합니다.

이미 정상 동작하는 첫 번째 코드처럼 `transform`을 사용하면 문제를 해결할 수 있습니다:

또는 아래와 같이 수정해야 합니다.

...

print('')

```
In [49]: # 일단 무시할 것!!!
'''
`apply`를 사용할 때에도 문제를 해결할 수 있는 방법이 있습니다.
`apply`를 사용하여 각 그룹별로 최대값을 구한 후, 원래 DataFrame의 인덱스를 유지하면서
이를 원래 DataFrame에 맞추는 방식으로 접근할 수 있습니다.

1. `apply`를 사용하여 각 그룹별로 최대값을 구하고, boolean Series를 반환합니다.
2. 이 boolean Series를 원래 DataFrame의 인덱스에 맞추어 정렬합니다.

아래 코드는 다음과 같은 작업을 수행합니다:

1. `groupby(df_basetable1.index.date)['TotalHour'].apply(lambda x: x == x.max())`는
   각 날짜별로 `TotalHour`의 최대값을 구하고, boolean Series를 반환합니다.
   이 Series는 원래 DataFrame과 같은 인덱스를 갖지 않을 수 있습니다.
2. `max_mask.index = max_mask.index.droplevel(0)`는 반환된 boolean Series의 인덱스를
   원래 DataFrame의 인덱스와 맞추기 위해 첫 번째 레벨을 제거합니다.
3. `df_basetable1.loc[max_mask, 'Hour']`는 boolean Series가 `True`인 행의 'Hour' 값을 선택합니다.
4. `value_counts().iloc[:2].index`를 사용하여 가장 많이 등장하는 두 개의 'Hour' 값을 선택합니다.

이 방식으로 `apply`를 사용하여 원하는 결과를 얻을 수 있습니다.
'''
print('')

# 아래 코드는 시험환경 가상환경에서 에러가 발생함, 아나콘다 가상환경에서는 잘 동작함

# 각 날짜별로 TotalHour의 최대값을 구하여 boolean Series 반환
#max_mask = df_basetable1.groupby(df_basetable1.index.date)['TotalHour'].apply(lambda x: x == x.max())

# 반환된 boolean Series의 인덱스를 원래 DataFrame에 맞추어 정렬
#max_mask.index = max_mask.index.droplevel(0) # 멀티 인덱스의 첫 번째 레벨 제거

# 'Hour' 열에서 최대값을 가진 행들을 선택하여 값의 개수 계산
#B, C = df_basetable1.loc[max_mask, 'Hour'].value_counts().iloc[:2].index
#B, C
```

Tip groupby ~ apply vs ~ transform

```
In [50]: # 여기서부터는 참고용 -> 단계 1-3 앞까지
```



```
In [51]: '''
코드에서 `x`는 `df_basetable1.index.date`로 그룹화된 각 날짜에 해당하는 'TotalHour' 컬럼의 데이터를 의미합니다.
그런 다음 `lambda` 함수 내에서 `x.mean()`을 통해 각 그룹의 'TotalHour' 값의 평균을 계산하게 됩니다.
'''

# groupby apply는 N:1, 1:1, N:M 연산 모두 가능합니다.
# 처리 함수에서 1개를 반환하면 하나의 값으로 집계가 됩니다.
df_basetable1.groupby(df_basetable1.index.date)['TotalHour'].apply(lambda x: x.mean())
```

```
Out[51]: 2021-01-01    184.708333
2021-01-02    185.041667
2021-01-03     91.083333
2021-01-04    485.791667
2021-01-05     87.750000
...
2021-09-10    536.250000
2021-09-11    171.333333
2021-09-12     92.833333
2021-09-13    443.208333
2021-09-14    529.958333
Name: TotalHour, Length: 257, dtype: float64
```

```
In [52]: df_basetable1.groupby(df_basetable1.index.date)['TotalHour'].agg('mean')
```

```
Out[52]: 2021-01-01    184.708333
2021-01-02    185.041667
2021-01-03     91.083333
2021-01-04    485.791667
2021-01-05     87.750000
...
2021-09-10    536.250000
2021-09-11    171.333333
2021-09-12     92.833333
2021-09-13    443.208333
2021-09-14    529.958333
Name: TotalHour, Length: 257, dtype: float64
```

```
In [53]: '''
코드는 `df_basetable1` 데이터프레임에서 각 날짜별로 'TotalHour' 열의 값이
해당 날짜 그룹의 평균값과 같은 행들을 선택하는 작업을 수행합니다.
1. `df_basetable1.groupby(df_basetable1.index.date)`:
```

```

데이터프레임 `df_basetable1`을 인덱스의 날짜 부분을 기준으로 그룹화합니다.
`df_basetable1.index.date`는 각 인덱스의 날짜 부분만 추출합니다.
2. `[ 'TotalHour'].apply(lambda x: x == x.mean())`:
각 그룹에 대해 'TotalHour' 열을 선택하고, `apply()` 함수를 사용하여 람다 함수를 적용합니다.
이 람다 함수는 각 그룹 내에서 'TotalHour' 값이 해당 그룹의 평균값과 같은지를 확인합니다.
그룹 내 각 행에 대해 'TotalHour' 값이 평균값과 같으면 `True`, 그렇지 않으면 `False`를 반환합니다.
3. `df_basetable1.loc[...]`:
`loc[]` 인덱서를 사용하여 위에서 구한 `True` 또는 `False` 값을 사용해 행을 선택합니다.
`True`인 행만 선택되어 최종 결과로 반환됩니다.
...

...

해당 예러는 `groupby`와 `apply`의 결과로 반환된 Boolean Series의 인덱스가 원래 DataFrame의 인덱스와 맞지 않기 때문에 발생합니다.
`apply`를 사용하면 그룹 단위로 연산이 수행되기 때문에 결과의 인덱스가 그룹의 인덱스와 일치하지 않게 됩니다.

# 수정 방법
`transform`을 사용하면 `apply`와 달리 원래의 인덱스를 유지할 수 있어 이 문제를 해결할 수 있습니다.
`transform`은 각 그룹별로 연산을 수행하고, 원래의 인덱스 구조를 유지하면서 결과를 반환합니다.
...

#아래 코드는 아나콘다 가상환경에서는 예러가 발생함, 시험환경 가상환경에서는 잘 동작함

# 처리 함수에서 행별로 반환하면 1:1 변환으로도 사용할 수 있습니다.
df_basetable1.loc[df_basetable1.groupby(df_basetable1.index.date)['TotalHour'].apply(lambda x: x == x.mean())]

```

Out[53]:

	15 분	30 분	45 분	60 분	DayName	Hour	AM	Weekend_yn	Holiday_yn	Avg	TotalHour	생 산 량	기 온	풍 속	습 도	강수 량	전기요금 (계절)	공장 인원	인건 비
DateHour																			
2021-07-04 12:00:00	24	23	23	23	6	12	1	1	0	23.25	93	0	30.6	4.0	58	24.7	191.6	0.0	1.0
2021-07-04 13:00:00	23	24	23	23	6	13	1	1	0	23.25	93	0	31.3	3.4	59	24.7	191.6	0.0	1.0
2021-07-04 22:00:00	23	24	23	23	6	22	1	1	0	23.25	93	0	23.1	0.8	97	24.9	191.6	0.0	1.5
2021-08-06 16:00:00	21	22	22	22	4	16	1	0	0	21.75	87	0	31.3	2.9	66	0.0	191.6	0.0	1.0
2021-08-06 18:00:00	22	21	22	22	4	18	1	0	0	21.75	87	0	29.7	1.3	78	0.0	191.6	0.0	1.5

In [54]:

```
'''
코드는 `df_basetable1` 데이터프레임에서 각 날짜별로 'TotalHour' 값이 해당 날짜 그룹의 평균값과 같은 행들을 선택하는 작업을 수행합니다.

1. 날짜별로 그룹화:
`df_basetable1` 데이터프레임의 인덱스가 날짜-시간 형식일 때, `.index.date`를 사용하여 날짜 부분만 추출합니다.
이로 인해 데이터프레임이 각 날짜별로 그룹화됩니다.

2. 그룹화된 각 날짜별 'TotalHour' 계산:
['TotalHour']
그룹화된 각 날짜별로 'TotalHour' 열을 선택합니다.

3. 각 그룹에서 평균값과 같은지 확인:
.transform(lambda x: x == x.mean())
`transform` 함수는 각 그룹에서 지정된 함수를 적용한 결과를 원래 데이터프레임의 인덱스에 맞게 반환합니다.
여기서는 람다 함수 `lambda x: x == x.mean()`를 사용하여 각 그룹에서 'TotalHour' 값이 그룹의 평균값과 같은지를 확인합니다.
- `x == x.mean()`는 각 그룹의 'TotalHour' 값이 그 그룹의 평균값과 같은지를 확인하여 `True` 또는 `False` 값을 반환합니다.
- `transform` 함수는 이 불리언 값을 원래 인덱스에 맞게 확장하여 반환합니다.
즉, 원래 데이터프레임과 동일한 길이의 불리언 시리즈가 생성됩니다.

4. 불리언 인덱싱으로 행 선택:
df_basetable1.loc[...]
`loc` 인덱서를 사용하여 `transform` 함수가 반환한 불리언 시리즈를 통해 행을 선택합니다.
'''
```

``True`` 값인 행들만 선택됩니다.

전체 동작 과정

1. ``df\_basetable1`` 데이터프레임을 날짜별로 그룹화합니다.
2. 각 그룹에 대해 ``TotalHour`` 열의 값을 가져옵니다.
3. 각 그룹 내에서 ``TotalHour`` 값이 해당 그룹의 평균값과 같은지 확인합니다.
결과는 원래 데이터프레임과 동일한 길이의 불리언 시리즈가 됩니다.
4. 이 불리언 시리즈를 사용하여 ``df\_basetable1`` 데이터프레임에서 ``True`` 값인 행들만 선택합니다.

```
df_basetable1.loc[df_basetable1.groupby(df_basetable1.index.date)['TotalHour'].transform(lambda x: x == x.mean())]
```

Out[54]:

	15 분	30 분	45 분	60 분	DayName	Hour	AM	Weekend_yn	Holiday_yn	Avg	TotalHour	생 산 량	기 온	풍 속	습 도	강수 량	전기요금 (계절)	공장 인원	인건 비				
DateHour																							
2021-07-04 12:00:00	24	23	23	23		6	12	1		1		0	23.25		93	0	30.6	4.0	58	24.7	191.6	0.0	1.0
2021-07-04 13:00:00	23	24	23	23		6	13	1		1		0	23.25		93	0	31.3	3.4	59	24.7	191.6	0.0	1.0
2021-07-04 22:00:00	23	24	23	23		6	22	1		1		0	23.25		93	0	23.1	0.8	97	24.9	191.6	0.0	1.5
2021-08-06 16:00:00	21	22	22	22		4	16	1		0		0	21.75		87	0	31.3	2.9	66	0.0	191.6	0.0	1.0
2021-08-06 18:00:00	22	21	22	22		4	18	1		0		0	21.75		87	0	29.7	1.3	78	0.0	191.6	0.0	1.5

In [55]:

```
'''
```

코드는 다음과 같은 작업을 수행합니다:

1. 인덱스의 날짜 부분 추출:
``df\_basetable1.index.date``는 ``df\_basetable1``의 인덱스에서 날짜 부분만 추출합니다.
인덱스가 ``DatetimeIndex``인 경우, ``date`` 속성을 사용하면 각 타임스탬프의 날짜 부분만 추출됩니다.
2. 그룹화:
``df\_basetable1.groupby(df\_basetable1.index.date)``는 인덱스의 날짜를 기준으로 그룹화합니다.
즉, 동일한 날짜를 가지는 행들이 하나의 그룹으로 묶입니다.
3. 열 선택:

```

`['TotalHour']`는 `df_basetable1`에서 `TotalHour` 열을 선택합니다.
각 그룹마다 `TotalHour` 열의 값들만 선택됩니다.
4. transform 적용:
`transform(lambda x: x.mean())`는 각 그룹의 `TotalHour` 값들에 대해 `lambda` 함수를 적용합니다.
이 경우 `lambda x: x.mean()`이므로, 각 그룹의 `TotalHour` 값들의 평균을 계산합니다.
그러나 `transform`은 그룹의 각 원소에 대해 그룹의 평균 값을 반환합니다.
즉, 원래의 데이터프레임과 동일한 크기의 시리즈를 반환하되, 각 원소는 해당 원소가 속한 그룹의 평균 값으로 채워집니다.
5. 슬라이싱:
`iloc[:5]`는 결과 시리즈의 처음 5개 값을 선택합니다.
여기서 각 날짜별로 `TotalHour`의 평균을 계산하고, 그 값을 원래 데이터프레임의 각 원소에 대해 반환합니다.
결과 시리즈의 처음 5개 값을 선택하여 출력합니다.

### 요약
이 코드는 `df_basetable1`을 날짜별로 그룹화한 후, 각 그룹의 `TotalHour` 값들의 평균을 계산하여
원래 데이터프레임과 동일한 크기의 시리즈로 반환합니다.
`iloc[:5]`는 결과 시리즈의 처음 5개 값을 선택하여 반환합니다.
...

# groupby transform은 1:1 변환만 이루어집니다.
# 처리 함수에서 하나의 값을 반환하면 동일한 그룹의
# 각각의 Series가 반환한 값으로 변환된 결과를 얻게 됩니다.
# 하나의 값으로 집계를 낸다면 동일한 그룹의 행들을 동일한 값을 지니는 형태로 값이 나옵니다.
df_basetable1.groupby(df_basetable1.index.date)['TotalHour'].transform(lambda x: x.mean()).iloc[:5]

```

```

Out[55]: DateHour
2021-01-01 00:00:00    184.708333
2021-01-01 01:00:00    184.708333
2021-01-01 02:00:00    184.708333
2021-01-01 03:00:00    184.708333
2021-01-01 04:00:00    184.708333
Name: TotalHour, dtype: float64

```

```

In [56]: df_basetable1[['TotalHour', 'Avg']].head()

```

Out[56]:

	TotalHour	Avg
DateHour		
2021-01-01 00:00:00	245	61.25
2021-01-01 01:00:00	418	104.50
2021-01-01 02:00:00	415	103.75
2021-01-01 03:00:00	421	105.25
2021-01-01 04:00:00	427	106.75

In [57]: `type(df_basetable1[['TotalHour', 'Avg']].head())`Out[57]: `pandas.core.frame.DataFrame`In [58]: `'''`

코드는 `df_basetable1` 데이터프레임을 날짜별로 그룹화하고, 각 그룹에 대해 `TotalHour`와 `Avg` 열을 선택하여 `apply` 함수를 적용합니다. 구체적으로 어떤 타입의 데이터가 각 그룹에 전달되는지, 그리고 각 그룹의 이름이 무엇인지 확인하는 작업을 수행합니다.

1. 인덱스의 날짜 부분 추출:

```
df_basetable1.index.date
```

`df_basetable1`의 인덱스에서 날짜 부분만 추출합니다. 인덱스가 `DatetimeIndex`인 경우, `date` 속성을 사용하여 각 타임스탬프의 날짜 부분만 추출합니다.

2. 그룹화:

```
df_basetable1.groupby(df_basetable1.index.date)
```

인덱스의 날짜를 기준으로 `df_basetable1`을 그룹화합니다. 즉, 동일한 날짜를 가지는 행들이 하나의 그룹으로 묶입니다.

3. 열 선택:

```
[['TotalHour', 'Avg']]
```

`df_basetable1`에서 `TotalHour`와 `Avg` 열을 선택합니다. 각 그룹마다 이 두 열의 값들만 선택됩니다.

4. apply 함수 적용:

```
.apply(lambda x: (type(x), x.name))
```

`apply` 함수는 각 그룹에 대해 람다 함수를 적용합니다.

람다 함수 `lambda x: (type(x), x.name)`는 각 그룹 `x`의 타입과 그룹의 이름을 반환합니다.

- `type(x)`는 각 그룹이 `DataFrame` 타입임을 반환합니다.

- `x.name`은 그룹의 이름을 반환합니다. 그룹의 이름은 각 그룹의 키인 날짜입니다.

5. 결과의 일부분 추출:

```
.iloc[:5]
```

최종적으로, 첫 5개의 결과를 추출합니다.

```

전체 코드를 실행한 결과는 각 날짜별로 그룹화된 `DataFrame`의 타입과 이름을 포함하는 시리즈의 처음 5개 요소가 됩니다.
'''

'''
,index,0
0,2021-01-01,"(<class 'pandas.core.frame.DataFrame'>, datetime.date(2021, 1, 1))"
1,2021-01-02,"(<class 'pandas.core.frame.DataFrame'>, datetime.date(2021, 1, 2))"
2,2021-01-03,"(<class 'pandas.core.frame.DataFrame'>, datetime.date(2021, 1, 3))"
3,2021-01-04,"(<class 'pandas.core.frame.DataFrame'>, datetime.date(2021, 1, 4))"
4,2021-01-05,"(<class 'pandas.core.frame.DataFrame'>, datetime.date(2021, 1, 5))"
'''

# groupby[컬럼리스트] apply는 처리함수에 groupby로 구분되어진 DataFrame이 넘어오고,
# DataFrame name에는 DataFrame이 속하는 수준이 담겨 있습니다.
df_basetable1.groupby(df_basetable1.index.date)[['TotalHour', 'Avg']].apply(lambda x: (type(x), x.name)).iloc[:5]

```

```

Out[58]: 2021-01-01    (<class 'pandas.core.frame.DataFrame'>, 2021-0...
        2021-01-02    (<class 'pandas.core.frame.DataFrame'>, 2021-0...
        2021-01-03    (<class 'pandas.core.frame.DataFrame'>, 2021-0...
        2021-01-04    (<class 'pandas.core.frame.DataFrame'>, 2021-0...
        2021-01-05    (<class 'pandas.core.frame.DataFrame'>, 2021-0...
        dtype: object

```

```

In [59]: res = df_basetable1.groupby(df_basetable1.index.date)[['TotalHour', 'Avg']].apply(lambda x: (type(x), x.name))
        type(res)

```

```

Out[59]: pandas.core.series.Series

```

```

In [60]: '''
`apply`와 `transform`은 각기 다른 방식으로 작동하며, 이로 인해 `transform`을 사용할 때
`ValueError`가 발생하는 이유를 설명하고 해결 방법을 제시하겠습니다.

### 문제 설명
`apply`는 각 그룹에 대해 연산을 수행하고 결과를 반환합니다.
이때, 반환되는 결과는 원래 데이터프레임과 크기와 인덱스가 일치하지 않을 수 있습니다.
반면, `transform`은 원래 데이터프레임과 동일한 크기와 인덱스를 유지하면서 변환을 수행해야 합니다.
`transform` 함수에서 반환된 결과의 크기와 인덱스가 원래 데이터프레임과 일치하지 않으면 에러가 발생합니다.

### 에러 원인
`transform` 함수에 사용된 람다 함수가 각 그룹의 Series 객체의 타입과 이름을 튜플로 반환하면서
크기와 인덱스가 맞지 않기 때문에 발생하는 문제입니다.

```

추가 설명

`apply`와 `transform`의 차이점:

- `apply`: 각 그룹에 대해 연산을 수행하고, 결과를 그룹 단위로 반환합니다.
반환되는 결과의 크기와 인덱스가 원래 데이터프레임과 일치하지 않을 수 있습니다.
- `transform`: 각 그룹에 대해 연산을 수행하고, 결과를 원래 데이터프레임과 동일한 크기와 인덱스로 반환합니다. 각 그룹의 **Series**를 변환한 후, 원래 데이터프레임의 크기와 인덱스를 유지해야 합니다.

따라서, `transform`을 사용할 때는 반환 값이 각 그룹의 **Series**와 동일한 크기여야 하며, 그렇지 않으면 `ValueError`가 발생합니다.

...

`transform` 함수에 사용된 람다 함수가 반환하는 값과 그로 인해 발생하는 에러에 대해 자세히 설명하겠습니다.

코드 분석

```
df_basetable1.groupby(df_basetable1.index.date)[['TotalHour', 'Avg']].transform(lambda x: (type(x), x.name)).iloc[:5]
```

이 코드에서 `transform` 함수는 각 그룹의 **Series**에 대해 `(type(x), x.name)` 튜플을 반환합니다.

`transform` 함수는 각 그룹에 대해 연산을 수행하고 원래 데이터프레임과 동일한 크기와 인덱스를 유지하는 **Series**를 반환해야 합니다. 그러나 람다 함수가 반환하는 `(type(x), x.name)`는 이 조건을 충족하지 않습니다.

람다 함수의 반환 값

람다 함수가 반환하는 값은 각 그룹의 **Series** 객체의 타입과 이름을 담은 튜플입니다.

예를 들어, 한 그룹의 `TotalHour` **Series**에 대해 다음과 같은 값을 반환할 수 있습니다:

- `type(x)`는 `>`
- `x.name`는 `TotalHour`

따라서, 람다 함수는 각 그룹에 대해 다음과 같은 튜플을 반환합니다:

```
(<class 'pandas.core.series.Series'>, 'TotalHour')
```

에러 발생 원인

`transform` 함수는 각 그룹에 대해 연산을 수행한 결과를 원래 데이터프레임과 동일한 크기와 인덱스를 유지하면서 반환해야 합니다.

하지만, 람다 함수가 반환하는 값은 하나의 튜플입니다.

즉, 각 그룹에 대해 단일 튜플을 반환하므로 원래 데이터프레임의 각 행에 대해 동일한 길이의 **Series**를 반환하지 못합니다.

이로 인해 `transform` 함수는 결과를 원래 데이터프레임에 맞게 확장할 수 없으며, `ValueError`가 발생합니다.

에러 메시지 설명

에러 메시지는 다음과 같습니다:

```
ValueError: Length mismatch: Expected axis has 6682 elements,
new values have 6168 elements
```

이 메시지는 원래 데이터프레임에는 6682개의 요소가 있지만, 람다 함수가 반환하는 결과는 6168개의 요소만 가지고 있음을 의미합니다.

이는 각 그룹에 대해 단일 튜플을 반환하기 때문에 발생하는 문제입니다.

해결 방법

`transform` 함수는 각 그룹의 Series에 대해 동일한 길이의 Series를 반환해야 합니다.
예를 들어, 각 그룹의 `TotalHour`의 평균을 계산하려면 다음과 같이 작성할 수 있습니다:

각 그룹의 'TotalHour'와 'Avg'의 평균을 계산합니다.

```
result = df_basetable1.groupby(df_basetable1.index.date)[['TotalHour', 'Avg']].transform(lambda x: x.mean())
```

결과를 확인합니다.

```
print(result.iloc[:5])
```

위 코드에서는 각 그룹의 `TotalHour`와 `Avg`에 대해 평균값을 계산하여 원래 데이터프레임과 동일한 크기와 인덱스를 유지하면서 반환합니다.

요약

`transform` 함수는 원래 데이터프레임과 동일한 크기와 인덱스를 유지하는 Series를 반환해야 합니다.

람다 함수가 단일 튜플을 반환하면 이 조건을 충족하지 못하므로 에러가 발생합니다.

따라서, `transform`을 사용할 때는 각 그룹에 대해 동일한 길이의 Series를 반환해야 합니다.

...

#아래 코드는 아나콘다 가상환경에서는 에러가 발생함, 시험환경 가상환경에서는 잘 동작함

반면, groupby[컬럼리스트] transform은 처리함수에 groupby로 구분되어진

각각의 Series들이 넘어오고, Series name에는 Series의 컬럼이 담겨있게됩니다.

```
df_basetable1.groupby(df_basetable1.index.date)[['TotalHour', 'Avg']].transform(lambda x: (type(x), x.name)).iloc[:5]
```

Out[60]:

	TotalHour	Avg
DateHour		
2021-01-01 00:00:00	(<class 'pandas.core.series.Series'>, TotalHour)	(<class 'pandas.core.series.Series'>, Avg)
2021-01-01 01:00:00	(<class 'pandas.core.series.Series'>, TotalHour)	(<class 'pandas.core.series.Series'>, Avg)
2021-01-01 02:00:00	(<class 'pandas.core.series.Series'>, TotalHour)	(<class 'pandas.core.series.Series'>, Avg)
2021-01-01 03:00:00	(<class 'pandas.core.series.Series'>, TotalHour)	(<class 'pandas.core.series.Series'>, Avg)
2021-01-01 04:00:00	(<class 'pandas.core.series.Series'>, TotalHour)	(<class 'pandas.core.series.Series'>, Avg)

```
In [61]: res = df_basetable1.groupby(df_basetable1.index.date)[['TotalHour', 'Avg']].transform(lambda x: (type(x), x.name))
         type(res)
```

Out[61]: pandas.core.frame.DataFrame

In [62]: *# 일단 시험환경에서는 아래 내용 볼 필요 없음*

...

`transform` 함수는 각 그룹별로 변환된 결과를 반환할 때 원래 `DataFrame`과 동일한 길이와 인덱스를 유지해야 합니다. 이것은 `transform` 함수가 각 그룹의 변환 결과를 원래 데이터의 각 행에 매핑할 수 있도록 하기 위해 필요합니다.

예를 들어, `transform` 함수에 전달된 람다 함수가 각 그룹에서 평균을 계산하면, 각 그룹의 모든 행에 대해 그 평균 값을 반환하여 원래 `DataFrame`과 동일한 길이의 `Series`를 생성합니다.

하지만 람다 함수 `lambda x: (type(x), x.name)`는 각 그룹에 대해 단일 튜플 `(type(x), x.name)`을 반환합니다. 이는 원래 `DataFrame`의 길이와 일치하지 않으며, `transform` 함수의 요구사항을 충족하지 못합니다. 그래서 에러가 발생합니다.

여기서 말하는 "단일 튜플"은 각 그룹에 대해 하나의 튜플을 반환한다는 의미입니다.

더 자세히 설명하자면, 람다 함수 `lambda x: (type(x), x.name)`은 각 그룹에 대해 다음과 같은 형식의 튜플을 반환합니다:

```
(type(x), x.name)
```

이 튜플은 그룹의 타입과 그룹 이름을 포함합니다.

예를 들어, `type(x)`는 그룹의 데이터 타입 (여기서는 `pandas.core.series.Series`), `x.name`은 그룹 이름 (여기서는 날짜)입니다.

`apply` 함수는 각 그룹에 대해 처리된 결과를 반환합니다. 예를 들어:

```
result_apply = df.groupby(df.index.date).apply(lambda x: (type(x), x.name))
print(result_apply)
```

결과는 각 그룹에 대해 하나의 튜플을 반환합니다:

```
2021-01-01    (<class 'pandas.core.frame.DataFrame'>, 2021-01-01)
2021-01-02    (<class 'pandas.core.frame.DataFrame'>, 2021-01-02)
2021-01-03    (<class 'pandas.core.frame.DataFrame'>, 2021-01-03)
2021-01-04    (<class 'pandas.core.frame.DataFrame'>, 2021-01-04)
2021-01-05    (<class 'pandas.core.frame.DataFrame'>, 2021-01-05)
2021-01-06    (<class 'pandas.core.frame.DataFrame'>, 2021-01-06)
2021-01-07    (<class 'pandas.core.frame.DataFrame'>, 2021-01-07)
2021-01-08    (<class 'pandas.core.frame.DataFrame'>, 2021-01-08)
2021-01-09    (<class 'pandas.core.frame.DataFrame'>, 2021-01-09)
2021-01-10    (<class 'pandas.core.frame.DataFrame'>, 2021-01-10)
dtype: object
```

여기서 각 날짜 그룹에 대해 하나의 튜플이 반환됩니다.

`transform` 함수는 각 그룹별 변환된 결과를 원래 `DataFrame`의 각 행에 매핑합니다. 예를 들어:

```
result_transform = df.groupby(df.index.date)[['TotalHour', 'Avg']].transform(lambda x: x.mean())
print(result_transform)
```

결과는 원래 DataFrame과 동일한 길이와 인덱스를 가진 Series를 반환합니다:

	TotalHour	Avg
2021-01-01	1.0	1.5
2021-01-02	2.0	2.5
2021-01-03	3.0	3.5
2021-01-04	4.0	4.5
2021-01-05	5.0	5.5
2021-01-06	6.0	6.5
2021-01-07	7.0	7.5
2021-01-08	8.0	8.5
2021-01-09	9.0	9.5
2021-01-10	10.0	10.5

에러 원인

`transform` 함수에 `lambda x: (type(x), x.name)`을 사용하면:

각 그룹에 대해 `(type(x), x.name)`라는 단일 튜플을 반환하게 되며, 이는 각 그룹에 대해 하나의 값만 반환하는 것입니다.

그러나 `transform` 함수는 각 그룹의 각 행에 대해 변환된 값을 반환해야 하므로, 반환된 결과가 원래 DataFrame의 길이와 일치하지 않게 됩니다. 이로 인해 크기 불일치 오류가 발생합니다.

요약

- 단일 튜플 반환: `lambda x: (type(x), x.name)`은 각 그룹에 대해 하나의 튜플을 반환합니다.
- transform의 요구사항: `transform` 함수는 각 그룹의 각 행에 대해 변환된 값을 반환해야 하며, 반환된 결과는 원래 DataFrame의 길이와 일치해야 합니다.
- 크기 불일치 오류: 각 그룹에 대해 단일 튜플을 반환하면, 결과의 길이가 원래 DataFrame과 일치하지 않게 되어 오류가 발생합니다.

...

"각 그룹"이라고 한 것은 `groupby` 함수가 DataFrame을 그룹화할 때 생성되는 각 그룹을 의미합니다.

그룹화는 지정한 기준 (여기서는 `df.basetable1.index.date`로 날짜별 그룹화)으로 DataFrame을 여러 작은 그룹으로 나누는 작업입니다.

DataFrame을 날짜별로 그룹화하면:

```
grouped = df.groupby(df.index.date)
```

이 코드는 날짜별로 그룹화된 DataFrameGroupBy 객체를 생성합니다.

그룹화의 결과

이 그룹화의 결과로 생성되는 각 그룹은 특정 날짜에 해당하는 행들로 구성됩니다.

예를 들어, 그룹화된 객체는 다음과 같은 그룹들을 포함하게 됩니다:

- 2021-01-01 그룹: 이 그룹에는 인덱스가 2021-01-01인 행이 포함됩니다.
- 2021-01-02 그룹: 이 그룹에는 인덱스가 2021-01-02인 행이 포함됩니다.
- 2021-01-03 그룹: 이 그룹에는 인덱스가 2021-01-03인 행이 포함됩니다.
- ... (계속)

`apply` 함수와 `transform` 함수에서의 그룹

`apply` 함수는 각 그룹에 대해 함수를 적용하고 결과를 반환합니다. 예를 들어:

```
result_apply = grouped.apply(lambda x: (type(x), x.name))
```

이 코드는 각 그룹에 대해 하나의 튜플 `(type(x), x.name)`을 반환합니다. 여기서 `x`는 각 그룹의 `DataFrame`입니다. 결과는 각 그룹별로 하나의 튜플을 반환한 `Series`입니다.

`transform` 함수는 각 그룹의 각 행에 대해 변환된 값을 반환해야 합니다. 예를 들어:

```
result_transform = grouped.transform(lambda x: x.mean())
```

이 코드는 각 그룹의 각 행에 대해 그룹의 평균 값을 반환합니다.

결과는 원래 `DataFrame`과 동일한 길이와 인덱스를 가진 `Series`입니다.

문제의 원인

문제의 코드는 다음과 같습니다:

```
result_transform = grouped[['TotalHour', 'Avg']].transform(lambda x: (type(x), x.name))
```

이 코드는 각 그룹에 대해 하나의 튜플 `(type(x), x.name)`을 반환합니다.

이는 각 그룹의 모든 행에 대해 단 하나의 값을 반환하는 것입니다.

그러나 `transform` 함수는 각 그룹의 각 행에 대해 변환된 값을 반환해야 하므로, 반환된 결과의 길이가 원래 `DataFrame`과 일치하지 않게 됩니다. 이로 인해 크기 불일치 오류가 발생합니다.

따라서, `transform` 함수를 사용할 때는 각 그룹의 각 행에 대해 변환된 값을 반환하도록 해야 합니다.

예를 들어, 각 그룹의 평균 값을 반환하는 경우:

```
result_transform = grouped[['TotalHour', 'Avg']].transform(lambda x: x.mean())
```

이렇게 하면 각 그룹의 평균 값이 각 행에 매핑되어 반환됩니다.

```
...
```

```
print('')
```

In [63]: `import pandas as pd`

```
data = {
    'TotalHour': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Avg': [1.5, 2.5, 3.5, 4.5, 5.5, 6.5, 7.5, 8.5, 9.5, 10.5]
}
```

```
index = pd.date_range('2021-01-01', periods=10, freq='D')

df = pd.DataFrame(data, index=index)

df
```

Out[63]:

	TotalHour	Avg
2021-01-01	1	1.5
2021-01-02	2	2.5
2021-01-03	3	3.5
2021-01-04	4	4.5
2021-01-05	5	5.5
2021-01-06	6	6.5
2021-01-07	7	7.5
2021-01-08	8	8.5
2021-01-09	9	9.5
2021-01-10	10	10.5

In [64]:

```
result_apply = df.groupby(df.index.date).apply(lambda x: (type(x), x.name))
result_apply
```

Out[64]:

```
2021-01-01    (<class 'pandas.core.frame.DataFrame'>, 2021-0...
2021-01-02    (<class 'pandas.core.frame.DataFrame'>, 2021-0...
2021-01-03    (<class 'pandas.core.frame.DataFrame'>, 2021-0...
2021-01-04    (<class 'pandas.core.frame.DataFrame'>, 2021-0...
2021-01-05    (<class 'pandas.core.frame.DataFrame'>, 2021-0...
2021-01-06    (<class 'pandas.core.frame.DataFrame'>, 2021-0...
2021-01-07    (<class 'pandas.core.frame.DataFrame'>, 2021-0...
2021-01-08    (<class 'pandas.core.frame.DataFrame'>, 2021-0...
2021-01-09    (<class 'pandas.core.frame.DataFrame'>, 2021-0...
2021-01-10    (<class 'pandas.core.frame.DataFrame'>, 2021-0...
dtype: object
```

```
In [65]: result_transform = df.groupby(df.index.date)['TotalHour'].transform(lambda x: x.mean())
result_transform
```

```
Out[65]: 2021-01-01    1
2021-01-02    2
2021-01-03    3
2021-01-04    4
2021-01-05    5
2021-01-06    6
2021-01-07    7
2021-01-08    8
2021-01-09    9
2021-01-10   10
Freq: D, Name: TotalHour, dtype: int64
```

```
In [66]: result_transform = df.groupby(df.index.date)[['TotalHour', 'Avg']].transform('mean')
result_transform
```

```
Out[66]:
```

	TotalHour	Avg
2021-01-01	1	1.5
2021-01-02	2	2.5
2021-01-03	3	3.5
2021-01-04	4	4.5
2021-01-05	5	5.5
2021-01-06	6	6.5
2021-01-07	7	7.5
2021-01-08	8	8.5
2021-01-09	9	9.5
2021-01-10	10	10.5

```
In [67]: '''
# 해결 방법
`transform`은 각 그룹의 Series를 변환한 후, 원래 데이터프레임과 동일한 크기와 인덱스를
유지해야 하므로 반환 값이 동일한 길이의 Series여야 합니다.
```

따라서, 타입과 이름을 반환하는 대신 원하는 변환을 수행해야 합니다.
여기서는 각 그룹의 `TotalHour`와 `Avg`의 평균을 구하는 예제를 사용하겠습니다.

위 코드에서는 `transform`을 사용하여 각 그룹의 `TotalHour`와 `Avg` 열에 대해 평균값을 계산하고, 원래 데이터프레임과 동일한 크기와 인덱스를 유지하면서 결과를 반환합니다.

```
'''
# 각 그룹의 'TotalHour'와 'Avg'의 평균을 계산합니다.
df_basetable1.groupby(df_basetable1.index.date)[['TotalHour', 'Avg']].transform(lambda x: x.mean()).iloc[:5]
```

Out[67]:

	TotalHour	Avg
DateHour		
2021-01-01 00:00:00	184.708333	46.177083
2021-01-01 01:00:00	184.708333	46.177083
2021-01-01 02:00:00	184.708333	46.177083
2021-01-01 03:00:00	184.708333	46.177083
2021-01-01 04:00:00	184.708333	46.177083

In [68]:

```
'''
`apply`와 `transform` 함수는 `pandas`에서 그룹화된 객체에 대해 함수를 적용할 때 중요한 차이점을 가지고 있습니다.
이를 설명하기 위해 각 함수가 어떻게 동작하는지와 그 결과가 어떻게 다른지를 자세히 비교해 보겠습니다.

### `apply`와 `transform`의 차이
1. `apply`: 각 그룹에 대해 지정된 함수를 적용하고, 결과를 각 그룹에 대해 반환합니다.
   `apply`의 결과는 각 그룹에 대해 별도의 데이터프레임 또는 시리즈일 수 있으며, 그룹화된 인덱스를 유지합니다.
2. `transform`: 각 그룹에 대해 지정된 함수를 적용하고, 결과를 원래 데이터프레임의 인덱스에 맞춰서 반환합니다.
   즉, 각 그룹의 결과를 원래 인덱스와 동일한 길이의 시리즈로 반환합니다.

### 예제 데이터프레임
import pandas as pd

# 예시 데이터프레임 생성
dates = pd.date_range('2023-01-01', periods=10, freq='D')
data = {
    'TotalHour': [1, 2, 2, 3, 1, 4, 2, 3, 3, 2],
    'Avg': [0.5, 1.5, 1.7, 2.8, 0.9, 3.9, 1.8, 2.7, 2.9, 1.9]
}
```

```
df_basetable1 = pd.DataFrame(data, index=dates)
```

```
print("원본 데이터프레임:")
print(df_basetable1)
```

`apply`의 결과

```
result_apply = df_basetable1.groupby(df_basetable1.index.date)[['TotalHour', 'Avg']].apply(lambda x: (type(x), x.name))
print(result_apply.iloc[:5])
```

- 각 그룹(날짜별)에서 'TotalHour'와 'Avg' 열을 선택하여 `lambda x: (type(x), x.name)` 함수를 적용합니다.
- 함수는 각 그룹에 대해 데이터프레임 객체와 그룹의 이름(날짜)을 반환합니다.
- 결과는 그룹화된 데이터프레임으로, 각 그룹에 대해 `(type, name)` 튜플이 반환됩니다.

출력 예시:

```
2023-01-01    (<class 'pandas.core.frame.DataFrame'>, datetime.da...
2023-01-02    (<class 'pandas.core.frame.DataFrame'>, datetime.da...
2023-01-03    (<class 'pandas.core.frame.DataFrame'>, datetime.da...
2023-01-04    (<class 'pandas.core.frame.DataFrame'>, datetime.da...
2023-01-05    (<class 'pandas.core.frame.DataFrame'>, datetime.da...
dtype: object
```

`transform`의 결과

```
result_transform = df_basetable1.groupby(df_basetable1.index.date)[['TotalHour', 'Avg']].transform(lambda x: (type(x), x.name))
print(result_transform.iloc[:5])
```

- 각 그룹(날짜별)에서 'TotalHour'와 'Avg' 열을 선택하여 `lambda x: (type(x), x.name)` 함수를 적용합니다.
- 함수는 각 그룹에 대해 데이터프레임 객체와 그룹의 이름(날짜)을 반환하지만, `transform`은 반환된 결과를 원래 데이터프레임의 인덱스에 맞춰서 확장합니다.
- 결과는 원래 데이터프레임과 동일한 길이의 시리즈로, 각 원소는 `(type, name)` 튜플입니다.

출력 예시:

```
      TotalHour  Avg
2023-01-01    (<class 'pandas.core.series.Series'>, datetime.date(2023, 1, 1))
2023-01-02    (<class 'pandas.core.series.Series'>, datetime.date(2023, 1, 2))
2023-01-03    (<class 'pandas.core.series.Series'>, datetime.date(2023, 1, 3))
2023-01-04    (<class 'pandas.core.series.Series'>, datetime.date(2023, 1, 4))
2023-01-05    (<class 'pandas.core.series.Series'>, datetime.date(2023, 1, 5))
```

차이점 비교

1. 결과 형식:

- `apply`는 그룹별로 결과를 반환합니다. 각 그룹에 대해 하나의 결과가 있으며, 결과는 그룹화된 데이터프레임 또는 시리즈 형태입니다.
- `transform`은 원래 데이터프레임의 인덱스와 길이를 유지하며, 각 행에 대해 결과를 반환합니다.

결과는 원래 데이터프레임과 동일한 길이의 시리즈 또는 데이터프레임입니다.

2. 반환된 객체의 형식:

- ``apply``는 각 그룹에 대해 ``DataFrame`` 또는 ``Series`` 객체와 그룹의 이름(날짜)을 튜플로 반환합니다.
- ``transform``은 각 행에 대해 ``Series`` 객체와 그룹의 이름을 튜플로 반환하며, 원래 데이터프레임의 길이를 유지합니다.

3. 사용 예:

- ``apply``는 각 그룹에 대해 독립적인 연산을 수행하고 그 결과를 반환할 때 유용합니다.
 - ``transform``은 그룹별로 계산된 결과를 원래 데이터프레임의 인덱스에 맞춰 반환할 때 유용합니다.
- 예를 들어, 그룹별 평균, 합계, 순위 등을 원래 데이터프레임과 동일한 길이로 반환할 때 사용됩니다.

요약

```
- `apply`는 그룹화된 데이터프레임을 그룹별로 처리하여 독립적인 결과를 반환합니다.
- `transform`은 그룹화된 데이터프레임을 원래 인덱스와 동일한 길이로 확장하여 반환합니다.
...
print('')
```

In [69]: # 여기까지는 참고용 -> 단계 1-3 앞까지

단계 1-3

일별 기준(DateHour 변수 사용)으로 시간당 최대수요전력 합계(TotalHour)의

범위(최대값-최소값)가 가장 큰 값을 찾고

이 값을 D라고 한다.

```
In [70]: # 방법 1: groupby ~ agg
D = df_basetable1.groupby(df_basetable1.index.date)['TotalHour'].agg(lambda x: x.max() - x.min()).max()
D
```

Out[70]: 706

```
In [71]: # 방법 2: groupby ~ apply
# apply의 N:1 집계 연산을 통해서도 가능합니다.
D = df_basetable1.groupby(df_basetable1.index.date)['TotalHour'].apply(lambda x: x.max() - x.min()).max()
D
```

Out[71]: 706

단계 1-4

공휴일과 주말을 제외한 평일 데이터만 추출을 한 후,

일별 기준으로 시간당 최대수요전력 합계(TotalHour)의 범위(최대값-최소값)가

가장 큰 일자의 요일명(DayName)을 E라고 한다.

```
In [72]: # 예제 코드

data = {'Age' : [1, 2, 3, 4],
        'Score' : [1, 1, 6, 7]}
df = pd.DataFrame(data)
df
```

```
Out[72]:
```

	Age	Score
0	1	1
1	2	1
2	3	6
3	4	7

```
In [73]: df[['Age', 'Score']] == 1
```

```
Out[73]:
```

	Age	Score
0	True	True
1	False	True
2	False	False
3	False	False

```
In [74]: (df[['Age', 'Score']] == 1).any(axis = 1) #행 단위
```

```
Out[74]: 0    True
         1    True
         2   False
         3   False
         dtype: bool
```

```
In [75]: (df[['Age', 'Score']] == 1).all(axis = 1) #행 단위
```

```
Out[75]: 0    True
         1   False
         2   False
         3   False
         dtype: bool
```

```
In [76]: (df[['Age', 'Score']] == 1).any(axis = 0) #열 단위
```

```
Out[76]: Age      True
         Score    True
         dtype: bool
```

```
In [77]: (df[['Age', 'Score']] == 1).all(axis = 0) #열 단위
```

```
Out[77]: Age      False
         Score    False
         dtype: bool
```

```
In [78]: '''
```

1. 조건에 맞는 행 선택:

```
df_tmp = df_basetable1.loc[
    ~(df_basetable1[['Weekend_yn', 'Holiday_yn']] == 1).any(axis=1)
]
- `df_basetable1[['Weekend_yn', 'Holiday_yn']] == 1`는 'Weekend_yn' 또는 'Holiday_yn'이 1인 행들을 `True`로 표시하는 불리언 데이터프레임을 만듭니다.
- `.any(axis=1)`는 행 단위로 `True` 값이 하나라도 있으면 `True`를 반환하는 불리언 시리즈를 만듭니다.
- `~`는 이 불리언 시리즈의 값을 반전시켜 'Weekend_yn'과 'Holiday_yn'이 모두 1이 아닌 행들을 선택합니다.
- `df_basetable1.loc[...]`는 이 불리언 시리즈를 이용해 해당 행들을 선택합니다.
```

2. 그룹화 및 계산:

```
E = df_tmp.groupby(df_tmp.index.date)['TotalHour'].apply(lambda x: x.max() - x.min()).idxmax().weekday()
- `df_tmp.groupby(df_tmp.index.date)`는 날짜별로 그룹화합니다.
- `[ 'TotalHour' ]`는 'TotalHour' 열을 선택합니다.
- `.apply(lambda x: x.max() - x.min())`는 각 그룹에서 'TotalHour'의 최댓값과 최솟값의 차이를 계산합니다.
```

```
- `.idxmax()`는 차이가 가장 큰 그룹의 인덱스를 반환합니다.  
- `.weekday()`는 그 인덱스의 요일을 반환합니다.  
...  
  
# 방법 1: 중간 결과를 임시테이블에 저장합니다.  
df_tmp = df_basetable1.loc[  
    ~(df_basetable1[['Weekend_yn', 'Holiday_yn']] == 1).any(axis=1)  
]  
  
E = df_tmp.groupby(df_tmp.index.date)['TotalHour'].apply(lambda x: x.max() - x.min()).idxmax().weekday()  
  
E
```

Out[78]: 0

```
In [79]: df_basetable1.loc[  
    ~(df_basetable1[['Weekend_yn', 'Holiday_yn']] == 1).any(axis=1)  
]
```

Out[79]:

	15 분	30 분	45 분	60 분	DayName	Hour	AM	Weekend_yn	Holiday_yn	Avg	TotalHour	생산 량	기 온	풍 속	습 도	강 수 량	전기요 금(계 절)	공장인원	인 건 비
DateHour																			
2021-01-04 00:00:00	72	67	66	66	0	0	0	0	0	67.75	271	0	-0.1	1.7	39	0.0	109.8	0.000000	1.5
2021-01-04 01:00:00	96	108	119	109	0	1	0	0	0	108.00	432	0	-0.9	1.3	44	0.0	109.8	0.000000	1.5
2021-01-04 02:00:00	106	106	111	116	0	2	0	0	0	109.75	439	0	-0.4	1.4	42	0.0	109.8	0.000000	1.5
2021-01-04 03:00:00	104	105	104	113	0	3	0	0	0	106.50	426	0	-2.1	1.5	50	0.0	109.8	0.000000	1.5
2021-01-04 04:00:00	115	109	108	103	0	4	0	0	0	108.75	435	0	-1.9	1.8	50	0.0	109.8	0.000000	1.5
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2021-09-14 19:00:00	152	151	171	139	1	19	1	0	0	153.25	613	1497	21.7	3.6	85	9.4	167.2	2.442088	1.5
2021-09-14 20:00:00	124	130	128	130	1	20	1	0	0	128.00	512	45	22.2	4.2	78	9.4	167.2	0.087891	1.5
2021-09-14 21:00:00	134	130	125	124	1	21	1	0	0	128.25	513	149	22.2	4.3	76	9.4	167.2	0.290448	1.5
2021-09-14 22:00:00	100	109	120	114	1	22	1	0	0	110.75	443	66	22.0	2.5	79	9.4	167.2	0.148984	1.5
2021-09-14 23:00:00	117	119	112	91	1	23	1	0	0	109.75	439	405	22.0	2.5	79	9.4	167.2	0.922551	1.5

4224 rows × 19 columns

In [80]:

```
'''
1. 조건에 맞는 행 선택:
df_basetable1.loc[
    ~(df_basetable1[['Weekend_yn', 'Holiday_yn']] == 1).any(axis=1)
]
```

```

- 이 부분은 첫 번째 코드와 동일합니다.

2. `pipe`를 이용한 연산:
    .pipe(
        lambda x: x.groupby(x.index.date)['TotalHour']
            .apply(lambda x: x.max() - x.min()).idxmax().weekday()
    )
- `.pipe()`는 데이터프레임을 인자로 받아 다른 함수에 전달합니다. 이를 통해 메소드 체이닝을 깔끔하게 유지할 수 있습니다.
- `lambda x: x.groupby(x.index.date)['TotalHour']
    .apply(lambda x: x.max() - x.min()).idxmax().weekday()`는 첫 번째 코드의 두 번째 부분과 동일한 연산을 수행합니다.
...

# 방법 2: pipe를 통해 연산을 이어갑니다.
E = df_basetable1.loc[
    ~(df_basetable1[['Weekend_yn', 'Holiday_yn']] == 1).any(axis=1)
].pipe(
    lambda x: x.groupby(x.index.date)['TotalHour'].apply(lambda x: x.max() - x.min()).idxmax().weekday()
)

E

```

Out[80]: 0

In [81]: ...

위의 두 코드 조각은 최종적으로 동일한 결과를 반환하지만, 그 접근 방식이 다릅니다.
차이점은 코드의 가독성과 연쇄 메소드 호출 방식에 있습니다.

차이점 비교

1. 코드 구조와 가독성:

- 첫 번째 코드 조각은 중간 변수 `df\_tmp`를 사용하여 조건에 맞는 행을 선택하고, 그 다음 단계에서 그룹화와 계산을 수행합니다. 중간 변수를 사용하여 코드의 각 단계를 명확하게 분리합니다.
- 두 번째 코드 조각은 `pipe` 메소드를 사용하여 메소드 체이닝을 통해 한 줄로 연결된 형태로 작성되어 있습니다. `pipe`를 사용하면 중간 변수를 생략하고, 코드가 더 간결해집니다. 그러나 복잡한 로직에서는 가독성이 떨어질 수 있습니다.

2. 연산 과정:

- 두 코드 조각은 기능적으로 동일한 연산을 수행합니다. 조건에 맞는 행을 선택하고, 날짜별로 'TotalHour'의 최댓값과 최솟값의 차이를 계산하여, 그 차이가 가장 큰 날짜의 요일을 반환합니다.

3. 유연성:

- 첫 번째 방법은 각 단계를 명확하게 나누기 때문에 디버깅이나 개별 단계의 수정이 용이합니다.
- 두 번째 방법은 `pipe`를 통해 연속적인 메소드 호출을 가능하게 하여 코드가 더 간결해지고, 함수형 프로그래밍 스타일을 따릅니다.

요약

- 첫 번째 코드 조각: 중간 변수를 사용하여 각 단계를 명확하게 분리하고, 가독성을 높입니다.

```
- 두 번째 코드 조각: `pipe`를 사용하여 메소드 체이닝을 통해 코드를 간결하게 유지하며, 함수형 프로그래밍 스타일을 구현합니다.
...
print('')
```

```
In [82]: # 방법 2-2: idxmax를 모르고 있다면 생각해볼수 있는 방법
E = df_basetable1.loc[
    ~(df_basetable1[['Weekend_yn', 'Holiday_yn']] == 1).any(axis=1)
].pipe(
    lambda x: x.groupby(x.index.date)['TotalHour'].apply(lambda x: x.max() - x.min())
).pipe(
    lambda x: x.loc[x == x.max()].index.values[0].weekday()
)

E
```

Out[82]: 0

```
In [83]: # 방법 2와 방법 2-2의 비교
...
1. 첫 번째 `pipe`:
    .pipe(
        lambda x: x.groupby(x.index.date)['TotalHour']
            .apply(lambda x: x.max() - x.min())
    )
    - 날짜별로 그룹화하여 각 날짜의 'TotalHour'의 최댓값과 최솟값의 차이를 계산합니다.
    - 이 결과는 각 날짜에 대해 최댓값과 최솟값의 차이를 담은 시리즈입니다.

2. 두 번째 `pipe`:
    .pipe(
        lambda x: x.loc[x == x.max()].index.values[0].weekday()
    )
    - `x == x.max()`는 최댓값을 가진 날짜를 찾습니다.
    - `loc[x == x.max()]`는 최댓값을 가진 날짜의 인덱스를 선택합니다.
    - `index.values[0].weekday()`는 그 인덱스의 첫 번째 값의 요일을 반환합니다.

### 차이점 비교
1. 중간 결과 처리:
    - 첫 번째 코드 조각에서는 그룹화된 결과에서 바로 `idxmax()`를 사용하여 최댓값의 인덱스를 찾고, 그 인덱스의 요일을 반환합니다.
    - 두 번째 코드 조각에서는 두 개의 `pipe`를 사용하여 중간 결과를 먼저 계산하고, 그 중 최댓값을 가진 인덱스를 찾아서 요일을 반환함
2. 가독성:
```

```

- 첫 번째 코드 조각은 더 간결하며, 연산을 한 줄로 연결하여 읽기 쉽습니다.
- 두 번째 코드 조각은 중간 결과를 명확히 분리하여 각 단계를 구체적으로 나타내지만, 다소 복잡해 보일 수 있습니다.
3. 유연성:
- 두 번째 코드 조각은 중간 결과를 명시적으로 처리하기 때문에, 중간 결과를 확인하거나 디버깅할 때 더 유리합니다.
- 첫 번째 코드 조각은 중간 변수를 사용하지 않기 때문에 코드가 더 간결하지만, 디버깅 시 중간 결과를 확인하기 어렵습니다.
...
print('')

```

In [84]: A, B, C, D, E, A + B + C + D + E

Out[84]: (830, 8, 11, 706, 0, 1555)

A + B + C + D + E를 적으시오.

문제 2

최대수요전력 평균(Avg) 값이 오전/오후에 따라 차이가 있는지 검토하십시오.

단계 2-1

basetable1 에서 공휴일과 주말을 제외하고 prob2 데이터프레임을 만든다.

- prob2에서 시간(Hour)이 8과 11사이 (8, 11 포함) 또는 13과 16 (13, 16 포함) 사이의 행들만 남긴다.
- prob2에서 Avg가 130이상인 행들만을 남긴다.

Hint] 최종 1233개 행만 남는다.

```

In [85]: '''
`pandas`의 `query()` 함수는 DataFrame에 대해 문자열 표현식을 사용하여 행을 필터링할 수 있는 편리한 방법을 제공합니다.
이 함수는 특히 복잡한 조건을 사용할 때 코드 가독성을 높이고 작성하기 쉽게 해줍니다.

### `query()` 함수의 구문
DataFrame.query(expr, inplace=False, kwargs)

```



```

### 매개 변수
1. `expr` (str):
    - 필터링 조건을 나타내는 문자열 표현식입니다. 표현식 내에서 DataFrame의 열 이름을 직접 사용하여 조건을 지정할 수 있습니다.

2. `inplace` (bool, default False):
    - `True`로 설정하면 원본 DataFrame을 변경합니다. `False`인 경우 필터링된 새로운 DataFrame을 반환합니다.

3. `kwargs`:
    - 다른 매개변수들을 포함할 수 있으며, 대부분의 경우에는 `query()`에서 사용할 수 있는 로컬 변수나 글로벌 변수를 전달하는 데 사용

### 리턴값
- 기본적으로 `query()`는 필터링된 새로운 DataFrame을 반환합니다.
  `inplace=True`로 설정된 경우에는 `None`을 반환하고 원본 DataFrame이 수정됩니다.

### 사용 예시

1. 기본 사용법
import pandas as pd

# 예제 DataFrame 생성
data = {
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'age': [24, 27, 22, 32],
    'score': [85, 62, 74, 91]
}
df = pd.DataFrame(data)

# query() 함수 사용
filtered_df = df.query('age > 25')
print(filtered_df)

결과:
   name  age  score
1  Bob   27     62
3 David  32     91

2. 복잡한 조건 사용
# 두 개의 조건을 사용하는 예제
filtered_df = df.query('age > 25 and score > 70')
print(filtered_df)

결과:

```

```

    name  age  score
3 David   32     91

```

3. 변수 사용

외부 변수를 사용하는 예제

```
age_threshold = 25
```

```
score_threshold = 70
```

```

filtered_df = df.query('age > @age_threshold and score > @score_threshold')
print(filtered_df)

```

결과:

```

    name  age  score
3 David   32     91

```

4. inplace 사용

inplace=True를 사용하는 예제

```
df.query('age > 25', inplace=True)
```

```
print(df)
```

결과:

```

    name  age  score
1  Bob   27     62
3 David   32     91

```

5. 열 이름에 공백이나 특수문자가 있는 경우

열 이름에 공백이나 특수문자가 있는 경우 backticks(`)을 사용

```

df = pd.DataFrame({
    'user name': ['Alice', 'Bob', 'Charlie', 'David'],
    'user age': [24, 27, 22, 32],
    'user score': [85, 62, 74, 91]
})

```

```

filtered_df = df.query("`user age` > 25 and `user score` > 70')
print(filtered_df)

```

결과:

```

   user name  user age  user score
3      David         32         91

```

요약

```
- `query()` 함수는 DataFrame을 필터링할 때 문자열 표현식을 사용하여 조건을 지정할 수 있습니다.
- 기본 매개변수는 `expr` (필터링 조건 표현식), `inplace` (기존 DataFrame을 수정할지 여부)이며,
  `kwargs`로 다른 변수들을 전달할 수 있습니다.
- 반환값은 기본적으로 필터링된 새로운 DataFrame이며, `inplace=True`로 설정된 경우에는 `None`을 반환합니다.
- 외부 변수나 공백이 포함된 열 이름도 사용할 수 있습니다.
  외부 변수는 `@` 기호를 사용하여 참조하고, 공백이 포함된 열 이름은 backticks(``)을 사용하여 참조합니다.
...
```

```
...
```

이 코드는 두 부분으로 나누어 설명할 수 있습니다.

첫 번째 부분은 특정 조건을 만족하는 행을 선택하고, 두 번째 부분은 추가적인 조건을 만족하는 행을 선택합니다.

첫 번째 부분: 특정 조건을 만족하는 행 선택

```
df_prob2 = df_basetable1.loc[
    (df_basetable1[['Weekend_yn', 'Holiday_yn']] == 0).all(axis=1)
]
```

1. 조건 설정:

```
(df_basetable1[['Weekend_yn', 'Holiday_yn']] == 0)
- `df_basetable1[['Weekend_yn', 'Holiday_yn']]`는 데이터프레임 `df_basetable1`에서
  `Weekend_yn`과 `Holiday_yn` 열을 선택합니다.
- `== 0`는 각 행의 `Weekend_yn`과 `Holiday_yn` 값이 0인지 확인합니다.
  결과는 `True` 또는 `False` 값을 가지는 데이터프레임이 됩니다.
```

2. 모든 조건 만족 여부 확인:

```
.all(axis=1)
- `all(axis=1)`는 각 행(row)에 대해 모든 조건이 `True`인지를 확인합니다.
  모든 조건이 `True`이면 `True`를 반환하고, 하나라도 `False`이면 `False`를 반환합니다.
```

3. 조건을 만족하는 행 선택:

```
df_basetable1.loc[...]
- `loc`는 조건이 `True`인 행을 선택합니다.
- 결과적으로, `Weekend_yn`과 `Holiday_yn` 열의 값이 모두 0인 행들만 선택됩니다.
```

두 번째 부분: 추가 조건을 만족하는 행 선택

```
df_prob2 = df_prob2.query('((Hour >= 8 and Hour <= 11) or (Hour >= 13 and Hour <= 16)) and Avg >= 130')
```

1. 조건 설정:

```
'((Hour >= 8 and Hour <= 11) or (Hour >= 13 and Hour <= 16)) and Avg >= 130'
- 이 문자열은 쿼리 조건을 나타냅니다.
- `(Hour >= 8 and Hour <= 11)`는 `Hour` 값이 8 이상 11 이하인 행을 선택합니다.
- `(Hour >= 13 and Hour <= 16)`는 `Hour` 값이 13 이상 16 이하인 행을 선택합니다.
- 두 조건은 `or` 연산자로 결합되어 있어, `Hour` 값이 8~11 또는 13~16 사이에 있는 행을 선택합니다.
- `and Avg >= 130`는 `Avg` 값이 130 이상인 행을 선택합니다.
- 전체 조건은 `Hour` 값이 8~11 또는 13~16 사이이고, 동시에 `Avg` 값이 130 이상인 행을 선택합니다.
```

2. 조건을 만족하는 행 선택:

```
df_prob2.query(...)
```

- ``query`` 메소드는 주어진 조건을 만족하는 행을 선택합니다.
- 결과적으로, ``df_prob2``는 ``Hour`` 값이 8~11 또는 13~16 사이이고, ``Avg`` 값이 130 이상인 행들만 포함하게 됩니다.

전체 코드 동작 요약

1. 첫 번째 부분:
 - ``df_basetable1``에서 ``Weekend_yn``과 ``Holiday_yn`` 값이 모두 0인 행을 선택합니다.
 - 즉, 주말이 아니고 공휴일이 아닌 날의 데이터를 선택합니다.
2. 두 번째 부분:
 - 선택된 데이터 중 ``Hour`` 값이 8~11 또는 13~16 사이이고, ``Avg`` 값이 130 이상인 행을 추가로 선택합니다.

최종적으로, ``df_prob2``는 주말이 아니고 공휴일이 아닌 날 중에서 특정 시간대(8~11 또는 13~16)와 ``Avg`` 값이 130 이상인 데이터를 포함하는 데이터프레임이 됩니다.

```
...
```

방법 1: *query*: 조건을 컬럼명으로만 표현할 수 있다면 유용한 방법입니다.

```
df_prob2 = df_basetable1.loc[
    (df_basetable1[['Weekend_yn', 'Holiday_yn']] == 0).all(axis=1)
]

df_prob2 = df_prob2.query('((Hour >= 8 and Hour <= 11) or \
    (Hour >= 13 and Hour <= 16)) and Avg >= 130')

len(df_prob2)
```

Out[85]: 1233

```
In [86]: # 방법 2: Boolean Indexing - 조건을 컬럼명으로 표현할 수 없을 경우에
# 논리 연산을 통해 구현합니다.
df_prob2 = df_basetable1.loc[
    (df_basetable1[['Weekend_yn', 'Holiday_yn']] == 0).all(axis=1)
]
df_prob2 = df_prob2.loc[
    (
        ((df_prob2['Hour'] >= 8) & (df_prob2['Hour'] <= 11)) |
        ((df_prob2['Hour'] >= 13) & (df_prob2['Hour'] <= 16))
    ) & (df_prob2['Avg'] >= 130)
]
len(df_prob2)
```

Out[86]: 1233

```
In [87]: # 방법 2-2: Boolean Indexing + between - between 메소드를 통해
# 사이에 해당하는 조건을 간소화합니다.
df_prob2 = df_prob2.loc[
    (
        df_prob2['Hour'].between(8, 11, inclusive=True) |
        df_prob2['Hour'].between(13, 16, inclusive=True)
    ) & (df_prob2['Avg'] >= 130)
]
len(df_prob2)
```

Out[87]: 1233

단계 2-2

오전(AM = 0) 인 행들을 대상으로 Avg에 대하여 Shapiro-Wilk 검정을 사용하여 정규성 검정한다.

검정 결과에서 p-value를 A라고 한다.

함수가이드

scipy.stats 제공 기능 활용

문제 지시사항 외 Default 값 사용

```
In [88]: '''
- `df_prob2`는 이미 이전 조건을 만족하는 데이터를 포함하고 있는 데이터프레임입니다.
- `df_prob2['AM'] == 0`는 `AM` 열이 0인 행을 선택하는 조건입니다.
- `df_prob2.loc[...]`는 이 조건을 만족하는 행을 선택합니다.
- 두 번째 인자인 ``Avg``는 선택된 행들에서 `Avg` 열만을 선택하도록 지정합니다.
- 결과적으로, `df_prob2.loc[df_prob2['AM'] == 0, 'Avg']`는 `AM` 열이 0인 행들의 `Avg` 열 값을 포함하는 시리즈(Series)가 됩니다.
'''

from scipy.stats import shapiro

result_2_2 = shapiro(df_prob2.loc[df_prob2['AM'] == 0, 'Avg'])
```

```
A = result_2_2.pvalue
result_2_2, A
```

```
Out[88]: (ShapiroResult(statistic=0.9945228695869446, pvalue=0.026428762823343277),
         0.026428762823343277)
```

단계 2-3

오후(AM = 1) 인 행들을 대상으로 Avg에 대하여 Shapiro-Wilk 검정을 사용하여 정규성 검정한다.

검정 결과에서 p-value를 B라고 한다.

```
In [89]: from scipy.stats import shapiro

result_2_3 = shapiro(df_prob2.loc[df_prob2['AM'] == 1, 'Avg'])
B = result_2_3.pvalue
result_2_3, B
```

```
Out[89]: (ShapiroResult(statistic=0.9950335025787354, pvalue=0.043661389499902725),
         0.043661389499902725)
```

단계 2-4

오전/오후에 따라 Avg의 모평균에 차이가 있는지 검정을 한다.

검정 방법은 T-검정을 사용하며, Bartlett 검정을 하고 결과를 T-검정 시 반영한다(유의 수준 5%).

검정 결과에서 검정통계량의 절대값을 C라고 한다.

함수가이드

scipy.stats 제공 기능 활용

문제 지시사항 외 Default 값 사용

In [90]:

```
'''
주어진 코드는 두 그룹의 평균을 비교하기 위해 Bartlett's test와 독립 표본 t-test를 수행합니다.
아래에 각 라인별로 동작하는 내용을 자세히 설명합니다:

result_bart = bartlett(
    df_prob2.loc[df_prob2['AM'] == 0, 'Avg'],
    df_prob2.loc[df_prob2['AM'] == 1, 'Avg']
)
1. `bartlett()` 함수는 Bartlett's test를 수행하여 두 그룹의 분산이 동일한지 확인합니다.
2. `df_prob2.loc[df_prob2['AM'] == 0, 'Avg']`는 `df_prob2` DataFrame에서 'AM' 열의 값이 0인 행의 'Avg' 열 값을 선택합니다.
3. `df_prob2.loc[df_prob2['AM'] == 1, 'Avg']`는 `df_prob2` DataFrame에서 'AM' 열의 값이 1인 행의 'Avg' 열 값을 선택합니다.
4. 선택된 두 그룹의 'Avg' 값을 `bartlett()` 함수에 전달하여 Bartlett's test를 수행하고 결과를 `result_bart` 변수에 저장합니다.

equal_var = result_bart.pvalue >= 0.05
5. `result_bart.pvalue`는 Bartlett's test의 p-value를 의미합니다.
6. Bartlett's test의 p-value가 0.05 이상인 경우, 두 그룹의 분산이 통계적으로 유의미하게 다르지 않다고 판단하여
   `equal_var` 변수를 `True`로 설정합니다. 그렇지 않은 경우 `False`로 설정합니다.

result_2_4 = ttest_ind(
    df_prob2.loc[df_prob2['AM'] == 0, 'Avg'],
    df_prob2.loc[df_prob2['AM'] == 1, 'Avg'],
    equal_var = equal_var
)
7. `ttest_ind()` 함수는 독립 표본 t-test를 수행하여 두 그룹의 평균이 동일한지 여부를 확인합니다.
8. `df_prob2.loc[df_prob2['AM'] == 0, 'Avg']`와 `df_prob2.loc[df_prob2['AM'] == 1, 'Avg']`는 앞서 설명한 것처럼
   각 그룹의 'Avg' 값을 선택합니다.
9. `equal_var = equal_var`는 Bartlett's test 결과에 따라 두 그룹의 분산이 동일한지 여부를 `ttest_ind()` 함수에 전달합니다.
10. `ttest_ind()` 함수는 두 그룹의 평균을 비교하고 그 결과를 `result_2_4` 변수에 저장합니다.

C = abs(result_2_4.statistic)
11. `result_2_4.statistic`는 t-test의 t-통계량 값을 의미합니다.

### 전체 코드 설명
주어진 코드는 다음과 같은 과정으로 두 그룹의 평균을 비교합니다:

1. Bartlett's test를 사용하여 두 그룹의 분산이 동일한지 확인합니다.
2. Bartlett's test의 결과에 따라 독립 표본 t-test에서 분산이 동일한지 여부를 설정합니다.
3. 독립 표본 t-test를 수행하여 두 그룹의 평균 차이가 통계적으로 유의미한지 확인합니다.
4. t-test의 t-통계량 절대값을 `C` 변수에 저장합니다.
'''
```

```

# ttest_1samp: H0: mu = a, H1 mu ≠ a
#           ttest_1samp 에서는 mu < a, mu > a 경우는 지원하지 않습니다.
# ttest_ind: H0: mu_1 = mu_2, H1 mu_1 ≠ mu_2
#           ttest_ind 에서는 mu_1 < mu_2, mu_1 > mu_2 경우는 지원하지 않습니다.
# ttest_rel: H0: mu = mu', H1 mu ≠ mu'
#           ttest_rel 에서는 mu < mu', mu > mu' 경우는 지원하지 않습니다.

from scipy.stats import ttest_ind, bartlett

result_bart = bartlett(
    df_prob2.loc[df_prob2['AM'] == 0, 'Avg'],
    df_prob2.loc[df_prob2['AM'] == 1, 'Avg']
)

equal_var = result_bart.pvalue >= 0.05

result_2_4 = ttest_ind(
    df_prob2.loc[df_prob2['AM'] == 0, 'Avg'],
    df_prob2.loc[df_prob2['AM'] == 1, 'Avg'],
    equal_var = equal_var
)

C = abs(result_2_4.statistic)

result_bart, equal_var, result_2_4, C

```

```

Out[90]: (BartlettResult(statistic=0.1500142357781029, pvalue=0.6985217544572233),
True,
Ttest_indResult(statistic=8.27772409322812, pvalue=3.2348808563995005e-16),
8.27772409322812)

```

점곱(내적)에 대한 설명

점곱(또는 내적)은 두 벡터의 곱셈 연산 중 하나로, 결과가 스칼라 값(단일 숫자)인 연산입니다.

점곱은 기하학적으로 두 벡터의 방향이 얼마나 유사한지를 나타내며, 선형 대수학에서 매우 중요한 개념입니다.

점곱의 정의

두 벡터 ($\mathbf{a} = [a_1, a_2, \dots, a_n]$)와($\mathbf{b} = [b_1, b_2, \dots, b_n]$)의 점곱은 다음과 같이 정의됩니다:

$$[\mathbf{a} \cdot \mathbf{b} = a_1b_1 + a_2b_2 + \dots + a_nb_n]$$

즉, 두 벡터의 각 성분을 곱한 후 그 결과를 모두 더한 값입니다.

예제

예를 들어, 두 벡터 ($\mathbf{a} = [1, 2, 3]$)와($\mathbf{b} = [4, 5, 6]$)의 점곱은 다음과 같이 계산됩니다:

$$[\mathbf{a} \cdot \mathbf{b} = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 4 + 10 + 18 = 32]$$

점곱의 의미

- **기하학적 의미:** 점곱은 두 벡터 사이의 각도와 관련이 있습니다.
- 벡터 ($\mathbf{a}$)와($\mathbf{b}$)가 이루는 각도를 (θ)라고 할 때, 점곱은 다음과 같은 관계를 가집니다:

$$[\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}|\cos(\theta)]$$

- 여기서 ($|\mathbf{a}|$)와($|\mathbf{b}|$)는 각각 벡터 ($\mathbf{a}$)와($\mathbf{b}$)의 크기(길이)입니다.
- **코사인 값:** 점곱이 0이면 두 벡터는 서로 수직(직교)입니다. 점곱이 양수면 두 벡터는 예각을 이루고, 음수면 두 벡터는 둔각을 이룹니다.

NumPy를 사용한 점곱 예제

```
import numpy as np
```

```
# 두 벡터 정의
```

```
a = np.array([1, 2, 3])
```

```
b = np.array([4, 5, 6])
```

```
# 점곱 계산
```

```
dot_product = np.dot(a, b)
```

```
print(dot_product) # 출력: 32
```

Pandas를 사용한 점곱 예제

```
import pandas as pd

# 두 시리즈 정의
s1 = pd.Series([1, 2, 3])
s2 = pd.Series([4, 5, 6])

# 점곱 계산
dot_product = s1.dot(s2)
print(dot_product) # 출력: 32
```

요약

점곱은 두 벡터의 곱셈 연산으로 결과가 스칼라 값이 됩니다.

각 성분을 곱한 후 더하는 방식으로 계산하며, 기하학적으로 두 벡터 사이의 각도와 밀접한 관련이 있습니다.

점곱을 사용하면 두 벡터의 방향이 얼마나 유사한지를 알 수 있습니다.

In [163...

```
'''
`dot()` 함수는 파이썬의 `numpy` 와 `pandas` 라이브러리에서 제공되는 함수로, 두 배열 간의 점곱(내적)을 계산하는 데 사용됩니다.

# NumPy에서 `dot()` 함수
`numpy.dot(a, b, out=None)`

## 인자
- `a`: 첫 번째 입력 배열. 스칼라, 1차원 배열, 2차원 배열일 수 있습니다.
- `b`: 두 번째 입력 배열. 스칼라, 1차원 배열, 2차원 배열일 수 있습니다.
    `a`와 `b`의 차원에 따라 다르게 처리됩니다.
- `out`: 선택적 인자. 출력 결과를 저장할 배열입니다. 배열의 형태는 결과와 같아야 합니다.

## 리턴값
- `a`와 `b`의 점곱 결과를 반환합니다. 결과는 스칼라, 벡터 또는 행렬이 될 수 있습니다.

## 예제
import numpy as np

# 벡터 간의 점곱
a = np.array([1, 2])
b = np.array([3, 4])
```

```
result = np.dot(a, b)
print(result) # 출력: 11

# 행렬 곱
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])
result = np.dot(a, b)
print(result)
# 출력:
# [[19 22]
#  [43 50]]

# Pandas에서 `dot()` 함수
`Series`와 `DataFrame` 클래스 모두에서 사용 가능합니다.

## Pandas Series에서 `dot()` 함수
`Series.dot(other)`

### 인자
- `other`: 점곱을 수행할 다른 `Series` 또는 배열입니다. 두 `Series`의 길이는 같아야 합니다.

## 리턴값
- 점곱 결과를 스칼라 값으로 반환합니다.

## 예제
import pandas as pd

s1 = pd.Series([1, 2, 3])
s2 = pd.Series([4, 5, 6])
result = s1.dot(s2)
print(result) # 출력: 32

# Pandas DataFrame에서 `dot()` 함수
`DataFrame.dot(other)`

## 인자
- `other`: `DataFrame`이나 배열입니다. `other`의 열 수는 호출하는 `DataFrame`의 행 수와 같아야 합니다.

## 리턴값
- 행렬 곱의 결과를 `DataFrame`으로 반환합니다.

## 예제
```

```
import pandas as pd

df1 = pd.DataFrame([[1, 2], [3, 4]])
df2 = pd.DataFrame([[5, 6], [7, 8]])
result = df1.dot(df2)
print(result)
# 출력:
#      0    1
# 0  19   22
# 1  43   50

## 요약
- NumPy에서 `dot()` 함수는 스칼라, 벡터, 행렬 간의 점곱을 수행하며, 입력 배열의 차원에 따라 결과가 달라집니다.
- Pandas에서 `dot()` 함수는 `Series`나 `DataFrame` 간의 점곱을 수행하며,
  `Series`의 경우 스칼라 값을, `DataFrame`의 경우 새로운 `DataFrame`을 반환합니다.
- 인자: 두 라이브러리 모두 첫 번째와 두 번째 인자로 점곱할 배열을 받습니다.
- 리턴값: NumPy에서는 점곱 결과가 스칼라, 벡터 또는 행렬이 될 수 있고,
  Pandas에서는 `Series`의 경우 스칼라 값을, `DataFrame`의 경우 `DataFrame`을 반환합니다.
...
print('')
```

In []:

Tip. Bartlett 검정(동질성 검정)을 직접해봅니다.

$$B = \frac{(N-k) \ln(s_p^2) - \sum_{i=1}^k (n_i - 1) \ln(s_i^2)}{1 + \frac{1}{3(k-1)} \left(\sum_{i=1}^k \left(\frac{1}{n_i - 1} \right) - \frac{1}{N-k} \right)} \sim \chi^2(k-1), \text{ k는 모집단의 수}$$

검정 종류: Right side

$$s_p^2 = \frac{\sum_i^k (n_i - 1) s_i^2}{\sum_i^k (n_i - 1)}$$

In [92]:

```
from scipy.stats import chi2
```

```
# 계산에 필요한 항목을 구합니다.
N = len(df_prob2)
k = df_prob2['AM'].nunique()
```

```

nm1 = df_prob2['AM'].value_counts() - 1
sp2_i = df_prob2.groupby('AM')['Avg'].var()

print('N={}, k={}'.format(N, k))
print(nm1)
print('-----')
print(sp2_i)
print('-----')

# 통합 분산을 구합니다.
sp2 = (nm1 * sp2_i).sum() / nm1.sum()
print('sp2= ', sp2)
print('-----')

# B의 분자항을 구합니다.
# 두 벡터(수열)의 곱의 합은 dot(내적)으로 나타낼 수 있습니다.
num = (N - k) * np.log(sp2) - nm1.dot(np.log(sp2_i))
print('num = ', num)
print('-----')

# B의 분모항을 구합니다.
den = 1 + 1 / (3 * (k - 1)) * ((1 / nm1).sum() - 1 / (N - k))
print('den = ', den)
print('-----')

B_bar = num / den
print('B_bar = ', B_bar)
print('-----')

B_bar, 1 - chi2.cdf(B_bar, k - 1), chi2.sf(B_bar, k - 1)

```

```

N=1233, k=2
1      617
0      614
Name: AM, dtype: int64
-----
AM
0      157.907328
1      153.051088
Name: Avg, dtype: float64
-----
sp2= 155.47329084778724
-----
num = 0.1501361004602586
-----
den = 1.0008123541177516
-----
B_bar = 0.1500142357781029
-----

```

Out[92]: (0.1500142357781029, 0.6985217544572233, 0.6985217544572233)

In []:

Tip. F 검정을 직접해봅니다.

$$F = \frac{s_1^2}{s_2^2} \sim F(n_1 - 1, n_2 - 1)$$

$$H_0 : \sigma_1^2 = \sigma_2^2$$

$$H_1 : \sigma_1^2 \neq \sigma_2^2$$

양측 검정

```

In [93]: from scipy.stats import f

s2_1 = df_prob2.loc[df_prob2['AM'] == 0, 'Avg'].var()
s2_2 = df_prob2.loc[df_prob2['AM'] == 1, 'Avg'].var()

dof1 = (df_prob2['AM'] == 0).sum() - 1

```

```
dof2 = (df_prob2['AM'] == 1).sum() - 1

F = s2_1 / s2_2

# 양측 검정은 정확한 pvalue를 구할 수 없습니다.
# 이에 대한 어림치는 양쪽 꼬리 면적 중에서 작은 쪽 × 2를 합니다.
F, min(f.cdf(F, dof1, dof2), 1 - f.cdf(F, dof1, dof2)) * 2
```

Out[93]: (1.031729536526785, 0.6984733124132894)

```
In [94]: # 분모항과 분자항을 바꾸어서도 검정을 수행할 수 있습니다.
s2_1 = df_prob2.loc[df_prob2['AM'] == 1, 'Avg'].var()
s2_2 = df_prob2.loc[df_prob2['AM'] == 0, 'Avg'].var()

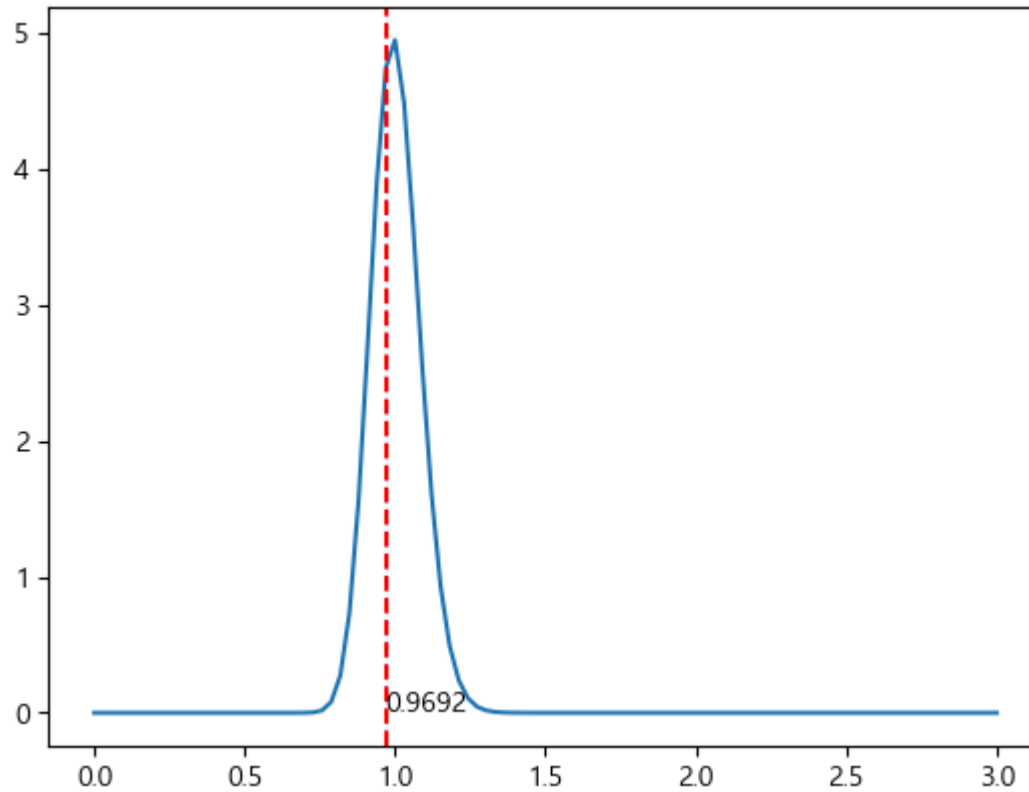
# 이에 따라 분자의 자유도 분모의 자유도 또한 바뀌어줍니다.
dof1 = (df_prob2['AM'] == 1).sum() - 1
dof2 = (df_prob2['AM'] == 0).sum() - 1

F = s2_1 / s2_2

# 검정통계량은 다르지만 pvalue는 같습니다.
F, min(f.cdf(F, dof1, dof2), 1 - f.cdf(F, dof1, dof2)) * 2
```

Out[94]: (0.9692462652241213, 0.6984733124132873)

```
In [95]: plt.plot(np.linspace(0, 3, 100), f.pdf(np.linspace(0, 3, 100), dof1, dof2))
plt.axvline(F, color='red', linestyle='--')
plt.text(F, 0.01, str(F)[:6])
plt.show()
```



In []:

Tip. 등분산일 경우 t 검정을 해봅니다.

$$S_p^2 = \frac{(n_1-1)S_1^2 + (n_2-1)S_2^2}{n_1+n_2-2} \leftarrow \text{통합 분산(Pooled variance)라고 합니다.}$$

$$t = \frac{\bar{X}_1 - \bar{X}_2}{S_p \sqrt{1/n_1 + 1/n_2}} \sim t(n_1 + n_2 - 2)$$

In [96]:

```
'''
코드는 `df_prob2`라는 DataFrame을 사용하여 두 개의 통계적 가설 검정을 수행합니다.

1. `sp2_2` 계산:
- `df_prob2` DataFrame을 `AM` 컬럼을 기준으로 그룹화한 후, `Avg` 컬럼의 분산(`var`)과 개수(`count`)를 계산합니다.
'''
```


- ``assign``을 사용하여 ``count``에서 1을 뺀 값을 ``cntm1``이라는 새로운 컬럼으로 추가합니다.
- ``pipe``를 사용하여 ``cntm1``과 ``var`` 컬럼의 곱의 합을 ``cntm1`` 컬럼의 합으로 나누어 계산합니다.
이 결과는 그룹별 분산을 가중 평균한 값으로 볼 수 있습니다.

2. 첫 번째 t-통계량(``t1``) 계산:

- 다시 ``df_prob2`` DataFrame을 ``AM`` 컬럼을 기준으로 그룹화한 후, ``Avg`` 컬럼의 평균(``mean``)과 개수(``count``)를 계산합니다.
- ``pipe``를 사용하여 다음의 계산을 수행합니다:
 - 그룹 0의 평균에서 그룹 1의 평균을 뺍니다.
 - 분모는 ``sp2_2``에 각 그룹 개수의 역수를 더한 값에 제곱근을 곱하여 계산합니다.
- 이 식은 두 그룹의 평균 차이를 비교하는 t-통계량을 계산하는 것입니다.

3. 두 번째 t-통계량(``t2``) 계산:

- ``t1``과 동일한 과정이지만, 이번에는 그룹 1의 평균에서 그룹 0의 평균을 빼는 형태로 계산합니다.
- 이 역시 두 그룹의 평균 차이를 비교하는 t-통계량을 계산하는 것입니다.

이 코드의 전체적인 목적은 두 그룹 (``AM``이 0인 그룹과 1인 그룹)의 평균을 비교하여 t-통계량을 계산하는 것입니다.

``sp2_2``는 두 그룹의 분산을 가중 평균하여 계산한 것이고, 이를 통해 두 그룹의 평균 차이에 대한 통계적 유의성을 평가하려는 것입니다.

```
from scipy.stats import t
```

```
df_prob2.groupby('AM')['Avg'].agg(['var', 'count'])
```

Out[96]:

	var	count
AM		
0	157.907328	615
1	153.051088	618

```
In [97]: df_prob2.groupby('AM')['Avg'].agg(['var', 'count']).assign(
          cntm1 = lambda x: x['count'] - 1
        )
```

Out[97]:

	var	count	cntm1
AM			
0	157.907328	615	614
1	153.051088	618	617

```
In [98]: sp2_2 = df_prob2.groupby('AM')['Avg'].agg(['var', 'count']).assign(
          cntm1 = lambda x: x['count'] - 1
        ).pipe(
          lambda x: x['cntm1'].dot(x['var']) / x['cntm1'].sum()
        )

sp2_2
```

Out[98]: 155.47329084778724

```
In [99]: df_prob2.groupby('AM')['Avg'].agg(['mean', 'count']).pipe(lambda x: print(x))
```

	mean	count
AM		
0	170.282520	615
1	164.403722	618

```
In [100... t1 = df_prob2.groupby('AM')['Avg'].agg(['mean', 'count']).pipe(
          lambda x: (x.loc[0, 'mean'] - x.loc[1, 'mean']) / ((sp2_2 * (1 / x['count']).sum()) ** 0.5)
        )

t2 = df_prob2.groupby('AM')['Avg'].agg(['mean', 'count']).pipe(
          lambda x: (x.loc[1, 'mean'] - x.loc[0, 'mean']) / ((sp2_2 * (1 / x['count']).sum()) ** 0.5)
        )

# 양측 검장이므로 t1, t2 를 모두이용하여 통계량을 만들어 낼 수 있습니다.
# 하지만 단측 검증이라면 좌향의 표본평균 - 우향의 표본 평균 ... 기반으로 통계량을 만들어야 합니다.
dof = len(df_prob2) - 2
t1, t2, sp2_2, sp2, t.cdf(min(t1, t2), dof)
```

Out[100]: (8.277724093228105,
-8.277724093228105,
155.47329084778724,
155.47329084778724,
1.617440428199925e-16)

```
In [101... '''
이 코드는 t-분포를 시각화하고, 두 그룹 간의 평균 차이에 대한 t-통계량을 강조하여 표시합니다.
이를 통해 t-통계량이 분포 내에서 어디에 위치하는지를 시각적으로 보여줍니다.

1. t-통계량(`t_stat`) 계산:
```

- `t1`과 `t2` 중 작은 값을 `t_stat`로 설정합니다.
이는 양쪽 방향의 t-통계량을 고려하여 최종적으로 하나의 t-통계량을 선택한 것입니다.
- 2. 플롯 크기 설정:
 - 플롯의 크기를 6x4 인치로 설정합니다.
- 3. t-분포 플롯:
 - -10에서 10까지 100개의 점을 사용하여 t-분포 곡선을 그립니다.
`t.pdf`는 주어진 자유도(`dof`)에 따른 확률 밀도 함수입니다.
- 4. t-통계량을 표시하는 수직선 그리기:
 - t-통계량(`t_stat`) 위치에 빨간색 점선으로 수직선을 그립니다.
 - `plt.text`를 사용하여 t-통계량 값(`t_stat`)을 텍스트로 표시합니다.
텍스트 위치는 `(t_stat, 0.03)`입니다.
- 5. t-통계량 왼쪽 영역 채우기:
 - -10에서 `t_stat`까지의 영역을 주황색으로 채웁니다.
`t.pdf` 함수로 t-분포 값을 계산하여 y축 값으로 사용합니다.
 - 이 채움은 t-통계량 값 이하의 영역을 강조하여 유의미한 구역을 시각화합니다.
- 6. -t-통계량에 대한 수직선 그리기:
 - -t-통계량 위치에 빨간색 점선으로 수직선을 그립니다.
 - `plt.text`를 사용하여 -t-통계량 값을 텍스트로 표시합니다. 텍스트 위치는 `(-t_stat, 0.01)`입니다.
- 7. -t-통계량 오른쪽 영역 채우기:
 - -t_stat에서 10까지의 영역을 주황색으로 채웁니다.
마찬가지로 `t.pdf` 함수로 t-분포 값을 계산하여 y축 값으로 사용합니다.
 - 이 채움은 -t-통계량 값 이상의 영역을 강조하여 유의미한 구역을 시각화합니다.

이 코드의 전체적인 목적은 t-분포 곡선을 시각화하고, 계산된 t-통계량의 위치를 강조하여 표시함으로써 통계적 검정 결과를 시각적으로 명확하게 보여주는 것입니다. t-통계량의 위치와 유의 수준에 대한 시각적 이해를 돕습니다.

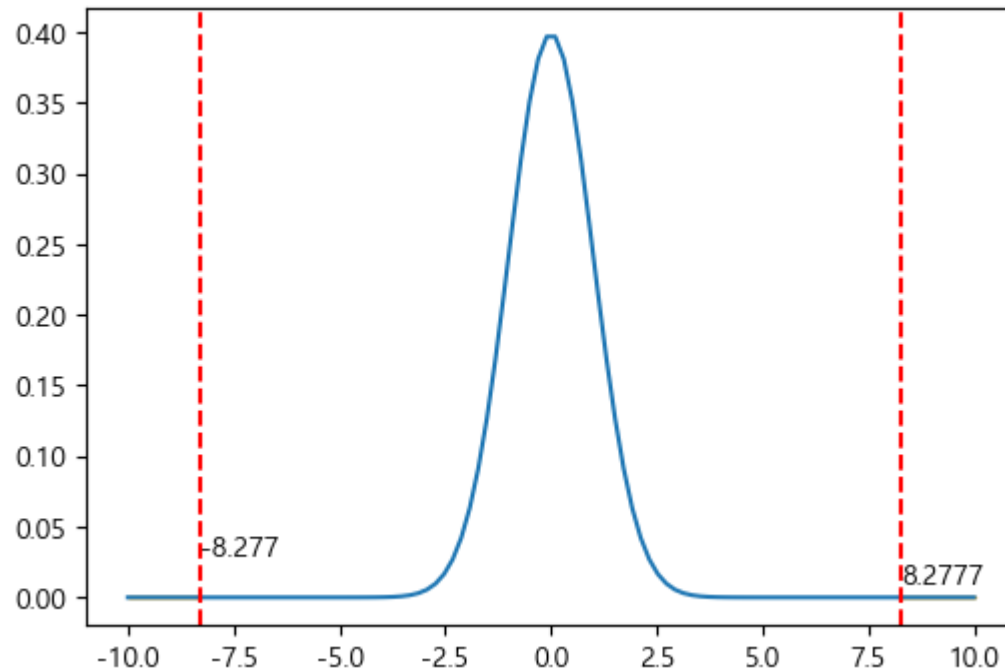
pvalue에 해당하는 영역을 도식화해봅니다.

```
t_stat = min(t1, t2)
plt.figure(figsize=(6, 4))

plt.plot(np.linspace(-10, 10, 100), t.pdf(np.linspace(-10, 10, 100), df=dof))

plt.axvline(t_stat, color='red', linestyle='--')
plt.text(t_stat, 0.03, str(t_stat)[:6])
plt.fill_between(np.linspace(-10, t_stat, 100), np.zeros(shape=100), t.pdf(np.linspace(-10, t_stat, 100), df=dof), color='orange')

plt.axvline(-t_stat, color='red', linestyle='--')
plt.text(-t_stat, 0.01, str(-t_stat)[:6])
plt.fill_between(np.linspace(-t_stat, 10, 100), np.zeros(shape=100), t.pdf(np.linspace(-t_stat, 10, 100), df=dof), color='orange')
plt.show()
```



단계 2-5

단계 2-2와 2-3의 검정 결과를 바탕으로, 단계 2-4의 T-검정의

가정을 만족한다면 D 값을 1 아니면 0으로 한다. (유의 수준을 5%)

```
In [102]: D = 1 if (A >= 0.05) and (B >= 0.05) else 0
          D
```

Out[102]: 0

단계 2-6

오전/오후에 따라 Avg를 구분하여 Mann-Whitney U 검정을 하라. 검정 결과 pvalue을 E값으로 한다.

Mann-Whitney U 검정 : 두 독립적인 정규분포를 따르지 않는 표본들 간의 median 차이 비교

대립 가설은 '오전의 Avg와 오후의 Avg의 중앙값은 같지 않다' 입니다.

함수가이드

scipy.stats.mannwhitneyu, alternative를 문제 요건에 반드시 맞추어 설정합니다.

문제 지시사항 외 Default 값 사용

```
In [103... from scipy.stats import mannwhitneyu

result_2_6 = mannwhitneyu(
    df_prob2.loc[df_prob2['AM'] == 0, 'Avg'],
    df_prob2.loc[df_prob2['AM'] == 1, 'Avg'],
    alternative='two-sided'
)

E = result_2_6.pvalue
result_2_6, E
```

```
Out[103]: (MannwhitneyuResult(statistic=239777.5, pvalue=1.7651446950415425e-15),
1.7651446950415425e-15)
```

```
In [104... from scipy.stats import mannwhitneyu

result_2_6 = mannwhitneyu(
    df_prob2.loc[df_prob2['AM'] == 1, 'Avg'],
    df_prob2.loc[df_prob2['AM'] == 0, 'Avg'],
    alternative='two-sided'
)

# 통계량은 다르지만 p-value는 동일
E = result_2_6.pvalue
result_2_6, E
```

```
Out[104]: (MannwhitneyuResult(statistic=140292.5, pvalue=1.7651446950415425e-15),
1.7651446950415425e-15)
```

```
In [105... A, B, C, D, E, A + B + C + D + E, round(A + B + C + D + E, 3)
```

```
Out[105]: (0.026428762823343277,
0.043661389499902725,
8.27772409322812,
0,
1.7651446950415425e-15,
8.347814245551367,
8.348)
```

A + B + C + D + E를 소수점 넷째 자리에서 반올림하여 셋째 자리까지 적으시오

8.348

단계 2-7, Optional

Avg에 대하여 AM, DayName 두 범주에 대하여

이원분산분석(Two-way ANOVA)을 수행하고 상호작용항의 검정통계량을 구하라.

함수가이드

statsmodels.formula.api.ols

statsmodels.api.stats

문제 지시사항 외 Default 값 사용

In [106...

```
'''
`ols('Avg ~ C(AM) + C(DayName) + C(AM):C(DayName)')`에서 `C()`를 사용하는 이유는
변수 `AM`과 `DayName`을 범주형 변수(categorical variable)로 처리하기 위해서입니다.
이 방식은 Python의 `statsmodels` 라이브러리에서 사용되는 구문입니다. 구체적인 이유는 다음과 같습니다:

1. 범주형 변수로 처리하기
- `C()` 함수는 지정된 변수를 범주형 변수로 간주하여 모델에 포함시킵니다.
  범주형 변수는 몇 가지 고유한 수준(또는 카테고리)만 가질 수 있는 변수입니다.
- 예를 들어, `AM`이 0 또는 1만 가질 수 있고, `DayName`이 요일을 나타내는 문자열(`Monday`, `Tuesday`, 등)을 가질 수 있는 경우,
  이러한 변수를 범주형으로 처리하면 각 수준에 대해 별도의 더미 변수를 생성합니다.

2. 더미 변수 생성
```

- ``C(AM)``과 ``C(DayName)``은 각각의 카테고리에 대해 더미 변수를 생성합니다.
예를 들어, ``AM``이 0과 1이라는 두 개의 카테고리를 가지면, 이 변수는 회귀 모델에서 두 개의 더미 변수로 확장됩니다.
- ``DayName``이 7개의 요일을 나타내면, 7개의 더미 변수로 확장됩니다.
- 3. 상호작용 항의 처리
 - ``C(AM):C(DayName)``는 ``AM``과 ``DayName`` 간의 상호작용 항을 포함시킵니다.
이는 두 범주형 변수의 조합이 종속 변수(``Avg``)에 미치는 영향을 모델링합니다.
 - 상호작용 항을 포함하면, ``AM``의 특정 수준과 ``DayName``의 특정 수준이 결합될 때 ``Avg``에 미치는 추가적인 효과를 평가할 수 있습니다.
- 4. 모델 해석의 용이성
 - 범주형 변수를 더미 변수로 변환하고, 이를 통해 각 수준이 종속 변수에 미치는 영향을 별도로 평가할 수 있습니다.
이 방식은 모델의 해석을 보다 직관적으로 만들고, 각 범주의 효과를 명확히 평가할 수 있게 합니다.
 - 예를 들어, ``AM``이 0일 때와 1일 때의 평균 차이, 특정 요일이 ``Avg``에 미치는 영향을 각각 평가할 수 있습니다.

요약

``C()``를 사용하여 범주형 변수를 지정하면, 회귀 모델은 각 범주의 영향을 개별적으로 평가할 수 있도록 더미 변수를 생성하고, 상호작용 항을 통해 변수 간의 복합적인 영향을 모델링할 수 있습니다.

이는 모델의 해석을 용이하게 하고, 데이터의 특성을 보다 명확히 반영할 수 있게 합니다.

...

...

선형 회귀의 결과를 ANOVA로 전달하는 방식은 모델의 다양한 요인과 그 상호작용이 종속 변수에 미치는 영향을 평가하는데 유용합니다. 다음은 이 방식이 사용되는 이유와 그 이점을 자세히 설명합니다:

1. 모델의 요인 분석

- 선형 회귀는 연속형 종속 변수와 하나 이상의 독립 변수 간의 관계를 설명합니다.
여기서, 독립 변수는 연속형일 수도 있고 범주형일 수도 있습니다.
모델 적합 후, 각 독립 변수의 회귀 계수와 이들의 유의성을 평가할 수 있습니다.
- ANOVA (Analysis of Variance)는 모델에 포함된 요인(독립 변수)의 각 수준이 종속 변수에 미치는 영향을 평가합니다.
특히, ANOVA는 범주형 변수와 그들의 상호작용을 평가하는 데 유용합니다.

2. 상호작용 효과 평가

- 선형 회귀 모델에서 여러 독립 변수가 포함될 때, 이들 간의 상호작용 효과를 포함시킬 수 있습니다.
상호작용 효과는 두 개 이상의 변수 간의 조합이 종속 변수에 미치는 추가적인 영향을 나타냅니다.
- ANOVA는 이러한 상호작용 효과의 통계적 유의성을 평가하는 데 매우 유용합니다.
상호작용 효과가 유의미하다면, 해당 변수들이 종속 변수에 복합적으로 영향을 미친다는 것을 의미합니다.

3. 모델의 적합성 평가

- ANOVA는 선형 회귀 모델의 적합성을 평가하는 중요한 도구입니다.
모델의 각 요인이 종속 변수에 미치는 영향을 분해하여 설명할 수 있으며, 각 요인의 F-값을 통해 해당 요인의 유의성을 평가합니다.
- F-값은 각 요인의 분산 비율을 나타내며, 이 값이 클수록 해당 요인이 종속 변수에 미치는 영향이 크다는 것을 의미합니다.

4. 통계적 유의성 검정

- 선형 회귀 모델에서는 회귀 계수의 t-검정을 통해 각 독립 변수의 유의성을 평가할 수 있습니다.
그러나, 여러 요인과 상호작용 효과를 포함하는 복잡한 모델에서는 ANOVA를 통해 각 요인과 상호작용 효과의 유의성을 종합적으로 평가하는 것이 더 유리합니다.

- ANOVA는 모델의 설명력과 잔차의 분산을 비교하여 요인의 유의성을 평가합니다.
이를 통해 전체 모델의 설명력을 향상시킬 수 있는 요인을 식별할 수 있습니다.
- 5. 다양한 요인의 비교
- ANOVA는 여러 요인과 그 조합을 동시에 비교할 수 있습니다.
이는 단순 회귀 분석으로는 어려운 작업입니다. 특히, 복잡한 데이터셋에서 다양한 요인과 상호작용을 동시에 고려하여 모델의 적합성을 평가할 수 있습니다.
- 이 방법은 데이터의 다차원적 특성을 반영하여 보다 포괄적인 분석을 가능하게 합니다.

코드 구현의 이유

구체적으로, 코드에서 선형 회귀의 결과를 ANOVA로 전달하는 이유는 다음과 같습니다:

1. 다양한 요인 및 상호작용 효과 평가:
`AM`과 `DayName`의 주 효과뿐만 아니라 이들의 상호작용 효과(`C(AM):C(DayName)`)가 `Avg`에 미치는 영향을 평가하기 위해서입니다.
2. 모델 적합성 검토:
선형 회귀 모델을 통해 각 요인의 회귀 계수를 계산한 후, ANOVA를 통해 이 요인들이 모델 전체에 미치는 영향을 평가합니다.
3. 통계적 유의성 판단:
F-값을 통해 요인과 상호작용의 통계적 유의성을 판단하여 중요한 요인들을 식별하고, 이를 통해 모델을 개선할 수 있습니다.

요약

선형 회귀의 결과를 ANOVA로 전달하는 방식은 복잡한 모델에서 다양한 요인과 상호작용 효과를 평가하고, 모델의 적합성을 종합적으로 검토하기 위해 사용됩니다.

이는 특히 범주형 변수와 그 상호작용을 포함하는 경우에 유용하며, 통계적 유의성을 평가하는 데 중요한 도구입니다.

...

...

이 코드는 두 그룹 간의 독립 t-검정을 수행한 후, 선형 회귀 분석 및 ANOVA(분산분석)를 통해 특정 상호작용 효과를 평가합니다.

1. 회귀 분석 및 ANOVA 준비:

```
C_ = C
```

```
del C
```

- 이전에 저장된 t-통계량의 절대값 `C`를 `C\_`에 백업합니다.

- `C` 변수를 삭제합니다. 이는 이후 코드에서 `C` 변수가 사용될 수 있도록 하기 위함입니다.

2. 선형 회귀 모델 적합:

```
lm = ols('Avg ~ C(AM) + C(DayName) + C(AM):C(DayName)', df_prob2).fit()
```

- `df\_prob2` 데이터프레임을 사용하여 선형 회귀 모델을 적합합니다.

- 종속 변수는 `Avg`이고, 독립 변수는 `AM`과 `DayName`의 범주형 변수와 이들 간의 상호작용 항입니다(`C(AM):C(DayName)`).

- `ols` 함수는 주어진 모델을 적합하고, `fit` 메서드는 모델을 피팅하여 `lm` 객체에 저장합니다.

3. ANOVA 수행:

```
C = C_
```

```
df_anova = stats.anova_lm(lm)
```

- `C` 변수를 원래 값으로 복원합니다.

- `anova\_lm` 함수를 사용하여 `lm` 객체에 대해 ANOVA를 수행하고 결과를 `df\_anova`에 저장합니다.

4. 특정 상호작용 효과의 F-값 추출:


```
df_anova.loc[
    'C(AM):C(DayName)', 'F'
]
```

- `df\_anova`에서 `C(AM):C(DayName)` 항목의 F-값을 추출합니다.
- 이는 `AM`과 `DayName` 간의 상호작용 효과가 통계적으로 유의한지 평가하는데 사용됩니다.

코드의 전체적인 목적은 `AM`과 `DayName` 간의 상호작용이 `Avg`에 미치는 영향을 평가하는 것입니다. t-검정을 통해 두 그룹 간 평균 차이를 확인한 후, 선형 회귀 및 ANOVA를 통해 상호작용 효과를 분석합니다. ANOVA의 F-값은 이 상호작용 효과의 통계적 유의성을 평가하는 데 사용됩니다.

```
'''

from statsmodels.formula.api import ols
from statsmodels.api import stats

# stats.anova_lm
C_ = C
del C

lm = ols('Avg ~ C(AM) + C(DayName) + C(AM):C(DayName)', df_prob2).fit()
print(type(lm))
print('-----')

C = C_

lm.summary()
```

```
<class 'statsmodels.regression.linear_model.RegressionResultsWrapper'>
```

```
-----
```

Out[106]:

OLS Regression Results

Dep. Variable:	Avg	R-squared:	0.056
Model:	OLS	Adj. R-squared:	0.049
Method:	Least Squares	F-statistic:	8.115
Date:	Wed, 31 Jul 2024	Prob (F-statistic):	9.11e-12
Time:	22:41:05	Log-Likelihood:	-4857.3
No. Observations:	1233	AIC:	9735.
Df Residuals:	1223	BIC:	9786.
Df Model:	9		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	171.0974	1.071	159.809	0.000	168.997	173.198
C(AM)[T.1]	-5.9687	1.514	-3.942	0.000	-8.939	-2.998
C(DayName)[T.1]	-1.0554	1.547	-0.682	0.495	-4.091	1.980
C(DayName)[T.2]	-0.5595	1.593	-0.351	0.726	-3.685	2.566
C(DayName)[T.3]	-0.7765	1.520	-0.511	0.609	-3.758	2.205
C(DayName)[T.4]	-1.8752	1.609	-1.165	0.244	-5.032	1.282
C(AM)[T.1]:C(DayName)[T.1]	0.5960	2.183	0.273	0.785	-3.688	4.880
C(AM)[T.1]:C(DayName)[T.2]	-0.1741	2.253	-0.077	0.938	-4.594	4.246
C(AM)[T.1]:C(DayName)[T.3]	0.5904	2.147	0.275	0.783	-3.622	4.803
C(AM)[T.1]:C(DayName)[T.4]	-0.7396	2.276	-0.325	0.745	-5.205	3.725

Omnibus:	4.990	Durbin-Watson:	0.766
Prob(Omnibus):	0.082	Jarque-Bera (JB):	5.064
Skew:	0.153	Prob(JB):	0.0795
Kurtosis:	2.926	Cond. No.	14.7

Warnings:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [107... df_anova = stats.anova_lm(lm)
df_anova
```

```
Out[107]:
```

	df	sum_sq	mean_sq	F	PR(>F)
C(AM)	1.0	10653.141233	10653.141233	68.336721	3.554886e-16
C(DayName)	4.0	657.233752	164.308438	1.053990	3.780555e-01
C(AM):C(DayName)	4.0	74.594339	18.648585	0.119625	9.755319e-01
Residual	1223.0	190655.792942	155.891899	NaN	NaN

```
In [108... '''
ANOVA는 변수들 간의 상호작용과 그들이 설명하는 분산의 양을 분석하는 데 사용됩니다.

### ANOVA 테이블 해석

- df (degrees of freedom): 각 항목에 대한 자유도입니다.
  - C(AM): 독립 변수 AM의 자유도는 1입니다.
  - C(DayName): 독립 변수 DayName의 자유도는 4입니다.
  - C(AM):C(DayName): AM과 DayName 간의 상호작용 항목의 자유도는 4입니다.
  - Residual: 잔차 항목의 자유도는 1223입니다.

- sum_sq (sum of squares): 각 항목에 대한 제곱합입니다.
  - 이 값은 각 항목이 전체 분산에서 설명하는 총 변동량을 나타냅니다.

- mean_sq (mean square): 각 항목에 대한 평균 제곱입니다.
  - mean_sq는 sum_sq를 해당 항목의 자유도로 나눈 값입니다.

- F (F-statistic): 각 항목에 대한 F 통계량입니다.
  - F 통계량은 해당 항목의 mean_sq를 잔차의 mean_sq로 나눈 값입니다.
  - F 통계량이 클수록 해당 항목이 모델에 유의미한 영향을 미칠 가능성이 높습니다.

- PR(>F) (p-value): F 통계량에 대한 p-값입니다.
  - 이 값이 작을수록 (보통 0.05보다 작으면) 해당 항목이 통계적으로 유의미하다고 판단합니다.
```

해석

1. C(AM)

- F 통계량: 68.336721
- p-값: 3.554886e-16 (0.0000000000000003554886)
- 해석: p-값이 매우 작기 때문에 AM 변수는 통계적으로 유의미한 영향을 미칩니다.
즉, AM 변수는 모델에서 중요한 역할을 하고 있다고 볼 수 있습니다.

2. C(DayName)

- F 통계량: 1.053990
- p-값: 3.780555e-01 (0.3780555)
- 해석: p-값이 0.05보다 크기 때문에 DayName 변수는 통계적으로 유의미하지 않습니다.
즉, DayName 변수는 모델에서 중요한 영향을 미치지 않습니다.

3. C(AM):C(DayName)

- F 통계량: 0.119625
- p-값: 9.755319e-01 (0.9755319)
- 해석: p-값이 0.05보다 훨씬 크기 때문에 AM와 DayName 간의 상호작용은 통계적으로 유의미하지 않습니다.
즉, 두 변수 간의 상호작용이 모델에 중요한 영향을 미치지 않습니다.

4. Residual

- 잔차 항목은 모델이 설명하지 못하는 변동을 나타냅니다.
- 잔차의 자유도는 1223이고, 제곱합은 190655.792942입니다.
- 평균 제곱은 155.891899입니다.

결론

- AM 변수는 모델에 유의미한 영향을 미치며, 모델에서 중요한 변수입니다.
- DayName 변수와 AM와 DayName 간의 상호작용은 모델에 유의미한 영향을 미치지 않으며, 중요한 변수가 아닙니다.
- 잔차 제곱합이 크므로 모델이 설명하지 못하는 변동이 상당히 존재합니다.

이러한 해석을 통해, ANOVA 결과를 바탕으로 모델을 개선하거나 변수 선택을 다시 고려할 수 있습니다.

```
'''
```

```
print('')
```

In [109...

```
df_anova.loc[
    'C(AM):C(DayName)', 'F'
]
```

```
Out[109]: 0.1196251046627005
```

문제 3

DateHour를 기준으로 다음날 같은 시각의 최대전력사용량을 예측하려고 한다.

단계 3-1

basetable1에 아래 그림과 같이, DateHour를 기준으로 다음날 같은 시각의 TotalHour로 파생 변수 target을 생성한다.

target에 결측치가 있는 행은 모두 제거하여 prob3 데이터셋을 만든다.

DateHour	TotalHour
2021-01-02 00:00:00	253
2021-01-02 01:00:00	418
2021-01-02 02:00:00	415
2021-01-02 03:00:00	421
2021-01-02 04:00:00	427

DateHour 기준으로 다음날 같은 시각의 TotalHour 를
파생변수로 추가

2021-01-02 00:00:00 이후 TotalHour

DateHour	15 분	30 분	45 분	60 분	DayName	Hour	AM	Weekend_yn	Holiday_yn	Avg	TotalHour	생 산 량	기 온	풍 속	습 도	강 수 량	전기요 금(계 절)	공장 인원	인 건 비	target
2021-01-01 00:00:00	62	61	61	61	4	0	True	0	1	61.25	245	0	-3.2	2.4	71	0.0	109.8	0.0	1.5	253.0
2021-01-01 01:00:00	96	93	116	113	4	1	True	0	1	104.50	418	0	-4.5	1.5	77	0.0	109.8	0.0	1.5	418.0
2021-01-01 02:00:00	106	96	106	107	4	2	True	0	1	103.75	415	0	-3.9	2.6	58	0.0	109.8	0.0	1.5	415.0
2021-01-01 03:00:00	92	110	110	109	4	3	True	0	1	105.25	421	0	-4.1	2.6	56	0.0	109.8	0.0	1.5	421.0
2021-01-01 04:00:00	108	105	106	108	4	4	True	0	1	106.75	427	0	-4.6	2.6	60	0.0	109.8	0.0	1.5	427.0

In [110... df_basetable1.index[1:]

```
Out[110]: DatetimeIndex(['2021-01-01 01:00:00', '2021-01-01 02:00:00',
                        '2021-01-01 03:00:00', '2021-01-01 04:00:00',
                        '2021-01-01 05:00:00', '2021-01-01 06:00:00',
                        '2021-01-01 07:00:00', '2021-01-01 08:00:00',
                        '2021-01-01 09:00:00', '2021-01-01 10:00:00',
                        ...
                        '2021-09-14 14:00:00', '2021-09-14 15:00:00',
                        '2021-09-14 16:00:00', '2021-09-14 17:00:00',
                        '2021-09-14 18:00:00', '2021-09-14 19:00:00',
                        '2021-09-14 20:00:00', '2021-09-14 21:00:00',
                        '2021-09-14 22:00:00', '2021-09-14 23:00:00'],
                        dtype='datetime64[ns]', name='DateHour', length=6167, freq=None)
```

```
In [111... df_basetable1.index[: -1]
```

```
Out[111]: DatetimeIndex(['2021-01-01 00:00:00', '2021-01-01 01:00:00',
                        '2021-01-01 02:00:00', '2021-01-01 03:00:00',
                        '2021-01-01 04:00:00', '2021-01-01 05:00:00',
                        '2021-01-01 06:00:00', '2021-01-01 07:00:00',
                        '2021-01-01 08:00:00', '2021-01-01 09:00:00',
                        ...
                        '2021-09-14 13:00:00', '2021-09-14 14:00:00',
                        '2021-09-14 15:00:00', '2021-09-14 16:00:00',
                        '2021-09-14 17:00:00', '2021-09-14 18:00:00',
                        '2021-09-14 19:00:00', '2021-09-14 20:00:00',
                        '2021-09-14 21:00:00', '2021-09-14 22:00:00'],
                        dtype='datetime64[ns]', name='DateHour', length=6167, freq=None)
```

```
In [112... # 방법 1: 모든 레코드가 1시간 간격으로 있다면, DateHour로 정렬했을 때,
# 24개 이후의 내용을 당긴다(shift -24) 연산을 통해 처리

# 확인 루틴: 모두 한시간 간격으로 있는지 확인하여, 이 방법이 성립할 지 살펴봅니다.
(df_basetable1.index[1:] - df_basetable1.index[: -1]).value_counts()
```

```
Out[112]: 01:00:00    6167
Name: DateHour, dtype: int64
```

```
In [113... '''
코드는 `pandas` 라이브러리를 사용하여 DataFrame을 변형하는 작업을 수행합니다.
`df_basetable1` DataFrame에서 특정 변수를 추가하고, 일부 행을 필터링하여 새로운 DataFrame `df_prob3`을 생성합니다.

1. `assign` 메서드 사용:
```

- `df_basetable1` DataFrame에 `assign` 메서드를 사용하여 새로운 열을 추가합니다.
- `target`이라는 이름의 새로운 열을 생성하는데, 이 열의 값은 `df_basetable1`의 `TotalHour` 열의 값을 24행 아래로 이동시킨 것입
`shift(-24)`는 원래 값이 있는 위치에서 24행 아래로 값을 이동시키며, 상위 24개의 값은 NaN이 됩니다.

예를 들어, 원래 `TotalHour` 열이 다음과 같다면:

```
0      1
1      2
2      3
3      4
...
23    24
24    25
25    26
...
47    48
```

`shift(-24)`를 사용하면 다음과 같이 변경됩니다:

```
0      25
1      26
2      27
3      28
...
23    48
24    NaN
25    NaN
...
47    NaN
```

2. `pipe` 메서드 사용:

- `pipe` 메서드는 체인 메서드 호출을 가능하게 해줍니다.
이 메서드는 DataFrame을 함수에 전달합니다.
- 여기서는 람다 함수를 사용하여 `target` 열의 값이 NaN이 아닌 행만 선택합니다.
- `x.loc[x['target'].notna()]`는 `target` 열의 값이 NaN이 아닌 행을 필터링합니다.
`notna()`는 값이 NaN이 아닌지 확인하는 메서드입니다.

위의 과정을 통해 `df_prob3`는 `df_basetable1`에서 `TotalHour` 열을 24행 아래로 이동시킨 `target` 열을 추가하고,
`target` 열의 값이 NaN이 아닌 행만 포함하는 DataFrame이 됩니다.

```
...
```

```
df_prob3 = df_basetable1.assign(
    target = df_basetable1['TotalHour'].shift(-24)
).pipe(
```



```

    lambda x: x.loc[x['target'].notna()]
)

display(df_prob3.head())
df_prob3.shape

```

	15 분	30 분	45 분	60 분	DayName	Hour	AM	Weekend_yn	Holiday_yn	Avg	TotalHour	생 산 량	기 온	풍 속	습 도	강 수 량	전기요 금(계 절)	공 장 인 원	인 건 비	target	
DateHour																					
2021-01-01 00:00:00	62	61	61	61		4	0	0		0	61.25	245	0	-3.2	2.4	71	0.0	109.8	0.0	1.5	253.0
2021-01-01 01:00:00	96	93	116	113		4	1	0		0	104.50	418	0	-4.5	1.5	77	0.0	109.8	0.0	1.5	418.0
2021-01-01 02:00:00	106	96	106	107		4	2	0		0	103.75	415	0	-3.9	2.6	58	0.0	109.8	0.0	1.5	415.0
2021-01-01 03:00:00	92	110	110	109		4	3	0		0	105.25	421	0	-4.1	2.6	56	0.0	109.8	0.0	1.5	421.0
2021-01-01 04:00:00	108	105	106	108		4	4	0		0	106.75	427	0	-4.6	2.6	60	0.0	109.8	0.0	1.5	427.0

Out[113]: (6144, 20)

In [114... df_basetable1['TotalHour'].index

```
Out[114]: DatetimeIndex(['2021-01-01 00:00:00', '2021-01-01 01:00:00',
                        '2021-01-01 02:00:00', '2021-01-01 03:00:00',
                        '2021-01-01 04:00:00', '2021-01-01 05:00:00',
                        '2021-01-01 06:00:00', '2021-01-01 07:00:00',
                        '2021-01-01 08:00:00', '2021-01-01 09:00:00',
                        ...
                        '2021-09-14 14:00:00', '2021-09-14 15:00:00',
                        '2021-09-14 16:00:00', '2021-09-14 17:00:00',
                        '2021-09-14 18:00:00', '2021-09-14 19:00:00',
                        '2021-09-14 20:00:00', '2021-09-14 21:00:00',
                        '2021-09-14 22:00:00', '2021-09-14 23:00:00'],
                        dtype='datetime64[ns]', name='DateHour', length=6168, freq=None)
```

```
In [115... pd.Timedelta(days=1)
```

```
Out[115]: Timedelta('1 days 00:00:00')
```

```
In [116... aaa = df_basetable1['TotalHour'].index - pd.Timedelta(days=1)
aaa
```

```
Out[116]: DatetimeIndex(['2020-12-31 00:00:00', '2020-12-31 01:00:00',
                        '2020-12-31 02:00:00', '2020-12-31 03:00:00',
                        '2020-12-31 04:00:00', '2020-12-31 05:00:00',
                        '2020-12-31 06:00:00', '2020-12-31 07:00:00',
                        '2020-12-31 08:00:00', '2020-12-31 09:00:00',
                        ...
                        '2021-09-13 14:00:00', '2021-09-13 15:00:00',
                        '2021-09-13 16:00:00', '2021-09-13 17:00:00',
                        '2021-09-13 18:00:00', '2021-09-13 19:00:00',
                        '2021-09-13 20:00:00', '2021-09-13 21:00:00',
                        '2021-09-13 22:00:00', '2021-09-13 23:00:00'],
                        dtype='datetime64[ns]', name='DateHour', length=6168, freq='H')
```

```
In [117...
```

```
'''
```

코드는 `pandas` 라이브러리를 사용하여 DataFrame을 변형하는 작업을 수행합니다.

주어진 DataFrame `df\_basetable1`에 새로운 열을 추가하고, 특정 조건을 만족하는 행만 필터링하여 새로운 DataFrame `df\_prob3`을 생성함

1. `assign` 메서드 사용:

- `df\_basetable1` DataFrame에 `assign` 메서드를 사용하여 `target`이라는 새로운 열을 추가합니다.

- `target` 열은 `pd.Series`를 사용하여 생성됩니다.

 `pd.Series`는 `df\_basetable1['TotalHour']` 열의 값을 사용하며, 이 값들의 인덱스를 원래 인덱스에서 1일을 뺀 값으로 설정합니

`pd.Timedelta(days=1)`을 사용하여 인덱스를 1일 앞으로 이동시킵니다.

이 작업의 결과로, `target` 열의 값은 `TotalHour` 열의 값과 동일하지만, 인덱스는 하루 전으로 설정됩니다. 예를 들어, 원래 인덱스가 날짜인 경우 2023-06-10의 값이 2023-06-09에 대응되도록 이동합니다.

원래 `TotalHour` 열이 다음과 같다면:

```
Date
2023-06-10    1
2023-06-11    2
2023-06-12    3
2023-06-13    4
```

`pd.Timedelta(days=1)`를 사용하면 다음과 같이 변경됩니다:

```
Date
2023-06-09    1
2023-06-10    2
2023-06-11    3
2023-06-12    4
```

2. `pipe` 메서드 사용:

- `pipe` 메서드는 체인 메서드 호출을 가능하게 해줍니다.
이 메서드는 `DataFrame`을 함수에 전달합니다.
- 여기서는 람다 함수를 사용하여 `target` 열의 값이 `NaN`이 아닌 행만 선택합니다.
- `x.loc[x['target'].notna()]`는 `target` 열의 값이 `NaN`이 아닌 행을 필터링합니다.
`notna()`는 값이 `NaN`이 아닌지 확인하는 메서드입니다.

위의 과정을 통해 `df_prob3`는 `df_basetable1`에서 `TotalHour` 열의 값을 하루 전의 인덱스로 이동시킨 `target` 열을 추가하고, `target` 열의 값이 `NaN`이 아닌 행만 포함하는 `DataFrame`이 됩니다.

...

방법 2 `pandas`는 인덱스 기준으로 대입 연산(`assign`)이 됨을 이용합니다.
위치기준이 아니고 인덱스 기준 대입 연산이므로 때문에
위치가 바뀌어도 대입의 결과는 변하지 않습니다.

```
df_prob3 = df_basetable1.assign(
    target = pd.Series(
        df_basetable1['TotalHour'].values,
        index = df_basetable1['TotalHour'].index - pd.Timedelta(days=1)
    )
).pipe(
    lambda x: x.loc[x['target'].notna()]
)
```

```
display(df_prob3.head())
df_prob3.shape
```

	15 분	30 분	45 분	60 분	DayName	Hour	AM	Weekend_yn	Holiday_yn	Avg	TotalHour	생 산 량	기 온	풍 속	습 도	강 수 량	전기요 금(계 절)	공 장 인 원	인 건 비	target	
DateHour																					
2021-01-01 00:00:00	62	61	61	61	4	0	0		0	1	61.25	245	0	-3.2	2.4	71	0.0	109.8	0.0	1.5	253.0
2021-01-01 01:00:00	96	93	116	113	4	1	0		0	1	104.50	418	0	-4.5	1.5	77	0.0	109.8	0.0	1.5	418.0
2021-01-01 02:00:00	106	96	106	107	4	2	0		0	1	103.75	415	0	-3.9	2.6	58	0.0	109.8	0.0	1.5	415.0
2021-01-01 03:00:00	92	110	110	109	4	3	0		0	1	105.25	421	0	-4.1	2.6	56	0.0	109.8	0.0	1.5	421.0
2021-01-01 04:00:00	108	105	106	108	4	4	0		0	1	106.75	427	0	-4.6	2.6	60	0.0	109.8	0.0	1.5	427.0

Out[117]: (6144, 20)

In [118...

```
'''
두 코드 모두 `df_basetable1` 데이터프레임을 기반으로 새로운 데이터프레임 `df_prob3`를 생성하고 있습니다.
이 과정에서 `TotalHour` 값을 이용하여 `target` 컬럼을 추가하고, `target` 값이 `NaN`이 아닌 행만을 선택합니다.
하지만, 두 코드가 `target` 컬럼을 생성하는 방식은 다릅니다.

# 첫 번째 코드:
df_prob3 = df_basetable1.assign(
    target = df_basetable1['TotalHour'].shift(-24)
).pipe(
    lambda x: x.loc[x['target'].notna()]
)

## 동작 방식:
1. `assign` 메서드:
   - `target` 컬럼을 추가합니다.
```

- `df_basetable1['TotalHour'].shift(-24)`는 `TotalHour` 컬럼을 24행 앞으로 이동합니다.
이는 각 행의 `target` 값이 현재 행에서 24시간 후의 `TotalHour` 값을 갖도록 합니다.
- 예를 들어, 현재 행이 2023-01-01 00:00이라면, `target` 값은 2023-01-02 00:00의 `TotalHour` 값이 됩니다.

2. `pipe` 메서드:

- `target` 값이 `NaN`이 아닌 행만 선택합니다.
- `shift(-24)`로 인해 마지막 24행은 `NaN`이 되어 제거됩니다.

예시:

Original:		Shifted by -24:	
DateHour	TotalHour	DateHour	TotalHour
2023-01-01	100	2023-01-02	150
2023-01-02	150	2023-01-03	200
...		...	
2023-01-24	300	NaT	NaN

두 번째 코드:

```
df_prob3 = df_basetable1.assign(
    target = pd.Series(
        df_basetable1['TotalHour'].values,
        index = df_basetable1['TotalHour'].index - pd.Timedelta(days=1)
    )
).pipe(
    lambda x: x.loc[x['target'].notna()]
)
```

동작 방식:

1. `assign` 메서드:

- `target` 컬럼을 추가합니다.
- `pd.Series(df_basetable1['TotalHour'].values, index = df_basetable1['TotalHour'].index - pd.Timedelta(days=1))`는 `TotalHour` 값을 원래 인덱스보다 1일 앞선 위치에 배치하는 `Series`를 생성합니다.
- 즉, `df_basetable1`의 각 행의 `target` 값은 현재 행보다 하루 전의 `TotalHour` 값이 됩니다.

2. `pipe` 메서드:

- `target` 값이 `NaN`이 아닌 행만 선택합니다.
- `index - pd.Timedelta(days=1)`로 인해 첫 번째 행의 `target` 값은 `NaN`이 되어 제거됩니다.

예시:

Original:		Shifted by -1 day index:	
DateHour	TotalHour	DateHour	TotalHour
2023-01-01	100	2023-01-02	100

```

2023-01-02      150      2023-01-03      150
...
2023-01-24      300      NaT              NaN

# 차이점 요약:
- 시간 이동 방식:
  - 첫 번째 코드: `shift(-24)`를 사용하여 24시간 후의 값을 `target`으로 설정합니다.
    이는 같은 인덱스를 유지한 채 값만 이동합니다.
  - 두 번째 코드: `index - pd.Timedelta(days=1)`를 사용하여 1일 전의 값을 `target`으로 설정합니다.
    이는 인덱스를 조정하여 1일 전 위치에 값을 배치합니다.

- NaN 행 제거:
  - 첫 번째 코드: `shift(-24)`로 인해 마지막 24행이 `NaN`이 되어 제거됩니다.
  - 두 번째 코드: 인덱스를 1일 전으로 조정함으로써 첫 번째 행이 `NaN`이 되어 제거됩니다.

# 결론:
두 코드 모두 `df_basetable1` 데이터프레임에서 `TotalHour` 값을 기반으로 `target` 컬럼을 생성하지만,
`target` 컬럼의 생성 방식과 이로 인해 `NaN`이 발생하는 위치가 다릅니다.
첫 번째 코드는 24시간 뒤의 값을 사용하고, 두 번째 코드는 1일 전의 값을 사용합니다.
'''

print('')

```

단계 3-2

prob3을 학습용과 테스트용 데이터셋을 2021년 8월 14일 전과 이후로 분리

. prob3_train: DateHour 변수 기준으로 2021년 8월 14일 전(8월 14일 미포함) 데이터 (행의 수: 5400개)

. prob3_test: DateHour 변수 기준으로 2021년 8월 14일 이후(8월 14일 포함) 데이터 (행의 수: 744 개)

```

In [119... # 비교연산 같은 경우에는 datetime 형식과 str도 성립하지만,
# 보다 명시적인 방법을 사용하여 datetime으로 나타내어 비교를 하도록하여
# 데이터를 나눕니다.
# 수정이 있을 경우 이슈가 없도록 copy를 합니다.

```

```

df_prob3_train = df_prob3.loc[df_prob3.index < pd.to_datetime('2021-08-14')].copy()
df_prob3_test = df_prob3.loc[df_prob3.index >= pd.to_datetime('2021-08-14')].copy()

```

```
df_prob3_train.shape, df_prob3_test.shape
```

```
Out[119]: ((5400, 20), (744, 20))
```

```
In [120... Image('8.png')
```

Out[120]:

단계 3-3

결정트리 회귀모델(Decision Tree Regression Model)을 사용하여

target을 예측하는 모델을 prob3_train으로 학습시킨다.

요건은 다음과 같다.

성능 측정 지표: $MAE(Y, \hat{Y}) = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$

입력 변수: 15분, 30분, 45분, 60분, DayName, Hour, AM, Weekend_yn, Holiday_yn, Avg, TotalHour,

생산량, 기온, 풍속, 습도, 강수량, 전기요금(계절), 공장인원, 인건비

※ 입력 변수의 순서는 반드시 나열한 순서로 해야 함.

대상 변수: target

결정 트리 구성 요건:

하이퍼 파라미터	값
트리의 최대 깊이	10
리프 노드(Leaf Node)의 최소 샘플 수	4
랜덤 시드	123

함수가이드

sklearn.tree 제공 기능 활용

문제 지시 사항 외 Default 값 사용

모델의 학습 결과 가장 중요도가 높은 변수의 입력 순서를 A (순서는 1부터 시작) 라고 한다.

결정 트리의 prob3_test에 대한 성능을 B라고 한다.


```
In [121... from sklearn.metrics import mean_absolute_error # 성능 측정 지표: MAE
from sklearn.tree import DecisionTreeRegressor, DecisionTreeClassifier

# 입력변수를 DataFrame에서의 입력 변수와 동일하게 제시 되었는데요,
# 문자열 연산을 통해 입력 변수 리스트를 간단히 만들어 봅니다.
X_dt = '15분, 30분, 45분, 60분, DayName, Hour, AM, Weekend_yn, Holiday_yn, Avg, \
      TotalHour, 생산량, 기온, 풍속, 습도, 강수량, 전기요금(계절), 공장인원, 인건비'
X_dt = [i.strip() for i in X_dt.split(',')]

print(X_dt)
print(type(X_dt))
```

```
['15분', '30분', '45분', '60분', 'DayName', 'Hour', 'AM', 'Weekend_yn', 'Holiday_yn', 'Avg', 'TotalHour', '생산량', '기온', '풍속', '습도', '강수량', '전기요금(계절)', '공장인원', '인건비']
<class 'list'>
```

```
In [122... reg_dt = DecisionTreeRegressor(
    max_depth=10, # 트리의 최대 깊이: 10
    min_samples_leaf=4, # 리프 노드(Leaf Node)의 최소 샘플 수: 4
    random_state=123 # 랜덤 시드:123
)

reg_dt.fit(df_prob3_train[X_dt], df_prob3_train['target'])
```

```
Out[122]: DecisionTreeRegressor(criterion='mse', max_depth=10, max_features=None,
    max_leaf_nodes=None, min_impurity_decrease=0.0,
    min_impurity_split=None, min_samples_leaf=4,
    min_samples_split=2, min_weight_fraction_leaf=0.0,
    presort=False, random_state=123, splitter='best')
```

```
In [123... reg_dt.feature_importances_
```

```
Out[123]: array([0.00568971, 0.01820437, 0.02433534, 0.03155363, 0.31249107,
    0.13836694, 0.0025863 , 0.0111507 , 0.00552584, 0.18448738,
    0.00698797, 0.00246119, 0.12222486, 0.00835318, 0.04782553,
    0.01473692, 0.05613957, 0.00594418, 0.00057904])
```

```
In [124... '''
이 코드는 `DecisionTreeRegressor` 모델을 사용하여 피처의 중요도를 계산한 후,
가장 중요한 피처의 인덱스와 이름을 찾는 과정을 보여줍니다.

1. 피처 리스트 생성:
```

```

- `X_dt` 문자열은 모델에 사용할 피처(특징)들의 이름을 나열합니다.
- `split(',')`를 사용하여 문자열을 쉼표(,) 기준으로 분리하여 리스트로 만듭니다.
- `strip()`을 사용하여 각 피처 이름에서 앞뒤 공백을 제거합니다.
- 최종적으로 `X_dt`는 피처 이름들의 리스트가 됩니다.

2. 결정 트리 회귀 모델 생성 및 학습:
- `DecisionTreeRegressor` 모델을 생성합니다.
  - `max_depth=10`: 트리의 최대 깊이를 10으로 설정합니다.
  - `min_samples_leaf=4`: 리프 노드의 최소 샘플 수를 4로 설정합니다.
- `reg_dt.fit(df_prob3_train[X_dt], df_prob3_train['target'])`는 `df_prob3_train` 데이터프레임을 사용하여 모델을 학습시킵니다.
  - `df_prob3_train[X_dt]`는 피처 데이터입니다.
  - `df_prob3_train['target']`는 타겟 변수입니다.

3. 가장 중요한 피처의 인덱스와 이름 찾기:
- `reg_dt.feature_importances_`는 각 피처의 중요도를 나타내는 배열을 반환합니다.
- `np.argmax(reg_dt.feature_importances_)`는 피처 중요도 배열에서 가장 큰 값의 인덱스를 반환합니다.
- `+ 1`은 인덱스를 1부터 시작하는 형식으로 변환합니다. (파이썬에서는 기본적으로 0부터 시작)
- `A`는 가장 중요한 피처의 인덱스를 의미합니다.
- `X_dt[np.argmax(reg_dt.feature_importances_)]`는 가장 중요한 피처의 이름을 반환합니다.
...

A = np.argmax(reg_dt.feature_importances_) + 1
A, X_dt[np.argmax(reg_dt.feature_importances_)]

```

Out[124]: (5, 'DayName')

```

In [125... # 요건에는 없지만 참고삼아, train에 대한 성능을 측정해봅니다.
B = mean_absolute_error(df_prob3_test['target'], reg_dt.predict(df_prob3_test[X_dt]))
(
    B,
    mean_absolute_error(df_prob3_train['target'], reg_dt.predict(df_prob3_train[X_dt]))
)

```

Out[125]: (37.27394388956255, 46.75807226356938)

```

In [126... # 위 설명과 약간 중복되지만 추가 설명
...

위 코드는 `DecisionTreeRegressor`를 사용하여 회귀 모델을 학습시키고, 모델의 성능 및 피처 중요도를 평가하는 과정을 포함하고 있습니다.

### 1. 데이터 전처리:
- `X_dt` 문자열 정의: 모델에 사용할 독립 변수들의 이름을 나열합니다.

```

- 리스트로 변환: 쉼표로 구분된 문자열을 리스트로 변환하고, 각 항목의 앞뒤 공백을 제거합니다.

2. 결정 트리 회귀 모델 학습:

```
reg_dt = DecisionTreeRegressor(
    max_depth=10, # 트리의 최대 깊이: 10
    min_samples_leaf=4, # 리프 노드의 최소 샘플 수: 4
    random_state=123 # 랜덤 시드: 123
)
reg_dt.fit(df_prob3_train[X_dt], df_prob3_train['target'])
```

- 모델 초기화:

- ``max_depth=10``: 결정 트리의 최대 깊이를 10으로 제한합니다. 이는 트리가 과적합(overfitting)되는 것을 방지합니다.
- ``min_samples_leaf=4``: 각 리프 노드에 최소한 4개의 샘플이 있어야 합니다.
이는 모델이 지나치게 세부적으로 학습하지 않도록 합니다.

- 모델 학습: ``df_prob3_train`` 데이터프레임의 ``X_dt`` 열을 독립 변수로, ``target`` 열을 종속 변수로 사용하여 모델을 학습시킵니다.

3. 가장 중요한 피처 찾기:

- 피처 중요도 계산: ``reg_dt.feature_importances_``는 각 피처의 중요도를 나타내는 배열을 반환합니다.
- 가장 중요한 피처 찾기: ``np.argmax(reg_dt.feature_importances_)``는 가장 중요한 피처의 인덱스를 반환합니다.
여기에 ``+1``을 더해 ``A``에 저장합니다(인덱스가 0부터 시작하므로 1을 더함).
- 피처 이름과 중요도 출력: 가장 중요한 피처의 인덱스와 그 이름을 출력합니다.

4. 모델 성능 평가:

- 테스트 데이터에 대한 성능:
 - ``mean_absolute_error(df_prob3_test['target'], reg_dt.predict(df_prob3_test[X_dt]))``는 테스트 데이터에서 예측값과 실제값 간의 평균 절대 오차(MAE)를 계산하여 ``B``에 저장합니다.
- 학습 데이터에 대한 성능:
 - ``mean_absolute_error(df_prob3_train['target'], reg_dt.predict(df_prob3_train[X_dt]))``는 학습 데이터에서의 평균 절대 오차를 계산합니다.
- 출력: 두 개의 MAE 값을 튜플로 반환하여 출력합니다. 첫 번째 값은 테스트 데이터에서의 MAE(``B``), 두 번째 값은 학습 데이터에서의 MAE(``A``).

전체적인 동작 요약:

1. ``X_dt`` 리스트를 생성하여 모델에 사용할 피처들을 정의합니다.
2. ``DecisionTreeRegressor`` 모델을 설정하고 학습시킵니다.
3. 모델에서 가장 중요한 피처를 식별하고 출력합니다.
4. 모델의 성능을 평가하기 위해 테스트 데이터와 학습 데이터에서의 평균 절대 오차(MAE)를 계산하여 출력합니다.

이 과정을 통해 모델이 주어진 데이터에서 어떤 피처가 가장 중요한지를 알 수 있고,
모델의 예측 성능을 학습 데이터와 테스트 데이터에서 평가할 수 있습니다.

```
'''
```

```
print('')
```

In [164... `Image('1-3.png')`

Out[164]: **단계 3-4**

결정트리 대신에 경사 부스팅(Gradient Boosting) 회귀모델을 사용한다.

경사 부스팅(Gradient Boosting) 회귀모델구성 요건:

하이퍼 파라미터	값
손실함수	huber
트리의 수	75
트리의 최대 깊이	7
리프 노드(Leaf Node)의 최소 샘플 수	4
랜덤 시드	123

prob3_train으로 학습하고 prob3_test에 대한 성능을 C라고 한다.

함수가이드

sklearn.ensemble 제공 기능 활용

문제 지시 사항 외 Default 값 사용

In [127...

```
'''
코드는 `GradientBoostingRegressor`를 사용하여 회귀 모델을 학습시키고, 모델의 성능을 평가하는 과정을 포함하고 있습니다.

### 1. 데이터 전처리:
- `X_gb` 문자열 정의: 모델에 사용할 독립 변수들의 이름을 나열합니다.
- 리스트로 변환: 쉼표로 구분된 문자열을 리스트로 변환하고, 각 항목의 앞뒤 공백을 제거합니다.

### 2. 그레디언트 부스팅 회귀 모델 학습:
- 모델 초기화:
  - `loss='huber'`: Huber 손실 함수를 사용합니다. Huber 손실 함수는 평균 제곱 오차(MSE)와 평균 절대 오차(MAE)의
```

```

특성을 결합하여, 이상치(outlier)에 강건한 모델을 만듭니다.
- `n_estimators=75`: 부스팅 과정에서 사용할 트리의 수를 75로 설정합니다.
  더 많은 트리는 모델의 성능을 향상시킬 수 있지만 과적합의 위험도 있습니다.
- `max_depth=7`: 각 결정 트리의 최대 깊이를 7로 제한합니다. 이는 모델이 과적합되는 것을 방지합니다.
- `min_samples_leaf=4`: 각 리프 노드에 최소한 4개의 샘플이 있어야 합니다. 이는 모델이 지나치게 세부적으로 학습하지 않도록 합니다.
- `random_state=123`: 랜덤 시드를 설정하여 재현성을 보장합니다.
- 모델 학습: `df_prob3_train` 데이터프레임의 `X_gb` 열을 독립 변수로, `target` 열을 종속 변수로 사용하여 모델을 학습시킵니다.

### 3. 모델 성능 평가:
- 테스트 데이터에 대한 성능:
  - `mean_absolute_error(df_prob3_test['target'], reg_gb.predict(df_prob3_test[X_gb]))`는
    테스트 데이터에서 예측값과 실제값 간의 평균 절대 오차(MAE)를 계산하여 `C`에 저장합니다.
- 학습 데이터에 대한 성능:
  - `mean_absolute_error(df_prob3_train['target'], reg_gb.predict(df_prob3_train[X_gb]))`는 학습 데이터에서의 평균절대오차를 계산함
- 출력: 두 개의 MAE 값을 튜플로 반환하여 출력합니다.
  첫 번째 값은 테스트 데이터에서의 MAE(`C`), 두 번째 값은 학습 데이터에서의 MAE입니다.

### 전체적인 동작 요약:
1. `X_gb` 리스트를 생성하여 모델에 사용할 피쳐들을 정의합니다.
2. `GradientBoostingRegressor` 모델을 설정하고 학습시킵니다.
3. 모델의 성능을 평가하기 위해 테스트 데이터와 학습 데이터에서의 평균 절대 오차(MAE)를 계산하여 출력합니다.

이 과정을 통해 모델이 주어진 데이터에서 얼마나 잘 예측하는지를 평가할 수 있으며,
테스트 데이터와 학습 데이터에서의 성능을 비교하여 모델의 과적합 여부를 확인할 수 있습니다.
...

from sklearn.metrics import mean_absolute_error # 성능 측정 지표: MAE
from sklearn.ensemble import GradientBoostingRegressor

X_gb = '15분, 30분, 45분, 60분, DayName, Hour, AM, Weekend_yn, Holiday_yn, Avg, \
        TotalHour, 생산량, 기온, 풍속, 습도, 강수량, 전기요금(계절), 공장인원, 인건비'
X_gb = [i.strip() for i in X_gb.split(',')]

reg_gb = GradientBoostingRegressor(
    loss = 'huber', # 손실함수: huber
    n_estimators = 75, # 트리의 수: 75
    max_depth = 7, # 트리의 최대 깊이: 7
    min_samples_leaf = 4, # 리프 노드(Leaf Node)의 최소 샘플 수: 4
    random_state = 123 # 랜덤 시드: 123
)

reg_gb.fit(df_prob3_train[X_gb], df_prob3_train['target'])

```

```
C = mean_absolute_error(df_prob3_test['target'], reg_gb.predict(df_prob3_test[X_gb]))
(
    C,
    mean_absolute_error(df_prob3_train['target'], reg_gb.predict(df_prob3_train[X_gb]))
)
```

Out[127]: (36.04762917818188, 35.842003600642194)

In [128... A, B, C, A + B + C, round(A + B + C, 3)

Out[128]: (5, 37.27394388956255, 36.04762917818188, 78.32157306774442, 78.322)

A + B + C를 소수점 넷째 자리에서 반올림하여 셋째 자리까지 적으시오

78.322

In []:

문제 4

DateHour를 기준으로 동일한 시각의 n일 이전의 TotalHour를 파생 변수 lag_n이라고 한다.

예를 들어 DateHour가 2021-01-07 00:00:00이라면,

lag_1은 2021-01-06 00:00:00의 TotalHour값인 96이 된다.

TotalHour		TotalHour		TotalHour		lag_1
DateHour		DateHour		DateHour		
2021-01-07 00:00:00	91	2021-01-06 00:00:00	96	2021-01-07 00:00:00	91	96.0
2021-01-07 01:00:00	92	2021-01-06 01:00:00	85	2021-01-07 01:00:00	92	85.0
2021-01-07 02:00:00	86	2021-01-06 02:00:00	85	2021-01-07 02:00:00	86	85.0
2021-01-07 03:00:00	89	2021-01-06 03:00:00	90	2021-01-07 03:00:00	89	90.0
2021-01-07 04:00:00	97	2021-01-06 04:00:00	93	2021-01-07 04:00:00	97	93.0

단계 4-1

prob3에 lag_1 ~ lag_6까지의 6개의 파생 변수를 추가하여 prob4를 만든다.

prob4에 lag_1 ~ lag_6까지 하나라도 결측치가 있다면 삭제한다.

In [129...

```
'''
코드는 `df_prob3` 데이터프레임에 대해 시계열 특성을 반영한 새로운 데이터프레임 `df_prob4`를 생성합니다.
구체적으로, `df_prob3`의 `TotalHour` 컬럼을 24시간 단위로 여러 번 시프트(이동)하여 새로운 컬럼으로 추가합니다.
이를 통해 각 시점에서의 이전 24시간, 48시간, ..., 144시간 전의 `TotalHour` 값을 포함하는 데이터프레임을 만듭니다.
그런 다음, 결측값이 있는 행을 제거합니다.

1. 리스트 컴프리헨션을 사용하여 시프트된 시리지를 생성:
    - `range(1, 7)`은 1부터 6까지의 정수 시퀀스를 생성합니다.
    - `df_prob3['TotalHour'].shift(24 * i)`는 `TotalHour` 컬럼을 24시간, 48시간, ..., 144시간(6일)만큼 시프트합니다.
    - `.rename('lag_{}'.format(i))`는 각각의 시프트된 시리지의 이름을 `lag_1`, `lag_2`, ..., `lag_6`으로 변경합니다.

2. pd.concat를 사용하여 원래 데이터프레임과 시프트된 시리지를 합치기:
    - `[df_prob3]`는 원래 데이터프레임을 리스트로 감쌉니다.
    - `[df_prob3] + [df_prob3['TotalHour'].shift(24 * i).rename('lag_{}'.format(i)) for i in range(1, 7)]`는
      원래 데이터프레임과 시프트된 시리지를 하나의 리스트로 결합합니다.
    - `pd.concat(..., axis=1)`는 이 리스트의 요소들을 열(axis=1) 기준으로 병합하여 새로운 데이터프레임을 생성합니다.

3. dropna를 사용하여 결측값이 있는 행 제거:
    - .dropna()
    - 시프트된 시리지는 앞쪽에 결측값(NaN)을 가질 수밖에 없으므로, 시프트된 시리지를 추가한 후에는 결측값이 있는 행이 생깁니다.
    - `.dropna()`는 이러한 결측값이 있는 행을 제거합니다.

### 결과
결과 데이터프레임 `df_prob4`는 원래 `df_prob3`에 `lag_1`, `lag_2`, ..., `lag_6` 열이 추가된 형태가 되며,
각 열에는 해당 시간만큼 시프트된 `TotalHour` 값이 들어갑니다.
결측값이 포함된 행들은 `dropna()`에 의해 제거되어, 모든 열에 값이 있는 행만 남게 됩니다.

이렇게 하면 시계열 데이터의 시점별 과거 값을 분석하거나 예측 모델에 사용할 수 있는 특성을 쉽게 추가할 수 있습니다.
'''

# 방법 1: shift를 이용하여 구해봅니다.
# 먼저 lag 변수들을 list comprehension을 이용해 만들고 이를 df_prob3와 함께 결합하여 df_prob4를 만듭니다.
# lag을 제외한 나머지 변수에 결측이 없습니다. dropna를 통해 일괄적으로 제거해도됩니다.
```

```
df_prob4 = pd.concat([df_prob3 + [df_prob3['TotalHour'].shift(24 * i).rename('lag_{}'.format(i)) for i in range(1, 7)], axis=1).dropna()

display(df_prob4.head())
df_prob4.shape
```

	15 분	30 분	45 분	60 분	DayName	Hour	AM	Weekend_yn	Holiday_yn	Avg	...	전기 요금 (계 절)	공장인원	인 건 비	target	lag_1	lag_2	lag_3	lag_4	lag_5	lag_6	
DateHour																						
2021-01-07 00:00:00	22	22	22	25		3	0	0	0	0	22.75	...	109.8	0.000000	1.5	252.0	96.0	96.0	271.0	88.0	253.0	24
2021-01-07 01:00:00	26	22	22	22		3	1	0	0	0	23.00	...	109.8	0.369565	1.5	396.0	85.0	85.0	432.0	99.0	418.0	41
2021-01-07 02:00:00	22	21	21	22		3	2	0	0	0	21.50	...	109.8	0.000000	1.5	411.0	85.0	85.0	439.0	88.0	415.0	41
2021-01-07 03:00:00	22	22	22	23		3	3	0	0	0	22.25	...	109.8	0.629213	1.5	398.0	90.0	90.0	426.0	88.0	421.0	42
2021-01-07 04:00:00	22	26	26	23		3	4	0	0	0	24.25	...	109.8	2.865979	1.5	420.0	93.0	93.0	435.0	95.0	427.0	42

5 rows × 26 columns

Out[129]: (6000, 26)

In [130]

'''
이 코드는 `df\_prob3` 데이터프레임에 대해 시계열 특성을 반영한 새로운 데이터프레임 `df\_prob4`를 생성합니다.
각 `TotalHour` 값을 1일부터 6일까지의 기간만큼 시프트하여 새로운 열로 추가합니다.

`pd.concat`을 사용하여 이 시프트된 열들을 원래 데이터프레임과 병합합니다.
중요한 점은 `join='inner'` 옵션을 사용하여 공통 인덱스에 대해서만 병합하는 것입니다.

코드의 각 부분을 상세히 설명하면 다음과 같습니다:

1. 리스트 컴프리헨션을 사용하여 시프트된 시리즈를 생성:

- `range(1, 7)`은 1부터 6까지의 정수 시퀀스를 생성합니다.
- `df_prob3['TotalHour'].values`는 `TotalHour` 열의 값을 배열로 반환합니다.
- `df_prob3['TotalHour'].index + pd.Timedelta(days=i)`는 인덱스를 `i`일 만큼 시프트합니다.
- `pd.Series(..., name='lag_{}'.format(i))`는 시프트된 값을 가지는 새로운 시리즈를 생성하고, 이름을 `lag_i`로 지정합니다.

2. `pd.concat`을 사용하여 원래 데이터프레임과 시프트된 시리즈들을 합치기:

- `[df_prob3]`는 원래 데이터프레임을 리스트로 감쌉니다.
- `[df_prob3] + ...`는 원래 데이터프레임과 시프트된 시리즈들을 하나의 리스트로 결합합니다.
- `pd.concat(..., axis=1, join='inner')`는 이 리스트의 요소들을 열(`axis=1`) 기준으로 병합하여 새로운 데이터프레임을 생성합니다. `join='inner'`는 공통 인덱스만 남겨 병합합니다.

3. `dropna`는 사용되지 않았지만 `join='inner'`가 같은 역할을 함:

- `join='inner'` 옵션은 공통 인덱스를 기준으로 병합하여 결측값이 포함된 행을 자동으로 제거합니다.

결과

결과 데이터프레임 `df_prob4`는 원래 `df_prob3`에 `lag_1`, `lag_2`, ..., `lag_6` 열이 추가된 형태가 되며, 각 열에는 해당 기간만큼 시프트된 `TotalHour` 값이 들어갑니다.
공통 인덱스만 남기고 병합하므로, 인덱스가 일치하지 않는 행들은 자동으로 제거됩니다.

이렇게 하면 시계열 데이터의 시점별 과거 값을 분석하거나 예측 모델에 사용할 수 있는 특성을 쉽게 추가할 수 있습니다.
...

방법 2: `index`를 변경하여 변수를 만듭니다.

먼저 `lag` 변수들을 `list comprehension`을 이용해 만들고 이를 `df_prob3`와 함께 결합하여 `df_prob4`를 만듭니다.

`join` 설정을 `inner`로 하면 결합하는 모든 요소들에 공통으로 등장하는 인덱스에 해당하는 행들만을 남깁니다.

```
df_prob4 = pd.concat(
    [df_prob3] + [
        pd.Series(df_prob3['TotalHour'].values, index=df_prob3['TotalHour'].index
                  + pd.Timedelta(days=i), name='lag_{}'.format(i)) for i in range(1, 7)
    ],
    axis=1, join='inner'
)
```

```
display(df_prob4.head())
df_prob4.shape
```

	15 분	30 분	45 분	60 분	DayName	Hour	AM	Weekend_yn	Holiday_yn	Avg	...	전기 요금 (계 절)	공장인원	인 건 비	target	lag_1	lag_2	lag_3	lag_4	lag_5	lag_6
DateHour																					
2021-01-07 00:00:00	22	22	22	25		3	0	0		0	22.75	...	109.8	0.000000	1.5	252.0	96	96	271	88	253
2021-01-07 01:00:00	26	22	22	22		3	1	0		0	23.00	...	109.8	0.369565	1.5	396.0	85	85	432	99	418
2021-01-07 02:00:00	22	21	21	22		3	2	0		0	21.50	...	109.8	0.000000	1.5	411.0	85	85	439	88	415
2021-01-07 03:00:00	22	22	22	23		3	3	0		0	22.25	...	109.8	0.629213	1.5	398.0	90	90	426	88	421
2021-01-07 04:00:00	22	26	26	23		3	4	0		0	24.25	...	109.8	2.865979	1.5	420.0	93	93	435	95	427

5 rows × 26 columns

Out[130]: (6000, 26)

In [131...

```
# 위 두가지 방식의 비교

...

두 코드 모두 `df_prob3` 데이터프레임을 기준으로 특정 시점만큼 시프트된 `TotalHour` 값을 새로운 열로 추가하여
`df_prob4`를 생성합니다. 그러나 두 접근 방식에는 중요한 차이점이 있습니다. 각각의 동작을 비교하여 자세히 설명하겠습니다.

### 첫 번째 코드
#### 동작 설명:
```

1. 리스트 컴프리헨션:

- ``shift(24 * i)``는 ``TotalHour`` 열을 ``24 * i``만큼 시프트합니다. 즉, 24시간씩 (1일씩) 시프트된 값을 생성합니다.
- ``rename('lag_{}'.format(i))``는 시프트된 시리즈의 이름을 ``lag_i``로 변경합니다.

2. `pd.concat`:

- ``[df_prob3] + ...``는 원래 데이터프레임과 시프트된 시리즈들을 하나의 리스트로 결합합니다.
- ``pd.concat(..., axis=1)``는 이 리스트의 요소들을 열(`axis=1`) 기준으로 병합하여 새로운 데이터프레임을 생성합니다.

3. `dropna`:

- 병합된 데이터프레임에서 결측값이 포함된 행을 제거합니다.

결과:

- 각 시프트된 시리즈는 원래 데이터프레임의 인덱스와 맞지 않는 부분이 `NaN`으로 채워집니다.
- ``dropna()``를 사용하여 이 `NaN` 행들을 제거합니다.
- 결과적으로, 시프트된 시리즈와 공통 인덱스를 가진 행들만 남게 됩니다.

두 번째 코드

동작 설명:

1. 리스트 컴프리헨션:

- ``df_prob3['TotalHour'].values``는 ``TotalHour`` 열의 값을 배열로 반환합니다.
- ``df_prob3['TotalHour'].index + pd.Timedelta(days=i)``는 인덱스를 ``i``일 만큼 시프트합니다.
- ``pd.Series(..., name='lag_{}'.format(i))``는 시프트된 값을 가지는 새로운 시리즈를 생성하고, 이름을 ``lag_i``로 지정합니다.

2. `pd.concat`:

- ``[df_prob3] + ...``는 원래 데이터프레임과 시프트된 시리즈들을 하나의 리스트로 결합합니다.
- ``pd.concat(..., axis=1, join='inner')``는 공통 인덱스를 기준으로 열(`axis=1`) 기준으로 병합하여 새로운 데이터프레임을 생성합니다.

결과:

- 시프트된 시리즈들은 원래 데이터프레임의 인덱스와 ``i``일 만큼 시프트된 인덱스를 가지게 됩니다.
- ``join='inner'`` 옵션을 사용하여 병합된 데이터프레임에서 공통 인덱스를 가진 행들만 남깁니다.
- 이로 인해 `NaN` 값이 생성되지 않고, 인덱스가 일치하는 행들만 포함됩니다.

비교 및 차이점:

1. 시프트 방식:

- 첫 번째 코드는 ``shift(24 * i)``를 사용하여 24시간 단위로 시프트합니다.
- 두 번째 코드는 ``pd.Timedelta(days=i)``를 사용하여 일 단위로 시프트합니다.

2. 결측값 처리:

- 첫 번째 코드는 ``dropna()``를 사용하여 병합 후 결측값이 포함된 행을 제거합니다.
- 두 번째 코드는 ``join='inner'``를 사용하여 병합 시 공통 인덱스만 남기므로 결측값이 포함되지 않습니다.

3. 결과 데이터프레임:

- 첫 번째 코드의 결과는 결측값이 제거된 데이터프레임입니다.
- 두 번째 코드의 결과는 공통 인덱스만 남긴 데이터프레임입니다.

...

```
print('')
```

Tip!

target의 시점을 기준으로 이전 7일의 TotalHour, Avg의 평균을 구하고,

이를 TotalHour_mean7, Avg_mean7으로 만들어 봅니다.

결측이 하나라도 있으면 뺍니다.

In [132...

```
'''
이 코드는 `df_prob3` 데이터프레임의 `TotalHour`와 `Avg` 열에 대해 각각의 시계열을 7일 간의 이동 평균으로 변환합니다.

1. 리스트 컴프리헨션:
  - `for i in range(0, 7)`는 0에서 6까지의 범위 내에서 반복합니다.
  - `df_prob3[j]`는 `df_prob3` 데이터프레임의 열(`TotalHour` 또는 `Avg`)을 선택합니다.
  - `shift(24 * i)`는 선택한 열을 `24 * i`만큼 시프트합니다. 즉, 시간 단위로 시프트하므로 `24 * i`는 i일을 의미합니다.
2. 첫 번째 pd.concat:
  - 시프트된 7개의 시리즈를 열 방향으로 병합합니다.
  - 결과는 7개의 열을 가진 데이터프레임이 됩니다.
3. dropna:
  - 병합된 데이터프레임에서 NaN 값을 포함한 행들을 제거합니다.
  - 시프트 연산으로 인해 일부 값이 NaN이 되므로, 이 행들을 제거하여 완전한 데이터만 남깁니다.
4. mean:
  - 각 행에 대해 7개의 열의 평균을 계산합니다.
  - `axis=1`은 행 방향으로 평균을 계산한다는 의미입니다.
  - 결과는 각 행에 대한 7일 이동 평균을 나타내는 시리즈입니다.
5. rename:
  - 계산된 평균 시리즈의 이름을 `TotalHour_mean7` 또는 `Avg_mean7`로 변경합니다.
6. 외부 리스트 컴프리헨션:
  - 위의 과정을 `TotalHour`와 `Avg` 열 각각에 대해 반복합니다.
  - 두 개의 시리즈 리스트를 생성합니다.
7. 두 번째 pd.concat:
  - 두 개의 시리즈(`TotalHour_mean7` 및 `Avg_mean7`)를 열 방향으로 병합합니다.
  - 결과는 두 개의 열(`TotalHour_mean7` 및 `Avg_mean7`)을 가진 데이터프레임이 됩니다.

### 종합적으로 설명:
1. `df_prob3` 데이터프레임의 `TotalHour` 및 `Avg` 열 각각에 대해 0일, 1일, ..., 6일 시프트된 버전을 생성합니다.
2. 시프트된 버전들을 열 방향으로 병합한 후, NaN 값을 제거합니다.
'''
```

3. 각 행에 대해 7일 이동 평균을 계산하고, 결과 시리즈를 적절한 이름으로 변경합니다.
 4. `TotalHour`와 `Avg`에 대한 각각의 결과 시리즈를 다시 열 방향으로 병합하여 최종 데이터프레임을 생성합니다.

이 최종 데이터프레임은 `TotalHour\_mean7` 및 `Avg\_mean7` 열을 포함하며, 이는 각각의 7일 이동 평균 값을 나타냅니다.

*# target을 기준으로 이전 7일의 동일 시간에 대한 TotalHour의 평균과 Avg의 평균을 구해봅니다.
 # target은 TotalHour를 기준으로 하루 앞선 시점의 동일 시간의 TotalHour이므로 0일도 포함 시킵니다.*

```
pd.concat(
    [
        pd.concat(
            [df_prob3[j].shift(24 * i) for i in range(0, 7)], axis=1
        ).dropna().mean(axis=1).rename('{_mean7'.format(j)) for j in ['TotalHour', 'Avg']
    ], axis=1
)
```

Out[132]:

	TotalHour_mean7	Avg_mean7
DateHour		
2021-01-07 00:00:00	162.857143	40.714286
2021-01-07 01:00:00	232.714286	58.178571
2021-01-07 02:00:00	230.428571	57.607143
2021-01-07 03:00:00	232.142857	58.035714
2021-01-07 04:00:00	238.142857	59.535714
...	...	...
2021-09-13 19:00:00	447.285714	111.821429
2021-09-13 20:00:00	370.571429	92.642857
2021-09-13 21:00:00	381.428571	95.357143
2021-09-13 22:00:00	336.285714	84.071429
2021-09-13 23:00:00	320.857143	80.214286

6000 rows × 2 columns

In [133...

```
'''
이 코드는 `df_prob3` 데이터프레임의 `TotalHour`와 `Avg` 열에 대해
각각의 시계열 데이터를 7일 간의 이동 평균으로 변환하는 작업을 수행합니다.

### 1. 리스트 컴프리헨션
이 부분은 `TotalHour`와 `Avg` 두 개의 열에 대해 동일한 처리를 반복합니다.
즉, 리스트 컴프리헨션은 각 열에 대해 다음 작업을 수행합니다:

#### a. `shift`와 `rename`
- `for i in range(0, 7)`: 0에서 6까지 반복합니다.
- `df_prob3[j]`: `df_prob3` 데이터프레임의 열(`TotalHour` 또는 `Avg`)을 선택합니다.
- `shift(i * 24)`: 선택된 열을 `i * 24`만큼 시프트합니다.
    이는 하루 단위(24시간)를 기준으로 시프트하므로, i일만큼 시프트하게 됩니다.
- `rename('lag_{}'.format(i))`: 시프트된 열의 이름을 `lag_i`로 변경합니다.

이 리스트 컴프리헨션은 시프트된 7개의 시리지를 생성합니다.

#### b. `pd.concat`
- 시프트된 7개의 시리지를 열 방향으로 병합합니다.
- 결과는 7개의 열을 가진 데이터프레임이 됩니다.

#### c. `dropna`
- 병합된 데이터프레임에서 NaN 값을 포함한 행들을 제거합니다.
    시프트 연산으로 인해 일부 값이 NaN이 되므로, 이 행들을 제거하여 완전한 데이터만 남깁니다.

#### d. `mean`
- 각 행에 대해 7개의 열의 평균을 계산합니다.
- `axis=1`은 행 방향으로 평균을 계산한다는 의미입니다.
- 결과는 각 행에 대한 7일 이동 평균을 나타내는 시리집니다.

#### e. `rename`
- 계산된 평균 시리지의 이름을 `TotalHour_mean7` 또는 `Avg_mean7`로 변경합니다.

### 2. 외부 `pd.concat`
- 위의 리스트 컴프리헨션에서 생성된 두 개의 시리지(`TotalHour_mean7` 및 `Avg_mean7`)를 열 방향으로 병합합니다.
- 결과는 두 개의 열(`TotalHour_mean7` 및 `Avg_mean7`)을 가진 데이터프레임이 됩니다.

### 종합적으로 설명
1. `df_prob3` 데이터프레임의 `TotalHour`와 `Avg` 열 각각에 대해 0일, 1일, ..., 6일 시프트된 버전을 생성합니다.
2. 시프트된 버전들을 열 방향으로 병합한 후, NaN 값을 제거합니다.
3. 각 행에 대해 7개의 열의 평균을 계산하고, 결과 시리지를 적절한 이름으로 변경합니다.

```

4. `TotalHour`와 `Avg`에 대한 각각의 결과 시리즈를 다시 열 방향으로 병합하여 최종 데이터프레임을 생성합니다.

이 최종 데이터프레임은 `TotalHour\_mean7` 및 `Avg\_mean7` 열을 포함하며, 이는 각각의 7일 이동 평균 값을 나타냅니다.

...
index 기반의 방법으로 만들어봅니다.

```
pd.concat(
    [
        pd.concat(
            [df_prob3[j].shift(i * 24).rename('lag_{}'.format(i)) for i in range(0, 7)], axis=1
        ).dropna().mean(axis=1).rename('{}_mean7'.format(j)) for j in ['TotalHour', 'Avg']
    ], axis=1
)
```

Out[133]:

	TotalHour_mean7	Avg_mean7
DateHour		
2021-01-07 00:00:00	162.857143	40.714286
2021-01-07 01:00:00	232.714286	58.178571
2021-01-07 02:00:00	230.428571	57.607143
2021-01-07 03:00:00	232.142857	58.035714
2021-01-07 04:00:00	238.142857	59.535714
...	...	...
2021-09-13 19:00:00	447.285714	111.821429
2021-09-13 20:00:00	370.571429	92.642857
2021-09-13 21:00:00	381.428571	95.357143
2021-09-13 22:00:00	336.285714	84.071429
2021-09-13 23:00:00	320.857143	80.214286

6000 rows × 2 columns

In [134... # 위 두가지 방식의 비교

```
'''
```

두 코드 모두 pandas 라이브러리를 사용하여 데이터 프레임 `df\_prob3`의 특정 열들에 대해 7일 간격으로 이동 평균을 계산하는 기능을 수행합니다. 두 코드는 매우 유사하지만, 약간의 차이가 있습니다.

코드 1 설명:

1. List comprehension:

- `for j in ['TotalHour', 'Avg']`: `df\_prob3` 데이터 프레임의 두 열 'TotalHour'와 'Avg'에 대해 반복합니다.

2. Nested list comprehension:

- `[df\_prob3[j].shift(24 \* i) for i in range(0, 7)]:`

각 열 `j`에 대해 0부터 6까지 7일 간격(24시간 * i)으로 데이터를 이동시킵니다.

예를 들어, `i=0`이면 원본 데이터를, `i=1`이면 하루 뒤로 이동한 데이터를 의미합니다.

3. Inner `pd.concat`:

- `pd.concat(..., axis=1)`: 이 이동된 7개의 열들을 옆으로 붙여 새로운 데이터 프레임을 만듭니다.

4. Drop NA and Mean Calculation:

- `.dropna().mean(axis=1)`: NaN 값을 가진 행을 제거하고, 각 행에 대해 7개의 이동된 열들의 평균을 계산합니다.

5. Rename:

- `.rename('{}\_mean7'.format(j))`: 계산된 평균 시리즈를 'TotalHour_mean7' 또는 'Avg_mean7'로 이름을 붙입니다.

6. Outer `pd.concat`:

- `pd.concat(..., axis=1)`: 'TotalHour_mean7'과 'Avg_mean7' 두 시리즈를 옆으로 붙여 최종 데이터 프레임을 만듭니다.

코드 2 설명:

1. List comprehension:

- `for j in ['TotalHour', 'Avg']`: 첫 번째 코드와 동일하게 'TotalHour'와 'Avg' 열에 대해 반복합니다.

2. Nested list comprehension:

- `[df\_prob3[j].shift(i \* 24).rename('lag\_{}'.format(i)) for i in range(0, 7)]:`

각 열 `j`에 대해 0부터 6까지 7일 간격으로 데이터를 이동시키고, 각 이동된 시리즈에 'lag_0', 'lag_1', ..., 'lag_6'라는 이름을

3. Inner `pd.concat`:

- `pd.concat(..., axis=1)`: 이 이동된 7개의 열들을 옆으로 붙여 새로운 데이터 프레임을 만듭니다.

4. Drop NA and Mean Calculation:

- `.dropna().mean(axis=1)`: NaN 값을 가진 행을 제거하고, 각 행에 대해 7개의 이동된 열들의 평균을 계산합니다.

5. Rename:

- `.rename('{}\_mean7'.format(j))`: 계산된 평균 시리즈를 'TotalHour_mean7' 또는 'Avg_mean7'로 이름을 붙입니다.

6. Outer `pd.concat`:

- `pd.concat(..., axis=1)`: 'TotalHour_mean7'과 'Avg_mean7' 두 시리즈를 옆으로 붙여 최종 데이터 프레임을 만듭니다.

차이점 비교:

1. 열 이름 붙이기:

- 첫 번째 코드에서는 이동된 열들에 별도의 이름을 붙이지 않습니다.

- 두 번째 코드에서는 각 이동된 시리즈에 'lag_0', 'lag_1', ..., 'lag_6'라는 이름을 붙입니다.

이는 디버깅이나 데이터 탐색 시 유용할 수 있습니다.

2. 기능적 차이:

- 두 코드 모두 같은 결과(각 열에 대해 7일 이동 평균)를 산출합니다.
- 하지만 두 번째 코드는 중간 결과물인 이동된 열들을 이름으로 명시함으로써 코드의 가독성을 약간 향상시킵니다.

결론:

두 코드 모두 비슷한 작업을 수행하며, 기능적으로 동일한 결과를 제공합니다.
그러나 두 번째 코드가 각 이동된 시리즈에 이름을 부여하여 가독성과 유지보수성을 조금 더 높이는 방식입니다.
이를 통해 코드 실행 시 중간 결과를 쉽게 확인할 수 있습니다.

```
print('')
```

단계 4-2

학습용과 테스트용 데이터셋을 2021년 8월 14일 전과 이후로 분리

. prob4_train: DateHour 변수 기준으로 2021년 8월 14일 전(8월 14일 미포함) 데이터 (행의 수: 5256개)

. prob4_test: DateHour 변수 기준으로 2021년 8월 14일 이후(8월 14일 포함) 데이터 (행의 수: 744 개)

In [135...

```
'''
코드는 `df_prob4` 데이터프레임을 특정 날짜를 기준으로 훈련 데이터와 테스트 데이터로 분할하는 작업을 수행합니다.

### 동작 설명
1. 기준 날짜 설정:
    pd.to_datetime('2021-08-14')
    - 문자열 `'2021-08-14'`을 `pandas`의 `to_datetime` 함수를 사용하여 `Timestamp` 객체로 변환합니다.
    - 이는 특정 날짜를 기준으로 데이터를 분할할 때 사용됩니다.

2. 훈련 데이터셋 분할:
    df_prob4_train = df_prob4.loc[df_prob4.index < pd.to_datetime('2021-08-14')].copy()
    - `df_prob4.index < pd.to_datetime('2021-08-14')`는 `df_prob4` 데이터프레임의 인덱스가 `2021-08-14` 이전인 행들을 선택합니다.
    - `df_prob4.loc[...]`를 사용하여 선택된 행들로 구성된 새로운 데이터프레임을 생성합니다.
    - `.copy()` 메소드를 호출하여 원본 데이터프레임으로부터 독립적인 복사본을 만듭니다.
    - 결과적으로, `df_prob4_train`은 `2021-08-14` 이전의 데이터를 포함하는 훈련 데이터셋이 됩니다.

3. 테스트 데이터셋 분할:
    df_prob4_test = df_prob4.loc[df_prob4.index >= pd.to_datetime('2021-08-14')].copy()
    - `df_prob4.index >= pd.to_datetime('2021-08-14')`는
```

```

`df_prob4` 데이터프레임의 인덱스가 `2021-08-14` 이후인 행들을 선택합니다.
- `df_prob4.loc[...]`를 사용하여 선택된 행들로 구성된 새로운 데이터프레임을 생성합니다.
- `.copy()` 메소드를 호출하여 원본 데이터프레임으로부터 독립적인 복사본을 만듭니다.
- 결과적으로, `df_prob4_test`는 `2021-08-14` 이후의 데이터를 포함하는 테스트 데이터셋이 됩니다.

### 요약
- 이 코드의 목적은 `df_prob4` 데이터프레임을 날짜 `2021-08-14`을 기준으로 훈련 데이터셋(`df_prob4_train`)과
  테스트 데이터셋(`df_prob4_test`)으로 나누는 것입니다.
- `df_prob4_train`은 `2021-08-14` 이전의 데이터를 포함하고, `df_prob4_test`는 `2021-08-14` 이후의 데이터를 포함합니다.
- `.copy()` 메소드를 사용하여 각 분할된 데이터셋이 원본 데이터프레임 `df_prob4`로부터 독립적으로 존재하도록 합니다.
...

df_prob4_train = df_prob4.loc[df_prob4.index < pd.to_datetime('2021-08-14')].copy()
df_prob4_test = df_prob4.loc[df_prob4.index >= pd.to_datetime('2021-08-14')].copy()
df_prob4_train.shape, df_prob4_test.shape

```

Out[135]: ((5256, 26), (744, 26))

단계 4-3

prob4_train에서 TotalHour와 lag_1~6 를 표준화하고,

prob4_train의 표준화 설정으로 prob4_test의 TotalHour와 lag_1~6를 표준화한다.

함수가이드

sklearn.preprocessing 제공 기능 활용

문제 지시 사항 외 Default 값 사용

In [136...

```

...
이 코드는 `StandardScaler`를 사용하여 훈련 데이터와 테스트 데이터의 특정 열을 표준화(정규화)하는 작업을 수행합니다.
표준화는 각 열의 값들이 평균이 0이고 표준편차가 1이 되도록 변환하는 과정입니다.

### 동작 설명

1. `StandardScaler` 객체 생성:

```

- ``StandardScaler``는 ``scikit-learn`` 라이브러리의 전처리 클래스입니다.
 - 각 열의 값들이 평균이 0이고 표준편차가 1이 되도록 변환합니다.
 - 이 객체를 생성함으로써 이후의 표준화 작업에 사용할 준비를 합니다.
2. 표준화할 열 목록 생성:
- ``X_std``는 표준화할 열 이름들의 리스트입니다.
 - ``'TotalHour'`` 열과 ``'lag_1'``부터 ``'lag_6'``까지의 열들이 포함됩니다.
 - ``['lag_{}'.format(i) for i in range(1, 7)]``는 ``lag_1``, ``lag_2``, ..., ``lag_6`` 열 이름들을 동적으로 생성합니다.
3. 훈련 데이터에 대해 ``StandardScaler``를 학습(`fit`):
- ``fit`` 메소드를 사용하여 ``df_prob4_train`` 데이터프레임의 ``X_std`` 열들에 대해 ``StandardScaler``를 학습시킵니다.
 - 여기서 학습이란 각 열의 평균과 표준편차를 계산하는 것을 의미합니다.
 - 이 평균과 표준편차는 이후의 변환(`transform`) 작업에 사용됩니다.
4. 훈련 데이터의 열들을 표준화:
- ``transform`` 메소드를 사용하여 ``df_prob4_train`` 데이터프레임의 ``X_std`` 열들을 표준화합니다.
 - 각 열의 값들이 평균이 0이고 표준편차가 1이 되도록 변환됩니다.
 - 변환된 값들을 원래의 데이터프레임에 다시 할당합니다.
5. 테스트 데이터의 열들을 표준화:
- 훈련 데이터에 대해 학습된 ``StandardScaler``를 사용하여 ``df_prob4_test`` 데이터프레임의 ``X_std`` 열들을 표준화합니다.
 - ``fit`` 메소드를 사용하지 않고 ``transform`` 메소드만 사용하는 이유는,
테스트 데이터는 훈련 데이터의 평균과 표준편차를 기준으로 변환해야 일관성을 유지할 수 있기 때문입니다.
 - 변환된 값들을 원래의 데이터프레임에 다시 할당합니다.

요약

- ``StandardScaler``를 사용하여 특정 열들을 표준화하는 과정입니다.
- 훈련 데이터(``df_prob4_train``)의 평균과 표준편차를 기준으로 ``StandardScaler``를 학습시킵니다.
- 학습된 ``StandardScaler``를 사용하여 훈련 데이터와 테스트 데이터(``df_prob4_test``)를 변환합니다.
- 이 작업을 통해 훈련 데이터와 테스트 데이터의 특정 열들이 평균이 0이고 표준편차가 1이 되도록 표준화됩니다.

```
from sklearn.preprocessing import StandardScaler
```

```
std_scaler = StandardScaler()
```

```
X_std = ['TotalHour'] + ['lag_{}'.format(i) for i in range(1, 7)]
print('X_std = ', X_std)
```

```
std_scaler.fit(df_prob4_train[X_std])
```

```
df_prob4_train[X_std] = std_scaler.transform(df_prob4_train[X_std])

df_prob4_test[X_std] = std_scaler.transform(df_prob4_test[X_std])

X_std = ['TotalHour', 'lag_1', 'lag_2', 'lag_3', 'lag_4', 'lag_5', 'lag_6']
```

단계 4-4

SVR(Support Vector Regressor) 모델을 아래 요건에 3가지 커널(Kernel)로 prob4_train을 학습하고

prob4_test로 성능을 측정한다.

그 중 가장 좋은 성능을 보인 커널의 번호를 A(No.), 그 때의 성능을 B라고 한다.

성능 측정 지표: $MAE(Y, \hat{Y}) = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$

입력 변수: TotalHour, lag_1 ~ 6

대상 변수: target

커널함수:

$$1: k(x_i, x_j) = \langle x_i, x_j \rangle$$

$$2: k(x_i, x_j) = (\gamma \langle x_i, x_j \rangle + 1)^2, \gamma = 0.1$$

$$3: k(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2), \gamma = 0.1$$

$$\langle x, x' \rangle : x, x' \text{의 내적}, \|x_i - x_j\|^2 : \text{L2-norm}$$

함수가이드

sklearn.svm.SVR, C=10

문제 지시 사항 외 Default 값 사용

In [137...

```
'''
`SVR()` 함수는 Scikit-learn 라이브러리에서 제공하는 서포트 벡터 회귀(Support Vector Regression)
모델을 생성하는 클래스입니다. 이 클래스는 회귀 문제를 해결하기 위해 사용됩니다.

### 1. `SVR` 클래스의 인자
- `kernel`: {'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'} 또는 callable, default='rbf'
  - 사용할 커널의 종류를 지정합니다. 기본값은 'rbf' (Radial Basis Function)입니다.
- `degree`: int, default=3
  - 커널이 다항식(poly)일 경우 다항식의 차수를 지정합니다.
- `gamma`: {'scale', 'auto'} 또는 float, default='scale'
  - 'scale' (기본값)은  $1 / (n\_features * X.var())$ 로 설정됩니다.
  - 'auto'는  $1 / n\_features$ 로 설정됩니다.
  - 커널 계수로 사용되는 gamma 값을 지정할 수도 있습니다.
- `coef0`: float, default=0.0
  - 커널이 'poly' 또는 'sigmoid'일 때 커널 함수의 상수를 지정합니다.
- `tol`: float, default=1e-3
  - 종료 기준을 위한 공차 (tolerance)입니다.
- `C`: float, default=1.0
  - 오류 항에 대한 벌칙 파라미터입니다. 클수록 모델이 과적합될 가능성이 높아집니다.
- `epsilon`: float, default=0.1
  - 무감각 영역(epsilon tube) 내의 오류에 대해 페널티를 부과하지 않습니다.
- `shrinking`: bool, default=True
  - 최적화 문제를 해결하기 위해 휴리스틱 방법을 사용할지 여부를 지정합니다.
- `cache_size`: float, default=200
  - 커널 캐시 크기 (MB)입니다.
- `verbose`: bool, default=False
  - 디버깅 출력을 활성화합니다.
- `max_iter`: int, default=-1
  - 최대 반복 횟수를 지정합니다. -1은 제한이 없음을 의미합니다.

### 2. 리턴값
- `SVR` 객체 자체를 반환합니다. 이 객체는 학습된 모델과 그에 따른 메서드들을 포함합니다.

### 3. 주요 속성
- `support_`: array of shape (n_SV,)
  - 서포트 벡터의 인덱스입니다.
- `support_vectors_`: array of shape (n_SV, n_features)
  - 서포트 벡터입니다.
- `dual_coef_`: array of shape (1, n_SV)
  - 각 서포트 벡터의 듀얼 계수입니다.
- `coef_`: array of shape (1, n_features) or (n_classes, n_features)
```

- 선형 커널의 경우 모델 계수입니다. 커널이 'linear'가 아닌 경우에는 제공되지 않습니다.
- ``intercept_``: array of shape (1,) or (n_classes,)
- 모델의 상수항 (절편)입니다.
- ``n_support_``: array of shape (n_classes,), dtype=int32
- 각 클래스의 서포트 벡터 개수입니다.

4. 주요 함수

- ``fit(X, y)``: 모델 학습 함수
 - ``X``: array-like, shape (n_samples, n_features)
 - ``y``: array-like, shape (n_samples,)
 - 모델을 학습 데이터에 맞게 학습시킵니다.
 - ``predict(X)``: 예측 함수
 - ``X``: array-like, shape (n_samples, n_features)
 - 학습된 모델을 사용하여 입력 데이터 ``X``에 대한 예측값을 반환합니다.
 - ``score(X, y)``: 평가 함수
 - ``X``: array-like, shape (n_samples, n_features)
 - ``y``: array-like, shape (n_samples,)
 - 모델의 결정 계수 R^2 점수를 반환합니다.
 - ``get_params(deep=True)``: 파라미터 반환 함수
 - 모델의 파라미터를 dictionary 형태로 반환합니다.
 - ``set_params(params)``: 파라미터 설정 함수
 - 모델의 파라미터를 설정합니다.
- ```
...
```
- ```
print('')
```

In [138...

```
'''
이 코드는 여러 SVM(Support Vector Machine) 모델을 다양한 매개변수 설정으로 학습시키고,
테스트 데이터와 훈련 데이터에서의 MAE(Mean Absolute Error)를 계산하여 최적의 매개변수를 찾는 작업을 수행합니다.
'''
```

동작 설명

1. 파라미터 설정:

- SVM 모델을 위한 다양한 매개변수 설정을 정의합니다.
- ``linear``, ``poly``(다항식), ``rbf``(방사형 기저 함수) 커널을 사용합니다.
- 각 설정에는 커널 유형과 해당 커널에 필요한 추가 매개변수들이 포함됩니다.

2. 특징 열 목록 생성:

- SVM 모델에 사용할 특징 열들의 리스트를 만듭니다.
- ``TotalHour``와 ``lag_1``부터 ``lag_6``까지의 열들이 포함됩니다.

3. MAE 저장 리스트 초기화:

- 테스트 데이터와 훈련 데이터에서의 MAE를 저장할 리스트를 초기화합니다.

4. 모델 학습 및 예측:

- `for` 루프를 사용하여 각 파라미터 설정에 대해 SVM 모델을 학습시킵니다.
- `SVR` 객체를 생성할 때, `param`을 사용하여 사전의 각 매개변수를 언팩(unpack)하여 전달합니다.
- `reg\_svr.fit(df\_prob4\_train[X\_svm], df\_prob4\_train['target'])`를 사용하여 훈련 데이터로 모델을 학습시킵니다.
- 테스트 데이터와 훈련 데이터에 대해 예측을 수행하고, 그 결과로부터 MAE를 계산하여 각각 `mae\_test`와 `mae\_train` 리스트에 추가합니다.
- 각 파라미터 설정에 대한 MAE 값을 출력합니다.

5. 최적 파라미터 찾기:

- `mae\_test` 리스트에서 가장 작은 MAE 값을 가지는 인덱스를 찾습니다.
- `np.argmin(mae\_test)`는 가장 작은 MAE 값을 가지는 인덱스를 반환하며, 여기서 1을 더하여 1부터 시작하는 인덱스로 만듭니다.
- `np.min(mae\_test)`는 가장 작은 MAE 값을 반환합니다.
- `A`는 최적의 매개변수 설정의 인덱스를 나타내고, `B`는 해당 설정에서의 MAE 값을 나타냅니다.

요약

- 다양한 매개변수 설정으로 SVM 모델을 학습시키고 테스트 데이터와 훈련 데이터에서의 MAE를 계산합니다.
- 가장 낮은 MAE 값을 가지는 최적의 매개변수 설정을 찾습니다.
- 최적의 매개변수 설정의 인덱스와 해당 MAE 값을 출력합니다.

```

# 3번의 개별 수행으로도 풀 수 있겠지만,
# dictionary unpacking + 반복문을 통해 해결해봅니다.
# dictionary unpacking: key는 파라미터 명, value는 값으로 하여 호출합니다.

```

```

from sklearn.svm import SVR # Support Vector Regressor
from sklearn.metrics import mean_absolute_error

params = [
    {'kernel': 'linear'}, # 1
    {'kernel': 'poly', 'degree': 2, 'gamma': 0.1, 'coef0': 1}, # 2
    {'kernel': 'rbf', 'gamma': 0.1} # 3
]

X_svm = ['TotalHour'] + ['lag_{}'.format(i) for i in range(1, 7)]
print('X_svm = ', X_svm)

mae_test, mae_train = list(), list()

for param in params:
    reg_svr = SVR(C=10, **param) # **: dictionary unpacking 연산자입니다.

```

```

reg_svr.fit(df_prob4_train[X_svm], df_prob4_train['target'])
mae_test.append(
    mean_absolute_error(df_prob4_test['target'], reg_svr.predict(df_prob4_test[X_svm]))
)
mae_train.append(
    mean_absolute_error(df_prob4_train['target'], reg_svr.predict(df_prob4_train[X_svm]))
)
print(param, mae_test[-1], mae_train[-1])

```

```
A = np.argmin(mae_test) + 1
```

```
B = np.min(mae_test)
```

```
A, B
```

```

X_svm = ['TotalHour', 'lag_1', 'lag_2', 'lag_3', 'lag_4', 'lag_5', 'lag_6']
{'kernel': 'linear'} 32.88361068048479 83.981486070874
{'kernel': 'poly', 'degree': 2, 'gamma': 0.1, 'coef0': 1} 32.04119565841312 73.6963235867671
{'kernel': 'rbf', 'gamma': 0.1} 30.017658858928023 70.43814330923178

```

```
Out[138]: (3, 30.017658858928023)
```

```
In [139... A, B, A + B, round(A + B, 2)
```

```
Out[139]: (3, 30.017658858928023, 33.01765885892802, 33.02)
```

A + B 를 소수점 셋째 자리에서 반올림하여 둘째 자리까지 적으시오

33.02

```
In [165... Image('1-4.png')
```


Out[165]:

단계 4-5, Optional

퍼셉트론 회귀모델을 다음과 같이 3가지 설정으로 prob4_train으로 학습하고,

prob4_test로 성능을 측정한다.

가장 좋은 성능을 보인 경우의 번호(No.) C, 그 때의 성능을 D라고 한다.

성능 측정 지표: $MAE(Y, \hat{Y}) = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$

입력 변수: TotalHour, lag_1 ~ 6

대상 변수: target

신경망 설정:

No.	은닉층의 수	은닉층의 활성화 함수
1	[8]	relu
2	[8, 4]	relu
3	[8, 4]	sigmoid

함수가이드

`sklearn.neural_network.MLPRegressor`,

`learning_rate_init=0.05, random_state=123, max_iter=500`

In [140...

```
'''
`MLPRegressor` 클래스는 Scikit-learn 라이브러리에서 제공하는 다층 퍼셉트론(MLP) 회귀 모델을 생성하는 클래스입니다.
이 클래스는 회귀 문제를 해결하기 위해 사용됩니다.

### 1. `MLPRegressor` 클래스의 인자

`MLPRegressor` 클래스는 다음과 같은 주요 인자를 가집니다:

- `hidden_layer_sizes`: tuple, default=(100,)
```

- 은닉층의 유닛 개수를 정의합니다. 예를 들어 `(100, 50)`이면 두 개의 은닉층이 각각 100, 50개의 유닛을 가집니다.
- `activation`: {'identity', 'logistic', 'tanh', 'relu'}, default='relu'
 - 은닉층의 활성화 함수를 지정합니다. 사용 가능한 함수는 다음과 같습니다:
 - 'identity': $f(x) = x$
 - 'logistic': 시그모이드 함수, $f(x) = 1 / (1 + \exp(-x))$
 - 'tanh': 하이퍼볼릭 탄젠트 함수, $f(x) = \tanh(x)$
 - 'relu': 렐루 함수, $f(x) = \max(0, x)$
- `solver`: {'lbfgs', 'sgd', 'adam'}, default='adam'
 - 가중치 최적화에 사용되는 알고리즘을 지정합니다.
 - 'lbfgs': L-BFGS 최적화
 - 'sgd': 확률적 경사 하강법
 - 'adam': Adam 최적화
- `alpha`: float, default=0.0001
 - L2 정규화(가중치 감쇠) 항의 계수입니다.
- `batch\_size`: int, default='auto'
 - 미니 배치 크기입니다. 'auto'이면 `solver`가 'lbfgs'일 경우 전체 데이터, 'sgd'나 'adam'일 경우 $\min(200, n_samples)$ 입니다.
- `learning\_rate`: {'constant', 'invscaling', 'adaptive'}, default='constant'
 - 학습률 스케줄을 정의합니다.
 - 'constant': 일정한 학습률
 - 'invscaling': 학습률이 $1/t$ 로 감소
 - 'adaptive': 학습이 개선되지 않으면 학습률을 감소
- `learning\_rate\_init`: float, default=0.001
 - 초기 학습률입니다.
- `power\_t`: float, default=0.5
 - 'invscaling' 학습률 스케줄에서의 거듭제곱 계수입니다.
- `max\_iter`: int, default=200
 - 최대 반복 횟수입니다.
- `shuffle`: bool, default=True
 - 각 에포크마다 데이터를 섞을지 여부를 지정합니다.
- `random\_state`: int, RandomState instance, default=None
 - 난수 생성기 시드입니다.
- `tol`: float, default=1e-4
 - 종료 기준을 위한 공차 (tolerance)입니다.
- `verbose`: bool, default=False
 - 진행 상황을 출력할지 여부를 지정합니다.
- `warm\_start`: bool, default=False
 - `True`이면 이전 학습 결과를 초기화하지 않고 다음 학습에서 재사용합니다.
- `momentum`: float, default=0.9
 - 'sgd'일 경우 모멘텀 계수입니다.
- `nesterovs\_momentum`: bool, default=True
 - 'sgd'일 경우 네스테로프 모멘텀을 사용할지 여부를 지정합니다.
- `early\_stopping`: bool, default=False

- `True`이면 검증 점수가 향상되지 않으면 학습을 조기에 종료합니다.
- `validation\_fraction`: float, default=0.1
 - 검증을 위해 훈련 데이터를 분할하는 비율입니다.
 - `early\_stopping=True`일 때만 사용됩니다.
- `beta\_1`: float, default=0.9
 - 'adam' 최적화에서의 1차 모멘트 추정의 감쇠 계수입니다.
- `beta\_2`: float, default=0.999
 - 'adam' 최적화에서의 2차 모멘트 추정의 감쇠 계수입니다.
- `epsilon`: float, default=1e-8
 - 'adam' 최적화에서 수치적 안정성을 위한 항입니다.
- `n\_iter\_no\_change`: int, default=10
 - `early\_stopping=True`일 때, 검증 점수가 향상되지 않을 경우 학습을 중지하기 전에 기다리는 에포크 수입니다.
- `max\_fun`: int, default=15000
 - 'lbfgs' 솔버에서 허용되는 최대 함수 호출 횟수입니다.

2. 리턴값

- `MLPRegressor` 객체 자체를 반환합니다.
- 이 객체는 학습된 모델과 그에 따른 메서드들을 포함합니다.

3. 주요 속성

- `coefs\_`: list, length = n_layers - 1
 - 모델의 가중치 행렬입니다.
- `intercepts\_`: list, length = n_layers - 1
 - 모델의 편향 벡터입니다.
- `n\_iter\_`: int
 - 훈련하는 동안 실행된 반복 횟수입니다.
- `n\_layers\_`: int
 - 모델의 레이어 수입니다.
- `n\_outputs\_`: int
 - 출력 레이어의 유닛 수입니다.
- `out\_activation\_`: string
 - 출력 레이어의 활성화 함수입니다.

4. 주요 함수

- `fit(X, y)`: 모델 학습 함수
 - `X`: array-like, shape (n_samples, n_features)
 - `y`: array-like, shape (n_samples,) 또는 (n_samples, n_outputs)
 - 모델을 학습 데이터에 맞게 학습시킵니다.
- `predict(X)`: 예측 함수
 - `X`: array-like, shape (n_samples, n_features)
 - 학습된 모델을 사용하여 입력 데이터 `X`에 대한 예측값을 반환합니다.

```
- `score(X, y)` : 평가 함수
- `X` : array-like, shape (n_samples, n_features)
- `y` : array-like, shape (n_samples,) 또는 (n_samples, n_outputs)
- 모델의 결정 계수  $R^2$  점수를 반환합니다.
- `get_params(deep=True)` : 파라미터 반환 함수
- 모델의 파라미터를 dictionary 형태로 반환합니다.
- `set_params(params)` : 파라미터 설정 함수
- 모델의 파라미터를 설정합니다.
```

위 내용은 `MLPRegressor` 클래스의 주요 기능과 사용 방법에 대한 설명입니다.
 각 속성과 함수는 Scikit-learn의 공식 문서를 참고하면 더 자세한 정보를 얻을 수 있습니다.

```
'''
print('')
```

In [141...

```
'''
이 코드는 다양한 MLPRegressor(다층 퍼셉트론 회귀모델) 매개변수 설정을 사용하여 모델을 학습시키고,
테스트 데이터와 훈련 데이터에서의 MAE(Mean Absolute Error)를 계산하여 최적의 매개변수를 찾는 작업을 수행합니다.

### 동작 설명

1. 특징 열 목록 생성 :
- MLP 모델에 사용할 특징 열들의 리스트를 만듭니다.
- `TotalHour`와 `lag_1`부터 `lag_6`까지의 열들이 포함됩니다.

2. 파라미터 설정 :
- MLP 모델을 위한 다양한 매개변수 설정을 정의합니다.
- 각 설정에는 은닉층의 구조(`hidden_layer_sizes`)와 활성화 함수(`activation`)가 포함됩니다.

3. MAE 저장 리스트 초기화 :
- 테스트 데이터와 훈련 데이터에서의 MAE를 저장할 리스트를 초기화합니다.

4. 모델 학습 및 예측 :
- `for` 루프를 사용하여 각 파라미터 설정에 대해 MLP 모델을 학습시킵니다.
- `MLPRegressor` 객체를 생성할 때, `param`을 사용하여 사전의 각 매개변수를 언팩(unpack)하여 전달합니다.
- `reg_mlp.fit(df_prob4_train[X_mlp], df_prob4_train['target'])`를 사용하여 훈련 데이터로 모델을 학습시킵니다.
- 테스트 데이터와 훈련 데이터에 대해 예측을 수행하고, 그 결과로부터 MAE를 계산하여
  각각 `mae_test`와 `mae_train` 리스트에 추가합니다.
- 각 파라미터 설정에 대한 MAE 값을 출력합니다.

5. 최적 파라미터 찾기 :
- `mae_test` 리스트에서 가장 작은 MAE 값을 가지는 인덱스를 찾습니다.
```

- ``np.argmax(mae_test)``는 가장 작은 MAE 값을 가지는 인덱스를 반환하며, 여기서 1을 더하여 1부터 시작하는 인덱스로 만듭니다.
- ``np.min(mae_test)``는 가장 작은 MAE 값을 반환합니다.
- 첫 번째 반환 값은 최적의 매개변수 설정의 인덱스를 나타내고, 두 번째 반환 값은 해당 설정에서의 MAE 값을 나타냅니다.

요약

- 다양한 매개변수 설정으로 MLP 모델을 학습시키고 테스트 데이터와 훈련 데이터에서의 MAE를 계산합니다.
 - 가장 낮은 MAE 값을 가지는 최적의 매개변수 설정을 찾습니다.
 - 최적의 매개변수 설정의 인덱스와 해당 MAE 값을 출력합니다.
- ...

MLPRegressor의 파라미터 탐색 루틴입니다.

```
from sklearn.metrics import mean_absolute_error # MAE
from sklearn.neural_network import MLPRegressor

X_mlp = ['TotalHour'] + ['lag_{}'.format(i) for i in range(1, 7)]

print('X_mlp = ', X_mlp)

params = [
    # 은닉층: [8], 활성화함수 : 'relu'
    {'hidden_layer_sizes': [8], 'activation': 'relu'},
    # 은닉층: [8, 4], 활성화함수 : 'relu'
    {'hidden_layer_sizes': [8, 4], 'activation': 'relu'},
    # 은닉층: [8, 4], 활성화함수 : 'sigmoid'
    {'hidden_layer_sizes': [8, 4], 'activation': 'logistic'},
]

mae_test = list()
mae_train = list()

for param in params:
    reg_mlp = MLPRegressor(learning_rate_init=0.05, random_state=123, max_iter=500, **param)
    reg_mlp.fit(df_prob4_train[X_mlp], df_prob4_train['target'])

    mae_test.append(
        mean_absolute_error(df_prob4_test['target'], reg_mlp.predict(df_prob4_test[X_mlp]))
    )

    mae_train.append(
        mean_absolute_error(df_prob4_train['target'], reg_mlp.predict(df_prob4_train[X_mlp]))
    )
```

```

    )
    print(param, mae_test[-1], mae_train[-1])

np.argmin(mae_test) + 1, np.min(mae_test)

X_mlp = ['TotalHour', 'lag_1', 'lag_2', 'lag_3', 'lag_4', 'lag_5', 'lag_6']
{'hidden_layer_sizes': [8], 'activation': 'relu'} 39.68283671541335 82.07593559886139
{'hidden_layer_sizes': [8, 4], 'activation': 'relu'} 38.57988917988748 80.88441314317811
{'hidden_layer_sizes': [8, 4], 'activation': 'logistic'} 45.59789336930046 75.64436787584789

```

Out[141]: (2, 38.57988917988748)

문제 5

info에서 기온, 풍속, 습도 강수량 데이터에서 이상치를 탐색하고자 한다.

단계 5-1

info1에서 기온, 풍속, 습도, 강수량 만을 떼어 weather1 데이터프레임을 만든다.

```

In [142... X_5 = ['기온', '풍속', '습도', '강수량']
df_weather1 = df_info1[X_5].copy()
df_weather1.head()

```

Out[142]:

	기온	풍속	습도	강수량
0	-3.2	2.4	71	0.0
1	-4.5	1.5	77	0.0
2	-3.9	2.6	58	0.0
3	-4.1	2.6	56	0.0
4	-4.6	2.6	60	0.0

단계 5-2

weather1의 기온, 풍속, 습도, 강수량을 표준화 한다.

함수가이드

sklearn.preprocessing 제공 기능 활용

문제 지시 사항 외 Default 값 사용

```
In [143... from sklearn.preprocessing import StandardScaler

std_scaler = StandardScaler()

df_weather1[X_5] = std_scaler.fit_transform(df_weather1[X_5])
df_weather1.head()
```

```
Out[143]:
```

	기온	풍속	습도	강수량
0	-2.085903	0.289680	0.039195	-0.233447
1	-2.227830	-0.483098	0.300129	-0.233447
2	-2.162325	0.461409	-0.526162	-0.233447
3	-2.184160	0.461409	-0.613140	-0.233447
4	-2.238748	0.461409	-0.439184	-0.233447

단계 5-3

DBSCAN 군집화를 입실론(epsilon) 반경, 즉 이웃으로 간주되기 위한 최대거리 0.5,

중심점(Core Point)이 되기 위한 최소 이웃의 수 7(자기 자신을 포함한다.) 로 설정한다.

weather1에 DBSCAN 결과에서 소속 군집이 없는 샘플들을 이상점으로 간주한다.

이상점 여부를 파생변수 outlier_dbscan로 하여 weather1에 추가한다. (소속 군집이 있으면: False, 없으면 True)

입력변수: 기온, 풍속, 습도, 강수량

함수가이드

sklearn.cluster.DBSCAN, algorithm='brute'

문제 지시 사항 외 Default 값 사용

In [144...

```
'''
`DBSCAN` (Density-Based Spatial Clustering of Applications with Noise) 클래스는
Scikit-learn 라이브러리에서 제공하는 밀도 기반 군집화 알고리즘입니다.
`DBSCAN`은 밀도가 높은 데이터 포인트의 영역을 클러스터로 식별하고, 밀도가 낮은 포인트는 이상치로 간주합니다.

### 1. `DBSCAN` 클래스의 인자
- `eps`: float, default=0.5
  - 두 포인트가 같은 클러스터에 속하기 위해 최대 허용 거리입니다.
- `min_samples`: int, default=5
  - 한 클러스터 내에서 포인트가 핵심 포인트로 간주되기 위해 필요한 최소 샘플 수입니다.
- `metric`: string, or callable, default='euclidean'
  - 포인트 간 거리 계산에 사용되는 거리 측정 방법입니다. 기본값은 'euclidean' 거리입니다.
- `metric_params`: dict, default=None
  - 거리 측정 방법에 추가적으로 전달할 파라미터입니다.
- `algorithm`: {'auto', 'ball_tree', 'kd_tree', 'brute'}, default='auto'
  - 최근접 이웃을 계산할 때 사용할 알고리즘입니다. 'auto'는 데이터의 특성에 따라 최적의 알고리즘을 선택합니다.
- `leaf_size`: int, default=30
  - `BallTree` 또는 `KDTree` 알고리즘에서 사용하는 리프의 크기입니다. 리프의 크기는 트리 생성과 쿼리 속도에 영향을 미칩니다.
- `p`: float, default=None
  - `minkowski` 거리 계산에 사용할 거리 차수입니다. `metric='minkowski'`인 경우에만 사용됩니다.
- `n_jobs`: int, default=None
  - 병렬로 실행할 작업 수입니다. `-1`은 모든 프로세서를 사용합니다. `None`이면 단일 작업만 사용합니다.

### 2. 리턴값
- `DBSCAN` 객체 자체를 반환합니다. 이 객체는 학습된 모델과 그에 따른 메서드들을 포함합니다.

### 3. 주요 속성
- `core_sample_indices_`: array, shape (n_core_samples,)
  - 핵심 샘플의 인덱스입니다.
- `components_`: array, shape (n_core_samples, n_features)
  - 핵심 샘플의 배열입니다.
- `labels_`: array, shape (n_samples,)
```


- 각 샘플의 클러스터 레이블입니다. 클러스터에 속하지 않는 샘플은 레이블 -1로 지정됩니다.

4. 주요 함수

```
- `fit(X, y=None, sample_weight=None)`: 모델 학습 함수
- `X`: array-like, shape (n_samples, n_features)
  - 학습 데이터입니다.
- `y`: Ignored
  - 타겟 값은 무시됩니다.
- `sample_weight`: array-like, shape (n_samples,), default=None
  - 각 샘플의 가중치입니다.
- `fit_predict(X, y=None, sample_weight=None)`: 모델 학습 및 예측 함수
- `X`: array-like, shape (n_samples, n_features)
  - 학습 데이터입니다.
- `y`: Ignored
  - 타겟 값은 무시됩니다.
- `sample_weight`: array-like, shape (n_samples,), default=None
  - 각 샘플의 가중치입니다.
- 학습된 모델을 사용하여 입력 데이터 `X`에 대한 클러스터 레이블을 반환합니다.
- `get_params(deep=True)`: 파라미터 반환 함수
  - 모델의 파라미터를 dictionary 형태로 반환합니다.
- `set_params(params)`: 파라미터 설정 함수
  - 모델의 파라미터를 설정합니다.
- `fit_transform(X, y=None, sample_weight=None)`: 모델 학습 및 변환 함수
- `X`: array-like, shape (n_samples, n_features)
  - 학습 데이터입니다.
- `y`: Ignored
  - 타겟 값은 무시됩니다.
- `sample_weight`: array-like, shape (n_samples,), default=None
  - 각 샘플의 가중치입니다.
- 학습된 모델을 사용하여 입력 데이터 `X`를 클러스터 레이블로 변환합니다.
...
print('')
```

In [145...

```
'''
이 코드는 기온, 풍속, 습도, 강수량이라는 네 가지 기상 요소를 기반으로
DBSCAN(밀도 기반 클러스터링) 알고리즘을 사용하여 데이터를 클러스터링하는 과정을 보여줍니다.

### 동작 설명

1. 특징 열 선택:
```

- ``df_info1`` 데이터프레임에서 ``기온``, ``풍속``, ``습도``, ``강수량`` 열을 선택하여 ``df_weather1``이라는 새로운 데이터프레임을 만듭니다.
- ``copy()`` 메서드를 사용하여 원본 데이터프레임을 수정하지 않도록 합니다.

2. 표준화(스케일링):

- ``StandardScaler`` 객체를 생성합니다.
- ``fit_transform`` 메서드를 사용하여 선택된 기상 요소 열을 표준화합니다.
- 표준화는 데이터의 평균을 0, 표준편차를 1로 맞추어 변환합니다.

3. DBSCAN 객체 생성 및 설정:

- DBSCAN 객체를 생성하고, 주요 매개변수를 설정합니다.
 - ``eps = 0.5``: 이웃으로 간주되기 위한 최대 거리(반경).
 - ``min_samples = 7``: 중심점(Core Point)이 되기 위한 최소 이웃의 수. 최소 6개의 이웃과 자기 자신으로 구성됩니다.
 - ``algorithm = 'brute'``: 거리 계산을 위한 알고리즘. 'brute'는 모든 쌍의 거리를 계산합니다.

4. DBSCAN 클러스터링 수행:

- 표준화된 기상 요소 데이터를 사용하여 DBSCAN 클러스터링을 수행합니다.
- ``fit`` 메서드를 사용하여 데이터를 클러스터링하고 각 데이터 포인트에 클러스터 레이블을 할당합니다.

DBSCAN (밀도 기반 클러스터링) 개요

DBSCAN(Density-Based Spatial Clustering of Applications with Noise)은 밀도 기반 클러스터링 알고리즘으로, 데이터 포인트의 밀도에 따라 클러스터를 형성합니다. 주요 개념은 다음과 같습니다:

- Epsilon (eps): 주어진 포인트에서 이웃으로 간주되기 위한 최대 거리.
- Min Samples (min_samples): 중심점이 되기 위한 최소 이웃의 수. (자기 자신 포함)
- Core Point: 이웃이 min_samples 이상인 포인트.
- Border Point: 이웃이 min_samples 미만이지만 Core Point의 이웃인 포인트.
- Noise: 어떤 클러스터에도 속하지 않는 포인트.

이 알고리즘은 밀도가 높은 지역을 클러스터로 형성하고, 밀도가 낮은 지역을 노이즈로 간주합니다.

...

```
from sklearn.cluster import DBSCAN
```

```
dbscan = DBSCAN(
    # 입실론(epsilon) 반경, 즉 이웃으로 간주되기 위한 최대거리 0.5,
    eps = 0.5,
    # 중심점(Core Point)이 되기 위한 최소 이웃의 수 7(자기 자신을 포함한다.)
    min_samples = 7,
    algorithm='brute'
)
# predict에서 train과 동일한 샘플이라고 판단할 식별정보는 없습니다.
# 자기자신에 대한 개념이 있는 모델은, predict를 지원하지 않습니다.
```

```
# 대신 fit_predict만을 지원합니다.
```

```
dbscan.fit(df_weather1[X_5])
```

```
Out[145]: DBSCAN(algorithm='brute', eps=0.5, leaf_size=30, metric='euclidean',
                 metric_params=None, min_samples=7, n_jobs=None, p=None)
```

```
In [146...
```

```
'''
```

이 코드는 DBSCAN 클러스터링 알고리즘을 사용하여 이상치(outlier)를 식별하고,
그 결과를 `df\_weather1` 데이터프레임에 추가하는 작업을 수행합니다.

동작 설명

1. DBSCAN 클러스터 레이블을 사용하여 이상치 식별:

- `dbscan.labels\_`는 DBSCAN 알고리즘이 데이터 포인트에 할당한 클러스터 레이블을 포함하는 배열입니다.
- DBSCAN에서 `labels\_`가 `-1`인 데이터 포인트는 어떤 클러스터에도 속하지 않는 것으로 간주되며,
이는 이상치(outlier)를 의미합니다.
- `dbscan.labels\_ == -1`은 레이블이 `-1`인 데이터 포인트를 `True`로,
그렇지 않은 포인트를 `False`로 변환하는 Boolean 배열을 만듭니다.
- 이 Boolean 배열을 `df\_weather1` 데이터프레임의 새로운 열 `outlier\_dbscan`에 추가합니다.
- `outlier\_dbscan` 열은 각 데이터 포인트가 이상치인지 아닌지를 나타내는 Boolean 값 (`True` 또는 `False`)을 포함하게 됩니다.

2. 이상치의 수 계산:

- `df\_weather1['outlier\_dbscan']`는 `True`와 `False` 값을 포함하는 열입니다.
- `sum()` 메서드는 이 열에서 `True` 값의 개수를 합산합니다.
- Boolean 값에서 `True`는 `1`로, `False`는 `0`으로 간주되므로, `sum()` 메서드는 이상치의 수를 반환합니다.

요약

- `df\_weather1['outlier\_dbscan'] = dbscan.labels\_ == -1`:
DBSCAN 알고리즘의 결과에서 레이블이 `-1`인 데이터를 식별하여, 이를 새로운 열 `outlier\_dbscan`에 Boolean 값으로 저장합니다.
`True`는 이상치를, `False`는 정상 데이터를 나타냅니다.
- `df\_weather1['outlier\_dbscan'].sum()`:
`outlier\_dbscan` 열에서 `True` 값의 개수를 세어, 이상치의 총 개수를 계산합니다.

이 과정의 최종 결과는 `df\_weather1` 데이터프레임에 이상치 여부를 나타내는 열이 추가되고,
전체 데이터 중에서 이상치로 식별된 데이터 포인트의 수를 알 수 있게 됩니다.

```
'''
```

```
df_weather1
```

Out[146]:

	기온	풍속	습도	강수량
0	-2.085903	0.289680	0.039195	-0.233447
1	-2.227830	-0.483098	0.300129	-0.233447
2	-2.162325	0.461409	-0.526162	-0.233447
3	-2.184160	0.461409	-0.613140	-0.233447
4	-2.238748	0.461409	-0.439184	-0.233447
...	...	...	...	...
6163	0.632553	1.320051	0.648042	0.744500
6164	0.687140	1.835237	0.343618	0.744500
6165	0.687140	1.921101	0.256640	0.744500
6166	0.665305	0.375545	0.387107	0.744500
6167	0.665305	0.375545	0.387107	0.744500

6168 rows × 4 columns

In [147...]

```
# labels_ 에 군집화 결과가 들어 있습니다.
df_weather1['outlier_dbscan'] = dbscan.labels_ == -1
df_weather1['outlier_dbscan'].sum()
```

Out[147]: 210

단계 5-4

IsolationForest를 사용하여 weather1의 이상점을 탐색한다.

학습시 bootstrapping은 하지 않고, 트리의 수는 30개를 사용한다.

IsolationForest으로 탐지한 이상점 여부는 파생변수 outlier_isof로 하여 weather1에 추가한다.

(이상치라면 True, 아니면 False)

입력변수: 기온, 풍속, 습도, 강수량

함수가이드

sklearn.ensemble.IsolationForest, random_state=123, contamination='auto', behaviour='auto'

문제 지시 사항 외 Default 값 사용

In [148...

```
'''
`IsolationForest` 클래스는 Scikit-learn 라이브러리에서 제공하는 이상치 탐지 알고리즘 중 하나입니다.
`IsolationForest`는 주로 이상치 탐지(Anomaly Detection)에 사용되며, 데이터를 분리하는 방식으로 작동합니다.
이 알고리즘은 데이터의 임의의 하위 집합을 선택하고 무작위로 선택된 특성을 사용하여 데이터를 분할합니다.
이상치는 정상 데이터보다 더 적은 분할로 쉽게 분리됩니다.

### 1. `IsolationForest` 클래스의 인자
- `n_estimators`: int, default=100
  - 모델에서 사용할 트리의 수입니다.
- `max_samples`: int or float, default='auto'
  - 각 트리에서 사용할 샘플의 최대 수입니다.
  - `int`: 사용할 샘플의 수.
  - `float`: 데이터셋의 비율 ( $0 < \text{max\_samples} \leq 1$ ).
  - 'auto': min(256, n_samples).
- `contamination`: 'auto', float, default='auto'
  - 데이터셋에 있는 이상치의 비율을 지정합니다.
  - 'auto': 0.1로 설정됩니다.
  - float: 0에서 0.5 사이의 값이어야 합니다.
- `max_features`: int or float, default=1.0
  - 각 트리에서 사용할 특성의 최대 수입니다.
  - `int`: 사용할 특성의 수.
  - `float`: 특성의 비율 ( $0 < \text{max\_features} \leq 1$ ).
- `bootstrap`: bool, default=False
  - True로 설정하면 부트스트래핑을 사용합니다. 즉, 샘플을 중복 허용하여 선택합니다.
- `n_jobs`: int, default=None
  - 병렬로 실행할 작업 수입니다. -1은 모든 프로세서를 사용합니다. `None`이면 단일 작업만 사용합니다.
- `random_state`: int, RandomState instance or None, default=None
  - 난수 생성기 시드입니다.
- `verbose`: int, default=0
  - 디버깅 출력을 활성화합니다.
- `warm_start`: bool, default=False
```

- True로 설정하면 기존 모델에 추정기를 추가하여 학습합니다.

2. 리턴값

- `IsolationForest` 객체 자체를 반환합니다. 이 객체는 학습된 모델과 그에 따른 메서드들을 포함합니다.

3. 주요 속성

- `estimators\_`: list of DecisionTreeClassifier
 - 학습된 각 트리 모델입니다.
- `max\_samples\_`: int
 - 각 트리에서 사용된 샘플의 수입니다.
- `n\_features\_in\_`: int
 - 입력 데이터의 특성 수입니다.
- `feature\_names\_in\_`: ndarray of shape (n_features_in_,)
 - 입력 데이터의 특성 이름입니다.

4. 주요 함수

- `fit(X, y=None, sample\_weight=None)`: 모델 학습 함수
 - `X`: array-like, shape (n_samples, n_features)
 - 학습 데이터입니다.
 - `y`: Ignored
 - 타겟 값은 무시됩니다.
 - `sample\_weight`: array-like, shape (n_samples,), default=None
 - 각 샘플의 가중치입니다.
 - 학습 데이터를 사용하여 `IsolationForest` 모델을 학습시킵니다.
- `predict(X)`: 예측 함수
 - `X`: array-like, shape (n_samples, n_features)
 - 예측할 데이터입니다.
 - 각 샘플에 대한 예측값을 반환합니다. 1은 정상 샘플, -1은 이상치 샘플을 나타냅니다.
- `fit\_predict(X, y=None, sample\_weight=None)`: 학습 및 예측 함수
 - `X`: array-like, shape (n_samples, n_features)
 - 학습 및 예측할 데이터입니다.
 - 학습 데이터를 사용하여 모델을 학습시키고, 각 샘플에 대한 예측값을 반환합니다.
- `decision\_function(X)`: 결정 함수
 - `X`: array-like, shape (n_samples, n_features)
 - 평가할 데이터입니다.
 - 각 샘플에 대한 이상치 점수를 반환합니다. 점수가 낮을수록 이상치일 가능성이 높습니다.
- `score\_samples(X)`: 샘플 점수 함수
 - `X`: array-like, shape (n_samples, n_features)
 - 평가할 데이터입니다.
 - 각 샘플에 대한 음의 이상치 점수를 반환합니다. 결정 함수의 반대입니다.

```
- `get_params(deep=True)` : 파라미터 반환 함수
- 모델의 파라미터를 dictionary 형태로 반환합니다.
- `set_params(params)` : 파라미터 설정 함수
- 모델의 파라미터를 설정합니다.
'''
print('')
```

In [149...

```
'''
이 코드는 Isolation Forest 알고리즘을 사용하여 데이터에서 이상치(outlier)를 탐지하는 과정을 수행합니다.

### 동작 설명

1. Isolation Forest 모델 생성:
- `IsolationForest`는 sklearn.ensemble 모듈에서 제공하는 이상치 탐지 모델입니다.
- `n_estimators=30`은 30개의 결정 트리를 사용하여 모델을 구성함을 의미합니다.
- `bootstrap=False`는 부트스트래핑을 사용하지 않음을 의미합니다.
  즉, 원본 데이터를 그대로 사용하여 트리를 학습합니다.
- `random_state=123`은 난수 발생기의 상태를 고정하여 결과의 재현성을 보장합니다.
- `contamination='auto'`는 이상치의 비율을 자동으로 결정하게 합니다.
- `behaviour='auto'`는 사이킷런의 버전 호환성을 위해 사용될 수 있지만, 최신 버전에서는 불필요할 수 있습니다.

2. Isolation Forest 모델 학습:
- `fit` 메서드는 모델을 데이터에 맞추어 학습시킵니다.
- `df_weather1[X_5]`는 `df_weather1` 데이터프레임의 `X_5` 열들, 즉 ['기온', '풍속', '습도', '강수량']를 학습 데이터로 사용합니다.
- Isolation Forest는 각 데이터 포인트를 분리하는 데 필요한 경로 길이를 기반으로 이상치를 탐지합니다.
  경로가 짧을수록 이상치일 가능성이 높습니다.

### 요약
- Isolation Forest 모델을 설정하고, 이를 사용하여 데이터의 이상치를 탐지합니다.
- `n_estimators=30`은 30개의 트리를 사용하여 모델을 학습합니다.
- `bootstrap=False`는 부트스트래핑 없이 원본 데이터를 사용하여 학습합니다.
- `random_state=123`은 난수 발생기를 고정하여 재현성을 보장합니다.
- `contamination='auto'`는 이상치 비율을 자동으로 결정합니다.
- `fit(df_weather1[X_5])`는 `df_weather1` 데이터프레임의 `X_5` 열들로 모델을 학습시킵니다.

이렇게 학습된 Isolation Forest 모델은 이후에 새로운 데이터에 대해 이상치 여부를 예측할 수 있게 됩니다.
'''

from sklearn.ensemble import IsolationForest

isof = IsolationForest(
```

```

n_estimators=30, # 트리의 수는 30개를 사용한다.
bootstrap=False, # 학습시 bootstrapping은 하지 않고,
random_state=123, contamination='auto', behaviour='auto'
)

isof.fit(df_weather1[X_5])

```

```

Out[149]: IsolationForest(behaviour='auto', bootstrap=False, contamination='auto',
                          max_features=1.0, max_samples='auto', n_estimators=30,
                          n_jobs=None, random_state=123, verbose=0, warm_start=False)

```

In [150...

```

'''
코드는 Isolation Forest 모델을 사용하여 데이터에서 이상치를 탐지하고, 그 결과를 데이터프레임에 저장한 후 이상치의 개수를 계산합니다

### 동작 설명

1. Isolation Forest 예측:
- `isof.predict` 메서드는 학습된 Isolation Forest 모델을 사용하여 `df_weather1[X_5]`의 각 데이터 포인트에 대해 이상치 여부를 예
- `df_weather1[X_5]`는 `df_weather1` 데이터프레임에서 '기온', '풍속', '습도', '강수량' 열을 포함한 부분입니다.
- `predict` 메서드는 각 데이터 포인트에 대해 1 또는 -1을 반환합니다. 1은 정상 데이터를 의미하고, -1은 이상치를 의미합니다.

2. 이상치 여부 저장:
- `isof.predict(df_weather1[X_5]) == -1`은 예측 결과가 -1인지 여부를 체크하여 불리언 배열을 만듭니다.
- `df_weather1['outlier_isof']`에 이 불리언 배열을 저장합니다.
  즉, 각 데이터 포인트가 이상치인 경우 `True`, 정상인 경우 `False`가 저장됩니다.

3. 이상치 개수 계산:
- `df_weather1['outlier_isof']` 열의 `True` 값의 개수를 계산합니다.
- 불리언 값은 `True`가 1, `False`가 0으로 간주되기 때문에, `sum` 메서드를 사용하여 `True` 값의 개수를 계산할 수 있습니다.
- 이 값은 데이터프레임에서 탐지된 이상치의 총 개수를 의미합니다.

### 요약
- `isof.predict(df_weather1[X_5])`는 Isolation Forest 모델을 사용하여 데이터에서 이상치 여부를 예측합니다.
- 예측 결과가 -1인 경우를 `True`로 변환하여 `df_weather1['outlier_isof']` 열에 저장합니다.
- `df_weather1['outlier_isof'].sum()`은 데이터프레임에서 탐지된 이상치의 총 개수를 계산합니다.

이 코드는 전체 데이터에서 이상치가 몇 개인지 확인하는 데 유용하며, 이후 분석이나 전처리 단계에서 이 정보를 사용할 수 있습니다.
'''

df_weather1

```


Out[150]:

	기온	풍속	습도	강수량	outlier_dbscan
0	-2.085903	0.289680	0.039195	-0.233447	False
1	-2.227830	-0.483098	0.300129	-0.233447	False
2	-2.162325	0.461409	-0.526162	-0.233447	False
3	-2.184160	0.461409	-0.613140	-0.233447	False
4	-2.238748	0.461409	-0.439184	-0.233447	False
...	...	...	...	...	...
6163	0.632553	1.320051	0.648042	0.744500	False
6164	0.687140	1.835237	0.343618	0.744500	True
6165	0.687140	1.921101	0.256640	0.744500	True
6166	0.665305	0.375545	0.387107	0.744500	True
6167	0.665305	0.375545	0.387107	0.744500	True

6168 rows × 5 columns

```
In [151]: df_weather1['outlier_isof'] = isof.predict(df_weather1[X_5]) == -1
df_weather1['outlier_isof'].sum()
```

Out[151]: 1108

```
In [152]: '''
코드는 Isolation Forest 모델의 `decision_function` 메서드와 `predict` 메서드를 사용하여 이상치 탐지의 일관성을 확인하는 작업을 수행

### 동작 설명
1. Decision Function 결과 계산:
- `isof.decision_function(df_weather1[X_5])`는 Isolation Forest 모델의 결정 함수 값을 반환합니다.
- 결정 함수 값은 각 데이터 포인트의 이상치 점수를 의미합니다. 값이 낮을수록 이상치일 가능성이 높습니다.

2. Decision Function 결과 비교:
isof.decision_function(df_weather1[X_5]) < 0
- 결정 함수 값이 0보다 작은지 여부를 확인합니다.
- 이는 이상치 점수가 0보다 낮은 경우 이상치로 간주하기 위한 조건입니다.
- 결과는 불리언 배열로, 각 데이터 포인트에 대해 결정 함수 값이 0보다 작은 경우 `True`, 그렇지 않은 경우 `False`입니다.
```

3. Predict 결과 계산:

- ``isof.predict(df_weather1[X_5])``는 Isolation Forest 모델을 사용하여 각 데이터 포인트가 이상치인지 정상인지 예측합니다.
- 예측 값은 1(정상) 또는 -1(이상치)입니다.

4. Predict 결과 비교:

- 예측 값이 -1인지 여부를 확인합니다.
- 이는 각 데이터 포인트가 이상치인지 여부를 불리언 배열로 나타냅니다. 이상치인 경우 ``True``, 그렇지 않은 경우 ``False``입니다.

5. Decision Function 결과와 Predict 결과의 불일치 계산:

- ``!=`` 연산자를 사용하여 결정 함수 결과와 예측 결과의 불일치를 확인합니다.
- 두 불리언 배열을 비교하여 값이 다르면 ``True``, 같으면 ``False``를 반환합니다.
- 결과는 불일치 여부를 나타내는 불리언 배열입니다.

6. 불일치 개수 계산:

- 불일치 여부를 나타내는 불리언 배열에서 ``True`` 값의 개수를 계산합니다.
- ``sum`` 메서드는 ``True`` 값의 개수를 합산하여, 결정 함수와 예측 결과가 불일치한 데이터 포인트의 총 개수를 반환합니다.

요약

- ``isof.decision_function(df_weather1[X_5]) < 0``와 ``isof.predict(df_weather1[X_5]) == -1``의 결과를 비교하여 불일치 여부를 확인합니다.
- 결정 함수 값이 0보다 작은 경우와 예측 값이 -1인 경우가 일치하지 않는 데이터 포인트의 개수를 계산합니다.
- 이 코드는 Isolation Forest 모델의 결정 함수와 예측 결과의 일관성을 검증하기 위한 것입니다.
- 불일치가 발생하면 모델의 이상치 탐지 결과에 문제가 있을 수 있습니다.

```
((isof.decision_function(df_weather1[X_5]) < 0) != (isof.predict(df_weather1[X_5]) == -1)).sum()
```

Out[152]: 0

단계 5-5

LocalOutlierFactor를 사용하여 weather1의 이상점을 탐색한다.

관찰할 주변의 샘플은 10개이다. 그리고, 샘플간의 거리는 유클리디언 거리를 사용한다.

주어진 데이터셋에 대해서만 이상치 판별을 위해 모델을 만들도록 설정한다.

LocalOutlierFactor로 탐지한 이상 여부를 outlier_lof로 하여 weather1에 추가한다.

입력변수: 기온, 풍속, 습도, 강수량

함수가이드

sklearn.neighbors.LocalOutlierFactor, contamination='auto'

In [153...

```
'''
코드는 `LocalOutlierFactor` (LOF) 알고리즘을 사용하여 데이터 프레임 `df_weather1`의 이상치를 탐지하는 과정을 보여줍니다.

### 동작 설명
1. LocalOutlierFactor 객체 생성:
    - `LocalOutlierFactor` 객체 `lof`를 생성합니다.
    - `n_neighbors=10`: 각 데이터 포인트의 이상치를 평가할 때 고려할 이웃의 수를 10개로 설정합니다.
    - `metric='euclidean`: 유클리드 거리를 사용하여 데이터 포인트 간의 거리를 계산합니다.
    - `novelty=False`: 모델을 주어진 데이터셋에 대해서만 훈련하고 이상치를 탐지합니다. 새로운 데이터를 예측하지 않습니다.
    - `contamination='auto': 데이터셋에서 자동으로 이상치의 비율을 추정합니다.

2. LOF 모델을 사용하여 이상치 탐지:
    - `lof.fit_predict(df_weather1[X_5])`:
        - `df_weather1[X_5]`에 있는 데이터 포인트에 대해 LOF 모델을 훈련하고 예측합니다.
        - `fit_predict` 메서드는 각 데이터 포인트가 정상인지 이상치인지를 예측합니다.
        - 결과는 1 (정상) 또는 -1 (이상치)입니다.
    - `== -1`:
        - 예측 결과가 -1인 데이터 포인트를 이상치로 간주합니다.
        - 결과는 불리언 배열로, 각 데이터 포인트에 대해 이상치인 경우 `True`, 그렇지 않은 경우 `False`입니다.
    - `df_weather1['outlier_lof']`에 이상치 여부를 저장합니다.

3. 이상치 개수 계산:
    df_weather1['outlier_lof'].sum()
    - `df_weather1['outlier_lof']` 열에서 `True` 값의 개수를 합산합니다.
    - 이는 데이터 프레임 `df_weather1`에서 탐지된 이상치의 총 개수를 반환합니다.

### 요약
- `LocalOutlierFactor` 객체를 생성하고, LOF 알고리즘을 사용하여 데이터 프레임 `df_weather1`의 이상치를 탐지합니다.
- `fit_predict` 메서드를 사용하여 각 데이터 포인트가 이상치인지 여부를 예측합니다.
- 예측 결과가 -1인 경우를 이상치로 간주하고, 이를 불리언 배열로 저장합니다.
- 데이터 프레임에서 탐지된 이상치의 총 개수를 계산하여 반환합니다.
'''

from sklearn.neighbors import LocalOutlierFactor
```

```

lof = LocalOutlierFactor(
    n_neighbors=10, # 관찰할 주변의 샘플은 10개이다.
    metric='euclidean', # 그리고, 샘플간의 거리는 유클리디언 거리를 사용한다.
    # 주어진 데이터셋에 대해서만 이상치 판별을 위해 모델을 만들도록 설정한다.
    novelty=False,
    contamination='auto'
)

df_weather1['outlier_lof'] = lof.fit_predict(df_weather1[X_5]) == -1
df_weather1

```

Out[153]:

	기온	풍속	습도	강수량	outlier_dbscan	outlier_isof	outlier_lof
0	-2.085903	0.289680	0.039195	-0.233447	False	False	False
1	-2.227830	-0.483098	0.300129	-0.233447	False	False	False
2	-2.162325	0.461409	-0.526162	-0.233447	False	False	False
3	-2.184160	0.461409	-0.613140	-0.233447	False	False	False
4	-2.238748	0.461409	-0.439184	-0.233447	False	False	False
...	...	...	...	...	...	...	...
6163	0.632553	1.320051	0.648042	0.744500	False	True	False
6164	0.687140	1.835237	0.343618	0.744500	True	True	False
6165	0.687140	1.921101	0.256640	0.744500	True	True	False
6166	0.665305	0.375545	0.387107	0.744500	True	False	False
6167	0.665305	0.375545	0.387107	0.744500	True	False	False

6168 rows × 7 columns

In [154... df_weather1['outlier_lof'].sum()

Out[154]: 83

In [155...
 '''
 `LocalOutlierFactor` (LOF) 클래스는 Scikit-learn 라이브러리에서 제공하는 이상치 탐지 알고리즘 중 하나입니다.

LOF는 각 데이터 포인트의 국지적인 밀도를 기반으로 이상치를 탐지합니다.
이 알고리즘에서 중요한 두 가지 속성은 ``negative_outlier_factor_``와 ``offset_``입니다.

1. ``negative_outlier_factor_`` 속성

- 정의: ``negative_outlier_factor_`` 속성은 각 데이터 포인트의 LOF 점수를 부호를 바꾸어 저장한 배열입니다.
- 값: LOF 점수는 원래 클수록 정상적인 포인트를 의미하며, 값이 작을수록 (즉, 음수 값의 절대값이 클수록) 이상치일 가능성이 높습니다.
- 용도: 이 속성은 각 데이터 포인트가 얼마나 이상치에 가까운지 나타냅니다.
값이 -1에 가까울수록 정상에 가깝고, 값이 작을수록 이상치일 가능성이 높습니다.

2. ``offset_`` 속성

- 정의: ``offset_`` 속성은 이상치와 정상 데이터를 구분하는 기준점을 의미합니다.
- 값: ``offset_``은 ``contamination`` 파라미터에 의해 결정됩니다.
``contamination``은 데이터셋에서 이상치의 예상 비율을 지정하는 파라미터입니다.
- 용도: ``offset_``은 ``negative_outlier_factor_`` 값과 비교하여 데이터 포인트가 이상치인지 아닌지를 판단하는 데 사용됩니다.

3. ``negative_outlier_factor_``와 ``offset_`` 사이의 관계

``negative_outlier_factor_``와 ``offset_``의 관계는 데이터 포인트를 이상치와 정상 데이터로 분류하는 데 핵심적입니다.

- 이상치 판별 기준:
 - ``negative_outlier_factor_`` 값이 ``offset_``보다 작으면 해당 데이터 포인트는 이상치로 간주됩니다.
 - ``negative_outlier_factor_`` 값이 ``offset_``보다 크거나 같으면 해당 데이터 포인트는 정상 데이터로 간주됩니다.

아래 예제에서 ``negative_outlier_factor_`` 속성은 각 데이터 포인트의 LOF 점수를 부호를 바꾸어 나타낸 배열을 출력합니다.

``offset_`` 속성은 이상치와 정상 데이터를 구분하는 기준값을 나타냅니다.

``fit_predict`` 메서드는 각 데이터 포인트가 이상치인지 정상 데이터인지를 나타내는 예측값을 반환합니다.

요약

- ``negative_outlier_factor_``는 각 데이터 포인트의 LOF 점수를 부호를 바꾸어 저장한 속성입니다.
값이 작을수록 이상치일 가능성이 높습니다.
- ``offset_``은 이상치와 정상 데이터를 구분하는 기준점을 나타내며, ``contamination`` 파라미터에 의해 결정됩니다.
- 두 속성은 함께 사용되어 데이터 포인트를 이상치와 정상 데이터로 분류합니다.
``negative_outlier_factor_`` 값이 ``offset_``보다 작으면 해당 데이터 포인트는 이상치로 간주됩니다.

예제

```
from sklearn.neighbors import LocalOutlierFactor
```

```
# 예제 데이터 생성
```

```
X = [[-1.1], [0.2], [101.1], [0.3]]
```

```
# LOF 모델 생성 및 학습
```

```
clf = LocalOutlierFactor(n_neighbors=2)
```

```

clf.fit(X)

# negative_outlier_factor_ 값 확인
print(clf.negative_outlier_factor_)

# offset_ 값 확인
print(clf.offset_)

# 이상치 판별
predictions = clf.fit_predict(X)
print(predictions)

print(clf.negative_outlier_factor_ < clf.offset_)

[ -0.98214286 -1.03703704 -73.36970899 -0.98214286]
-51.66990740371985
[ 1  1 -1  1]
[False False  True False]

```

In [156...

```

'''
코드는 `LocalOutlierFactor` (LOF) 모델에서 이상치 판별에 사용된 두 가지 기준의 일치 여부를 확인하는 과정을 보여줍니다.
각각의 기준에 의해 판별된 이상치가 일치하지 않는 경우의 수를 계산합니다.

### 동작 설명
1. LOF 모델 훈련 및 예측:
  - `lof.fit_predict(df_weather1[X_5])`:
    - `df_weather1[X_5]` 데이터에 대해 LOF 모델을 훈련하고 예측합니다.
    - 결과는 각 데이터 포인트가 정상인지 이상치인지를 나타내는 배열입니다. 1은 정상, -1은 이상치를 의미합니다.

2. Negative Outlier Factor 및 Offset 계산:
  - `lof.negative_outlier_factor_`:
    - LOF 모델이 각 데이터 포인트에 대해 계산한 negative outlier factor(NOF)를 반환합니다.
    - NOF 값이 작을수록 해당 데이터 포인트가 더 이상치에 가까움을 나타냅니다.
  - `lof.offset_`:
    - LOF 모델이 사용하는 기준값 (offset)입니다. NOF 값이 이 offset 값보다 작으면 해당 데이터 포인트를 이상치로 간주합니다.

3. 이상치 판별 기준 비교:
  - `lof.negative_outlier_factor_ < lof.offset_`:
    - 각 데이터 포인트의 NOF 값이 offset 값보다 작은지 여부를 판단합니다.
    - 결과는 불리언 배열로, `True`는 이상치, `False`는 정상 데이터를 의미합니다.
  - `lof.fit_predict(df_weather1[X_5]) == -1`:
    - `fit_predict` 메서드에 의해 예측된 이상치 여부를 나타내는 불리언 배열입니다.

```

- `True`는 이상치, `False`는 정상 데이터를 의미합니다.

4. 두 기준의 불일치 여부 계산:

```
- `(lof.negative_outlier_factor_ < lof.offset_) != (lof.fit_predict(df_weather1[X_5]) == -1)`:
```

- 두 기준에 의해 판별된 이상치 여부가 서로 다른지 여부를 판단합니다.
- 결과는 불리언 배열로, `True`는 두 기준이 불일치하는 데이터 포인트를 의미합니다.

5. 불일치한 데이터 포인트의 개수 계산:

```
- `.sum()`:
```

- 불리언 배열에서 `True` 값의 개수를 합산합니다.
- 이는 두 기준이 불일치하는 데이터 포인트의 총 개수를 반환합니다.

요약

이 코드는 `LocalOutlierFactor` 모델에서 이상치 판별에 사용된 두 가지 기준 (`negative_outlier_factor_`와 `fit_predict` 메서드에 의한 판별)의 일치 여부를 비교합니다. 그런 다음, 일치하지 않는 데이터 포인트의 수를 계산하여 반환합니다. 이를 통해 모델의 두 가지 판별 기준이 얼마나 일치하는지 확인할 수 있습니다.

```
...
```

```
# 참고 -LOF: lof.negative_outlier_factor_ 입니다.
# 이 값이 lof.offset_ 보다 작으면 outlier로 분류 됩니다.
# 가이드에서는 contamination='auto'이면 offset_ = -1.5로 세팅이 되게됩니다.
# 그렇지 않으면 contamination에서 정의한 비율에 의해 offset_이 지정이 됩니다.
```

```
((lof.negative_outlier_factor_ < lof.offset_) != (lof.fit_predict(df_weather1[X_5]) == -1)).sum()
```

Out[156]: 0

In [157...

```
...
```

이 코드는 `LocalOutlierFactor` (LOF) 모델을 사용하여 데이터의 이상치를 탐지하는 과정을 보여줍니다. LOF 모델은 주어진 데이터셋을 기반으로 각 데이터 포인트가 이상치인지 정상치인지를 판단합니다.

동작 설명

1. `LocalOutlierFactor` 객체 생성:

```
- `lof = LocalOutlierFactor(...)`:
```

- `LocalOutlierFactor` 클래스의 객체를 생성합니다.
- 매개변수 설명:
 - `contamination='auto'`: 이상치로 간주할 데이터 포인트의 비율을 자동으로 결정합니다.
 - `n_neighbors=10`: 각 데이터 포인트의 이상치 여부를 판단할 때 고려할 이웃 샘플의 수를 10개로 설정합니다.
 - `metric='euclidean'`: 유클리디언 거리 척도를 사용하여 샘플 간의 거리를 계산합니다.
 - `novelty=True`: 모델을 새로운 데이터의 이상치를 탐지할 수 있도록 설정합니다.

이 설정은 `fit` 메서드로 모델을 훈련하고, `predict` 메서드를 사용하여 새로운 데이터의 이상치를 예측할 수 있게 합니다.

2. 모델 훈련:

- `lof.fit(df_weather1[X_5])`:`
- `df_weather1`` 데이터프레임의 `X_5`` 컬럼에 해당하는 데이터를 사용하여 LOF 모델을 훈련합니다.
- `fit`` 메서드는 주어진 데이터셋을 사용하여 각 데이터 포인트의 이상치 여부를 판단하기 위한 기준을 학습합니다.

3. 이상치 예측:

- `lof.predict(df_weather1[X_5])`:`
- 훈련된 LOF 모델을 사용하여 `df_weather1[X_5]` 데이터의 이상치를 예측합니다.
- 반환 값은 배열로, 각 데이터 포인트가 정상치인 경우 `1`, 이상치인 경우 `-1`을 나타냅니다.

요약

이 코드는 `LocalOutlierFactor`` 모델을 생성하고, 특정 데이터셋을 사용하여 모델을 훈련한 후, 같은 데이터셋에 대해 이상치를 예측합니다. 이 과정에서 LOF 모델은 각 데이터 포인트의 이상치 여부를 판단하기 위해 설정된 이웃 샘플의 수와 유클리디언 거리를 기준으로 계산을 수행합니다. `novelty=True`` 설정을 통해 모델을 새로운 데이터에 대해 이상치 여부를 예측할 수 있게 설정합니다.

참고 novelty=True로 하면 신규성 탐색(novelty detection)으로 동작합니다.

```
lof = LocalOutlierFactor(
    contamination='auto',
    n_neighbors = 10, # 관찰할 주변의 샘플은 10개이다.
    metric='euclidean', # 샘플간의 거리는 유클리디언 거리를 사용한다.
    novelty=True # 신규성 탐색을 위해 모델을 만들도록 설정한다.
)
```

```
lof.fit(df_weather1[X_5])
lof.predict(df_weather1[X_5])
```

Out[157]: array([1, 1, 1, ..., 1, 1, 1])

단계 5-6

outlier_dbscan가 True인 행의 개수를 A, outlier_isof가 True인 행의 개수 B

, outlier_lof가 True인 행의 개수 C 이다.

In [158... *# True == 1 입니다. True인 개수는 sum을 통해서도 구할 수 있습니다.*
sum을 통한 개수 구하는 방법의 응용을 확장시키면,
여러 처리 과정을 단순화 시킬 수 있는 좋은 포인트입니다.


```
A, B, C = df_weather1[['outlier_dbscan', 'outlier_isof', 'outlier_lof']].sum()
A, B, C
```

Out[158]: (210, 1108, 83)

단계 5-7

outlier_dbscan, outlier_isof 그리고 outlier_lof 모두가 True인 행의 개수는 D이다.

In [159...

```
'''
코드는 `df_weather1` 데이터프레임에서 이상치를 탐지하는 세 가지 방법
(`DBSCAN`, `IsolationForest`, `LocalOutlierFactor`)의 결과를 요약하여 계산합니다.

### 동작 설명

1. 각 이상치 탐지 방법의 결과 합계 계산:
- `df_weather1[['outlier_dbscan', 'outlier_isof', 'outlier_lof']].sum()`
- `df_weather1` 데이터프레임의 `outlier_dbscan`, `outlier_isof`, `outlier_lof` 열을 선택합니다.
- 각 열의 합계를 계산합니다.
- `outlier_dbscan`: `DBSCAN`으로 탐지된 이상치 수
- `outlier_isof`: `IsolationForest`로 탐지된 이상치 수
- `outlier_lof`: `LocalOutlierFactor`로 탐지된 이상치 수
- 결과는 세 개의 값 (A, B, C)으로 저장됩니다.

2. 모든 방법에 의해 이상치로 판별된 데이터 포인트 수 계산:
- `df_weather1[['outlier_dbscan', 'outlier_isof', 'outlier_lof']].all(axis=1).sum()`
- `df_weather1` 데이터프레임의 `outlier_dbscan`, `outlier_isof`, `outlier_lof` 열을 선택합니다.
- `all(axis=1)`: 각 행에 대해 모든 열의 값이 `True`인 경우에만 `True`를 반환합니다.
- 즉, `outlier_dbscan`, `outlier_isof`, `outlier_lof` 세 가지 방법 모두에서 이상치로 판별된 데이터 포인트를 찾습니다.
- 그런 다음, `True` 값의 수를 합산합니다.
- 결과는 D에 저장됩니다.

### 요약
- `A, B, C`는 각각 `DBSCAN`, `IsolationForest`, `LocalOutlierFactor` 방법으로 탐지된 이상치의 수를 나타냅니다.
- `D`는 세 가지 방법 모두에서 이상치로 판별된 데이터 포인트의 수를 나타냅니다.

코드를 통해 각 이상치 탐지 방법의 결과를 비교하고, 공통으로 이상치로 탐지된 데이터 포인트의 수를 확인할 수 있습니다.
'''
```

```
D = df_weather1[['outlier_dbscan', 'outlier_isof', 'outlier_lof']].all(axis=1).sum()
```

```
In [160]: A, B, C, D, A + B + C + D
```

```
Out[160]: (210, 1108, 83, 16, 1417)
```

A + B + C + D를 적는다.

1417

```
In [166]: Image('1-5.png')
```

Out[166]: **문제 6**

(kaggle 형 문제) prob4 를 바탕으로 아래 데이터셋을 만든다.

. prob6_train: DateHour 변수 기준으로 2021년 8월 14일 전(8월 14일 미포함) 데이터 (행의 수: 5256개)

. prob6_test: DateHour 변수 기준으로 2021년 8월 14일 이후(8월 14일 포함) 데이터 (행의 수: 744 개)

일 때, prob6_train으로 target을 예측하는 모델을 만들어,

prob6_test의 target에 대한 MAE를 최소화하는 모델을 만든다.

prob6_test의 예측 결과를 아래와 같은 형식으로 출력한다. 파일명은 answer6.csv 이다.

DateHour	TotalHour
2021-08-14 00:00:00	102.607580
2021-08-14 01:00:00	94.078890
....	

※ 실제 상황에 가까운 연습을 위해 prob6_test의 target 값은 모른다는 상황을 가정하고 문제를 푼다.

_kaggle 노트에서 이어 갑니다.

In []: